VOLTA: MULTI-TERRAIN HUMANOID LOCOMOTION VIA MODEL PREDICTIVE CONTROL (MPC) AND WHOLE-BODY CONTROL (WBC)

MR. TANAWAT PAWANTA

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF MASTER OF ENGINEERING (ROBOTICS AND AUTOMATION)
INSTITUTE OF FIELD ROBOTICS
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2025

VOLTA: Multi-Terrain Humanoid Locomotion via Model Predictive Control (MPC) and Whole-Body Control (WBC)

Mr. Tanawat Pawanta B. Eng.(Robotics and Automation Engineering)

A Thesis Submitted in Partial Fulfillment of the Requirements for
the Degree of Master of Engineering
Institute of Field Robotics
King Mongkut's University of Technology Thonburi
2025

..... (Assoc. Prof. Supachai Vongbunyong, Ph.D.)	Chairman of Thesis Committee
..... (Asst. Prof. Eakkachai Pengwang, Ph.D.)	Member and Thesis Advisor
..... (Assoc. Prof. Ronnapee Chaichaowarat, Ph.D.)	Member
..... (Asst. Prof. Suriya Natsupakpong, Ph.D.)	Member
..... (Asst. Prof. Thavida Maneewarn, Ph.D.)	Member

Copyright reserved

Thesis Title	VOLTA: Multi-Terrain Humanoid Locomotion via Model Predictive Control (MPC) and Whole-Body Control (WBC)
Thesis Credits	12
Candidate	Mr. Tanawat Pawanta
Thesis Advisor	Asst. Prof. Dr. Eakkachai Pengwang
Program	Master of Engineering
Field of Study	Robotics and Automation
Faculty	Institute of Field Robotics
Academic Year	2025

Abstract

Humanoid robots possess significant potential for real-world deployment in hazardous or uneven environments where traditional wheeled robots fail. However, maintaining stable and robust locomotion across mixed terrains remains a fundamental challenge due to complex dynamics and unpredictable surface properties. This thesis proposes VOLTA robot platform and a multi-terrain locomotion framework that integrates an online Intrinsically Stable Model Predictive Control (IS-MPC) gait planner with a passivity-based Whole-Body Control (WBC) gait regulator.

The gait planner utilizes IS-MPC to generate stable Center of Mass (CoM) and swing foot trajectories while satisfying Zero Moment Point (ZMP) stability and kinematic constraints. The WBC regulator translates these task-space trajectories into optimal joint torques.

The framework was evaluated through extensive multibody simulations in Drake and initial physical tests on the VOLTA torque-controlled humanoid platform. Simulation results demonstrate the robot's ability to perform omnidirectional walking—including straight, diagonal, circular, and L-shaped walking—on both rigid and compliant floors. Furthermore, the system successfully navigated unmapped multi-property and stepped terrain with varying heights. Physical experiments verified the accuracy of the robot's dynamic model through successful pure gravity compensation and wrench distribution tests. Despite current limitations in rejecting high-magnitude disturbances due to fixed footstep locations, the VOLTA framework establishes a robust foundation for autonomous humanoid locomotion in diverse environments

Keywords: Humanoid Locomotion/ Model Predictive Model(MPC)/ Whole-Body Control(WBC)

หัวข้อวิทยานิพนธ์	VOLTA: การพัฒนาระบบควบคุมการเดินของหุ่นยนต์ฮิวมานอยบนพื้นผิวหลายประเภทด้วย Model Predictive Control (MPC) และ Whole-Body Control (WBC)
หน่วยกิต	12
ผู้เขียน	นาย ธนวัฒน์ พาวันทา
อาจารย์ที่ปรึกษา	ผศ. ดร.เอกชัย เป็งวัง
หลักสูตร	วิศวกรรมศาสตรมหาบัณฑิต
สาขาวิชา	วิทยาการหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2568

บทคัดย่อ

หุ่นยนต์ฮิวแมนนอยด์มีศักยภาพอย่างมากในการนำไปใช้งานจริงในสภาพแวดล้อมที่เป็นอันตรายหรือขรุขระ ซึ่งหุ่นยนต์แบบมีล้อแบบดั้งเดิมไม่สามารถทำงานได้ อย่างไรก็ตาม การรักษาเสถียรภาพและความทนทานในการเคลื่อนที่บนพื้นผิวที่หลากหลายยังคงเป็นความท้าทายพื้นฐาน เนื่องจากพลศาสตร์ที่ซับซ้อนและคุณสมบัติของพื้นผิวที่ไม่สามารถคาดเดาได้ วิทยานิพนธ์ฉบับนี้นำเสนอหุ่นยนต์ VOLTA และกรอบการทำงานสำหรับการเคลื่อนที่บนพื้นผิวหลากหลายรูปแบบ โดยบูรณาการตัววางแผนการก้าวเดินแบบออนไลน์ด้วยวิธีการควบคุมแบบจำลองคาดการณ์ที่เสถียรในตัว (Intrinsically Stable Model Predictive Control: IS-MPC) เข้ากับตัวควบคุมการก้าวเดินแบบทั้งตัวที่อาศัยหลักการพาสซีฟ (Passivity-based Whole-Body Control: WBC)

ตัววางแผนการก้าวเดินใช้ IS-MPC ในการสร้างวิถีการเคลื่อนที่ของจุดศูนย์กลางมวล (Center of Mass: CoM) และเท้าที่แกว่ง (Swing foot) ที่มีเสถียรภาพ ในขณะเดียวกันก็สอดคล้องกับข้อจำกัดทางจลนศาสตร์และเสถียรภาพของจุดโมเมนต์เป็นศูนย์ (Zero Moment Point: ZMP) จากนั้นตัวควบคุม WBC จะแปลงวิถีการเคลื่อนที่ในปริภูมิงาน (Task-space trajectories) เหล่านี้ให้เป็นแรงบิดที่ข้อต่อ (Joint torques) ที่เหมาะสมที่สุด

กรอบการทำงานนี้ได้รับการประเมินผ่านการจำลองพลศาสตร์ระบบหลายวัตถุ (Multibody simulations) อย่างครอบคลุมในโปรแกรม Drake และการทดสอบทางกายภาพเบื้องต้นบนแพลตฟอร์มหุ่นยนต์ฮิวแมนนอยด์ VOLTA ที่ควบคุมด้วยแรงบิด ผลการจำลองแสดงให้เห็นถึงความสามารถของหุ่นยนต์ในการเดินแบบรอบทิศทาง ซึ่งรวมถึงการเดินเป็นเส้นตรง แนวทแยง วงกลม และรูปตัวแอล (L-shaped) ทั้งบนพื้นแข็งและพื้นที่มีความยืดหยุ่น นอกจากนี้ ระบบยังประสบความสำเร็จในการเคลื่อนที่ผ่านพื้นที่ที่ไม่ได้จัดทำแผนที่ไว้ล่วงหน้า (Unmapped) ซึ่งมีคุณสมบัติหลากหลายและเป็นขั้นบันไดที่มีความสูงแตกต่างกัน การทดลองทางกายภาพได้ตรวจสอบความถูกต้องของแบบจำลองพลศาสตร์ของหุ่นยนต์ผ่านความสำเร็จในการทดสอบการชดเชยแรงโน้มถ่วง (Pure gravity compensation) และการกระจายแรง (Wrench distribution) แม้ว่าปัจจุบันจะมีข้อจำกัดในการต้านทานสิ่งรบกวนขนาดใหญ่เนื่องจากตำแหน่งการวางเท้าที่ถูกกำหนดไว้ตายตัว แต่กรอบการทำงาน VOLTA นี้ได้สร้างรากฐานที่แข็งแกร่งสำหรับการเคลื่อนที่ของหุ่นยนต์ฮิวแมนนอยด์แบบอัตโนมัติในสภาพแวดล้อมที่หลากหลาย

คำสำคัญ: การควบคุมแบบจำลองคาดการณ์/ การควบคุมแบบทั้งตัว/ การเคลื่อนที่ของหุ่นยนต์ฮิวแมนนอยด์

ACKNOWLEDGEMENTS

This research was made possible through the support, guidance, and encouragement of many individuals and organizations who have contributed significantly to my academic journey.

First and foremost, I would like to express my deepest gratitude to my advisor and co-advisor, Asst. Prof. Eakkachai Pengwang and Dr. Kitti Thamrongaphichartkul. Their invaluable mentorship, constructive feedback, and unwavering support were instrumental in the successful completion of this research.

I also extend my sincere appreciation to the distinguished members of my thesis examination committee: Assoc. Prof. Ronnapee Chaichaowarat, Assoc. Prof. Supachai Vongbunyong, Asst. Prof. Suriya Natsupakpong, and Asst. Prof. Thavida Maneewarn. Their insightful comments and constructive suggestions substantially elevated the quality and rigor of this thesis.

A special note of thanks goes to Jirattanun Leeudomwong for his vital technical assistance in developing the force-sensing module.

I gratefully acknowledge the financial support provided by the National Science, Research and Innovation Fund (Fundamental Fund under Grant 69A306000039). Furthermore, I am deeply thankful to the Hospital Automation Research Center (HAC) at the Institute of Field Robotics (FIBO), King Mongkut's University of Technology Thonburi, for providing the essential hardware and laboratory facilities required for the development of the VOLTA platform.

Finally, I am profoundly thankful to my family, friends, and colleagues at the Institute of Field Robotics. Their constant encouragement, patience, and support throughout my graduate studies have been my greatest source of strength.

CONTENTS

	PAGE
ENGLISH ABSTRACT	**ii**
THAI ABSTRACT	**iii**
ACKNOWLEDGEMENTS	**iv**
CONTENTS	**v**
LIST OF TABLES	**vii**
LIST OF FIGURES	**ix**
LIST OF SYMBOLS	**x**
1. INTRODUCTION	**1**
1.1 Background and Motivation	1
1.2 Objectives	2
1.3 Scopes	2
LIST OF PUBLICATIONS	**1**
CHAPTER	
2. LITERATURE REVIEW	**3**
2.1 Related Works	3
2.1.1 Intrinsically Stable MPC (IS-MPC)	3
2.1.2 Nonlinear MPC	3
2.1.3 Cascaded MPC Approach	4
2.1.4 Passivity-based Whole-body Control	5
2.1.5 Research Gap	5
2.2 Background Theories	5
2.2.1 Basic Keywords of Humanoid Walking	6
2.2.2 Dynamics of Humanoid robots	7
2.2.3 Model Predictive Control (MPC)	10
2.2.4 Whole-Body Control (WBC)	14
3. METHODOLOGY	**15**
3.1 Humanoid Robot Platform	15
3.2 Drake : Model-Based Design and Verification for Robotics	19
3.3 System Overview: Simulation	21
3.4 Gait Planning	21
3.4.1 Offline Gait Planning	21
3.4.2 Online Gait Plannig	24
3.5 Gait Regulation	33
3.5.1 Robot Dynamics	33
3.5.2 Controller Formulation	33
3.6 State Estimator	37
3.6.1 Forward Kinematics	37
3.6.2 ZMP Calculation	37

CONTENTS (CONT.)

	PAGE
3.6.3 Kalman Filter	38
3.7 System Overview: Physical Robot	39
4. EXPERIMENT AND RESULT	**40**
4.1 Experiments in Simulation	40
4.1.1 Experiment 1: State Estimator	40
4.1.2 Experiment 2: Whole-Body Control	43
4.1.3 Experiment 3: Footsteps Planner	48
4.1.4 Experiment 4: Integrate with Gait planner	50
4.2 Experiments in Physical Robot	66
4.2.1 Experiment 5: Pure Gravity compensation	66
5. DISCUSSION AND CONCLUSION	**68**
5.1 Discussion	68
5.1.1 Current Limitations and Implementation Challenges	68
5.2 Conclusion	69
5.2.1 Future Work	69
REFERENCES	**71**
CURRICULUM VITAE	**73**

LIST OF TABLES

TABLE		PAGE
3.1	Table of Properties	15
3.2	Link length	16
3.3	Joint Information	16
3.4	Feature comparison of Drake, Gazebo, and Pinocchio	19
4.1	Contact Parameter Mapping for Simulation Surfaces	40
4.2	Kalman filter parameters and value	41
4.3	Controller Gain and Values	43
4.4	Footsteps and Geometric Parameters	48
4.5	MPC parameters and values	50
4.6	Floor properties and result	55
4.7	Material Properties by Tile Color	62

LIST OF FIGURES

FIGURE **PAGE**

2.1 Randomly-generated elliptic staircase environment 4
2.2 Overview of Cascaded MPC 4
2.3 Passivity-based Whole-body Control 5
2.4 Support polygon(area within the red line) during DS and SS phase 6
2.5 Zero Moment Point(ZMP) 6
2.6 Capture region(set of all possible capture points) with N refer to the number of steps require to reach the capture point 7
2.7 The floating-base coordinate system 8
2.8 LIP in the x direction 9
2.9 Model Predictive Control 10
2.10 Control Horizon and Prediction (Preview) Horizon 11
2.11 classifications of MPC 11
2.12 Overview of IS-MPC 12
2.13 Example output from the candidate footstep generation: L walk (top), diagonal walk (bottom). 12
2.14 Overview of IS-MPC for uneven terrain(2 states IS-MPC). 13
3.1 Isometric View and Real Robot Platform 15
3.2 Link length 16
3.3 Joints' name and home configuration of robot 17
3.4 Axis of rotation of each joints 18
3.5 Wiring Diagram 18
3.6 Example Diagram 19
3.7 Robot collision(red) and inertia(blue) Visualization with Drake 20
3.8 Command joint position with slider bar on the right. 20
3.9 System overview in simulation 21
3.10 LIP in the x direction 22
3.11 Symmetric and asymmetric CoM trajectory 22
3.12 CoM planning with LIP 24
3.13 Swing leg trajectory 24
3.14 Kinematics constraint 25
3.15 Diagonal walking's footsteps 26
3.16 Support polygon when robot is in single stance phase 28
3.17 Moving constraint when robot is in double stance phase 28
3.18 Prediction result of MPC 31
3.19 Robot's contact points 35
3.20 Loadcells on physical robot's sole 37
3.21 Contact point of robot's foot 38
3.22 Physical robot's system overview 39
4.1 Filtered CoM position 41
4.2 Filtered ZMP position 42
4.3 Filtered CoM velocity 42
4.4 External forces applied to base frame. 43

LIST OF FIGURES (CONT.)

FIGURE **PAGE**

4.5 CoM Stabilization 44
4.6 External load estimation 45
4.7 CoM trajectory tracking 46
4.8 Effect from orientation controller while tracking trajectory 46
4.9 Contact forces distribution 47
4.10 Comparison of vertical ground reaction forces (F_z) with and without the proposed force pre-calculation. 47
4.11 6 Differences footsteps 48
4.12 Combined footsteps 49
4.13 Straight walk on rigid floor 51
4.14 Diagonal walk on rigid floor 51
4.15 Circular walk on rigid floor 52
4.16 L-shape walk on rigid floor 52
4.17 Forward/Backward walk on rigid floor 53
4.18 Combined walk on rigid floor 53
4.19 CoM position tracking (Combined walk) 54
4.20 CoM orientation tracking (Combined walk) 54
4.21 Snapshot from simulation when MPC solver fail in Experiment 5.2c 55
4.22 Stiffness effect to stance leg position 56
4.23 Dissipation effect to stance leg velocity 56
4.24 Effect of stance leg orientation controller 57
4.25 CoM orientation tracking comparison 58
4.26 CoM position tracking comparison 59
4.27 Left foot orienation tracking comparison 60
4.28 Left foot position tracking comparison 61
4.29 The robot walking on muti-properties floor, which the difference tile colors represent diference properties 62
4.30 Snapshot when walking on muti-properties floor. 63
4.31 CoM, ZMP and reference velocity tracking 63
4.32 The robot walking on step floor, which the difference tile's color represent diference heigh (the maximum height difference is 1 cm) 64
4.33 Snapshot when walking on step floor. 64
4.34 CoM, ZMP and reference velocity tracking 65
4.35 CoM trajectory when an external force is applied. The dashed line is original trajectory if no external forces allpied. 65
4.36 ZMP trajectory when an external force is applied. The dashed line is original trajectory if no external forces allpied. 66
4.37 Pure gravity compensation effect in physical robot 66
4.38 Wrench distribution in physical robot 67

LIST OF SYMBOLS

p_{ZMP}	Position of the Zero Moment Point (ZMP).
p_{CoM}	Position of the Center of Mass (CoM).
p_{CP}	Position of the Capture point.
q	Set of generalized coordinates for the robot.
p_b, ϕ_b	Position and orientation of the body reference frame relative to the world frame.
q_j	Row vector of actuated joint positions.
$M(q)$	Mass (inertia) matrix.
$H(q, \dot{q})$	Coriolis and gravitational term.
B	Actuation matrix.
u	Control input vector.
J	Jacobian of the robot.
λ	Wrench applied to the robot.
m	Total mass of the robot.
L	Angular momentum of the CoM.
g	Gravitational acceleration.
z_{CoM}	Constant height of the CoM.
T_c	Natural time constant of the Linear Inverted Pendulum.
η	Natural frequency parameter defined as $\sqrt{g/z_{CoM}}$.
x_f^*, y_f^*, θ_f^*	List of footstep poses (position and orientation) with respect to the world frame.
T_s	Step timing for footstep sequence.
J	Jacobian matrix
$\vec{W}_{grf}$	Ground reaction forces (contact forces) subject to optimization.
$\vec{W}_{imp,c}$	CoM impedance wrench.
$\vec{W}_{imp,sw}$	Swing leg impedance wrench.
τ	Vector of actuated joint torques.
K_p, K_d	Positive diagonal stiffness (proportional) and damping (derivative) matrices for translation.
K_r, D_r	Positive diagonal stiffness and damping matrices for rotational orientation.

CHAPTER 1 INTRODUCTION

1.1 Background and Motivation

Humanoids are classified as one type of legged robots, which have an advantage over wheeled robots in terms of mobility, particularly in accessing environments with uneven and discontinuous surfaces [1]. For this reason, there has been increasing effort to develop and implement humanoids in real-world scenarios, as seen in the DARPA Robotics Challenge (DRC), which aimed to test the ability of humanoids to perform tasks in hazardous situations, such as natural disasters or incidents dangerous to humans [2]. Additionally, research has demonstrated the use of humanoids in real-world testing of equipment designed for humans, providing data such as torque at each joint, which is easier to collect compared to human subjects [3]. Furthermore, humanoids have been used to study human behavior [4].

One of the current challenges of humanoid robots is real-world deployment. In real-world environments, besides external factors that are uncontrollable compared to simulations, there are also various and mixed terrain conditions, especially in the case of natural disasters or general natural surfaces. To enable humanoids to overcome environmental limitations, gait planning that responds to the environment and ensures stability and robustness of walking is essential [5][6].

To achieve this goal, Model Predictive Control (MPC) theory is a suitable choice as MPC is use an online optimization method that allows the robot to adjust its walking pattern or re-plan based on the current surface conditions and the current state of the robot, as well as manage external disturbances. Moreover, MPC can handle the high complexity of legged robot dynamics, which cannot be solved using an optimal control law in a general form [1][7].

This work proposes a locomotion framework featuring an online gait planner based on Model Predictive Control (MPC). This approach enables the humanoid robot to walk with both stability and robustness across mixed terrains.

1.2 Objectives

This work aim to design a locomotion framework for humanoid robot to enable humanoid robots to walk reliably across diverse terrains.

1.3 Scopes

1. The robot will operate on flat(full contact) and nonslip surfaces.
2. The control system will focus exclusively on the lower body.
3. This work will not focus on the design of high-level planners, such as obstacle avoidance and localization task.
4. The robot will either follow a predefined path or accept user inputs to control its linear velocity (x, y) and angular velocity (z).
5. The framework will be implemented on torque-controlled humanoid robot platform.

CHAPTER 2 LITERATURE REVIEW

2.1 Related Works

In this section, the content will cover research related to humanoid walking.Based on several studies, the walking structure of humanoids can be divided into two components: motion planning, which involves planning the foot pose and Center of Mass (CoM) trajectory for each step, and motion regulation, which controls each joint to ensure the robot can track the planned motion.

In the past 10 years, Model Predictive Control (MPC) has gained prominence in humanoid walking, serving as an online trajectory optimization tool for motion planning, and in [8], MPC has been applied to the motion regulation part of humanoids, with the following details.

2.1.1 Intrinsically Stable MPC (IS-MPC)

In 2016, Scianca et al. [9] proposed the use of MPC for motion planning in humanoid walking on flat, rigid surfaces by using a Linear Inverted Pendulum (LIP) as the prediction model. Later, in 2018 [10], the approach was extended to walking on piecewise-horizontal ground, resembling steps, which led to the use of a Variable Height Inverted Pendulum (VH-IP) as the prediction model instead. Kinematic Control (based on Inverse Kinematics) was employed to track the CoM trajectory. Both works were simulations with the NAO robot, where MPC computations were done at 20 Hz.

In 2020 [11], the IS-MPC framework for flat-ground walking was applied to a real-world humanoid capable of receiving velocity commands from a user. This was done with the HRP-4 robot, which was controlled using joint velocities derived from the kinematic control that transformed the CoM trajectory from the MPC into joint velocities. MPC computations were carried out at 200 Hz on an external PC, with data sent to the robot via TCP/IP.

In 2022 [12], the framework for walking on piecewise-horizontal ground was further developed, enabling the robot to run on inclined surfaces. Walking and running simulations were conducted with the HRP-4, while real-world walking tests were performed with the OP3 robot, with computations running at 40 Hz on an INTEL NUC with an Intel Core i3-7100U. Before testing, a 3D map was prepared. In the same year [13], a method was proposed to handle non-convex constraints, which arise in the case of obstacle avoidance.

2.1.2 Nonlinear MPC

The IS-MPC framework primarily utilizes Linear MPC (LMPC), which imposes limitations on the selection of the prediction model and the formulation of constraints. In [14], the use of Nonlinear MPC (NMPC) was proposed, computed at 30 Hz. This approach employed a Floating Base Inverted Pendulum (FIP) as the prediction model and used constrained LQR for motion regulation. The work was demonstrated through simulations using the HRP-4 robot, where the robot walked on a randomly-generated elliptic staircase environment.

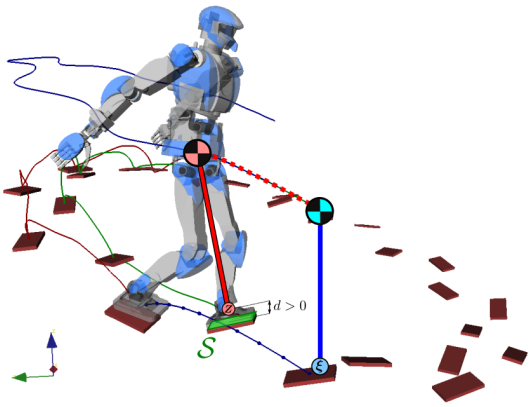

Figure 2.1 Randomly-generated elliptic staircase environment

2.1.3 Cascaded MPC Approach

In addition to the previously mentioned approaches,[15] have utilized MPC in different structures. In 2022 , a cascaded MPC approach was proposed, which combined both LMPC and NMPC. LMPC, based on the Variable Height Inverted Pendulum (VH-IP), operated at 100 Hz and was responsible for regulating angular momentum. Meanwhile, NMPC, using an Inverted Pendulum with Flywheel model, operated at 10 Hz and calculated the CoM trajectory for walking on flat surfaces with varying heights. Additionally, a High Order Sliding Mode Observer (HOSMO) was employed as the state estimator. This approach was demonstrated through simulations with the COMAN robot and real-world experiments with the Walker2 robot

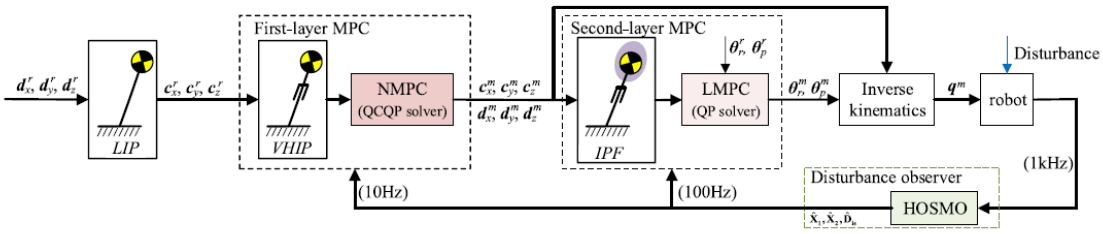

Figure 2.2 Overview of Cascaded MPC

2.1.4 Passivity-based Whole-body Control

In real-world environments, it is inevitable to encounter compliant surfaces (e.g., grass, rubber) and rough terrain (e.g., gravel, small rocks). To address the challenge of dynamic walking on compliant terrain, [16] proposed a Passive-based Whole-body Control (WBC) as a motion regulator that incorporates a stance foot stabilization controller to maintain stability for the stance foot on compliant terrain. This is necessary because, unlike rigid surfaces, contact feet on compliant terrain are not guaranteed to remain static (they may sink or rotate), which can lead to foot oscillations. The approach was tested in real-world scenarios, demonstrating that the TORO robot could walk on grass and a gym mattress.

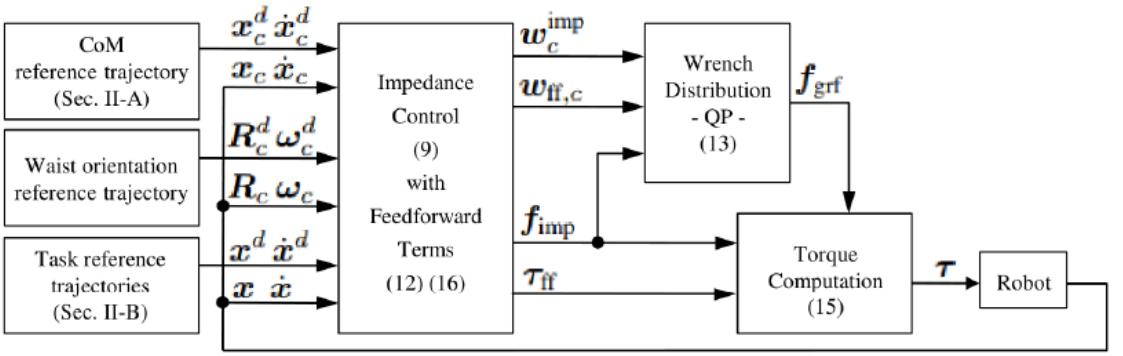

Figure 2.3 Passivity-based Whole-body Control

2.1.5 Research Gap

From the aforementioned works, using NMPC requires a high computational resource, and furthermore, employing the Cascaded MPC approach necessitates the computation of both LMPC and NMPC. Thus, these two approaches are unsuitable for on-board robot processors and typically require external computation, with the results then transmitted to the robot's processor. In many studies, Inverse Kinematics is used to transform the motion planning output into joint states, which introduces the risk of singularities, leading to a loss of Degrees of Freedom (DOF) in the robot's pose. Additionally, many of these works control the robot through joint position/velocity rather than torque control, which offers advantages in terms of interaction with the environment, especially in cases of multi-contact locomotion. Moreover, none of the works presented walking on non-rigid surfaces such as compliant (grass, rubber) or rough (gravel, small rocks) terrains.

Although [16] introduced WBC for motion regulation to address terrain challenges, its motion planning capabilities were limited. The system only planned for straight-line walking and did not integrate user inputs. The planning was restricted to setting the step length and the duration of a single gait cycle, without more advanced interaction with user inputs.

2.2 Background Theories

This section will first explain several terms that are frequently used in research on humanoid walking, and then it will go over the dynamics model that is used to determine the robot's state. The application of Model Predictive Control (MPC) to motion planning

will next be covered, and the last section will cover motion regulation with Whole-Body Control (WBC).

2.2.1 Basic Keywords of Humanoid Walking

1. Gait Cycle and Support Polygon
   In humanoid walking, the gait can be divided into two phases: Single Support Phase (SS), where the robot has only one foot in contact with the ground (Stance Foot) while the other foot (Swing Foot) is moving toward the next position, and Double Support Phase (DS), where both feet are in contact with the ground for a short duration before the Swing-Stance roles switch, and the robot enters the next SS phase. In human walking, the ratio between these phases is typically 20 % for DS and 80 % for SS. The contact of the feet with the ground also affects the determination of the support polygon (the convex hull of the contact points) at any given moment [5][6].

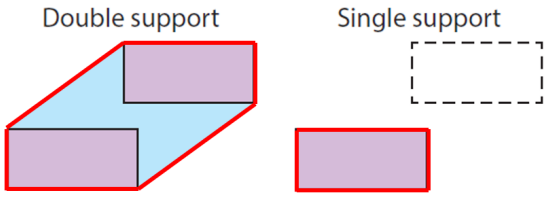

Figure 2.4 Support polygon(area within the red line) during DS and SS phase

2. Dynamics Walking and Zero Moment Point(ZMP)

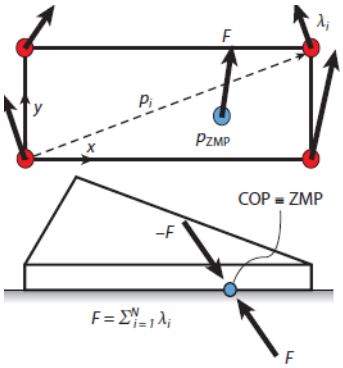

Figure 2.5 Zero Moment Point(ZMP)

When the robot's feet make contact with a flat ground or coplanar surface, the Zero Moment Point (ZMP) or Center of Pressure (CoP) refers to the point where the sum of the moments around it is zero. In other words, when multiple forces from the ground act on different points of the foot, the ZMP represents the point where the resultant force can be considered to act. This is the point that characterizes the overall force distribution on the foot [5][6].

$$p_{ZMP} = \frac{\sum_i p_i \times \lambda_i}{\sum_i \lambda_i} \tag{2.1}$$

where $p_{ZMP} = [x_{ZMP}, y_{ZMP}, z_{ZMP}]^T$ is the position of Zero Moment Point and p_i is the point that force λ_i was applied to robot's sole.

If a robot can walk while keeping the ZMP within the support polygon at all times, this represents dynamic walking, which closely resembles human walking. On the other hand, if the robot walks by keeping the Center of Mass (CoM) within the support polygon instead, this is referred to as static walking. Dynamic walking allows for more fluid, efficient movement, while static walking is more stable but less natural, as it requires the CoM to remain within the base of support at all times.

3. Capture point

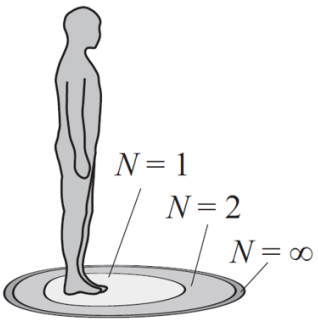

Figure 2.6 Capture region(set of all possible capture points) with N refer to the number of steps require to reach the capture point

The Capture Point is the point on the xy-plane where, if the robot steps to that location, the Center of Mass (CoM) will come to a complete stop. If the robot's feet can be controlled to move and step onto the specified Capture Point, as determined by the equations, the CoM will eventually converge to that point and come to rest [17].

$$p_{CP} = p_{CoM} + \sqrt{\frac{z_{CoM}}{g}}\dot{p}_{CoM} \tag{2.2}$$

where p_{CP} is the position of capture point and $p_{CoM} = [x_{CoM}, y_{CoM}, z_{CoM}]^T$ is the position of Center of Mass (CoM)

2.2.2 Dynamics of Humanoid robots

1. Full-body Dynamics
   - Configuration Space
     Let the robot contain n actuated joints and $\{R_b\}$ be a body reference frame, referenced from the world frame $\{R_0\}$, which serves as the inertial frame see Fig. 2.7. The position and orientation of $\{R_b\}$ relative to $\{R_0\}$ are denoted by $P_b \in \Re^3$ and $\phi_b \in SO(3)$ (in 3D space), respectively. Additionally, let $q_j \in \Re^n$ represent a row vector of actuated joints position Thus, the set of coordinates q can be defined as follows:
     $$q = [p_b^T, \phi_b^T, q_j]^T \tag{2.3}$$
   - Continuous Floating-base Dynamics
     When considering the robot as being composed of rigid linkages in a tree structure, the dynamics of the system can be expressed in Lagrangian form
     $$M(q)\ddot{q} + H(q, \dot{q}) = Bu + J(q)^T\lambda \tag{2.4}$$

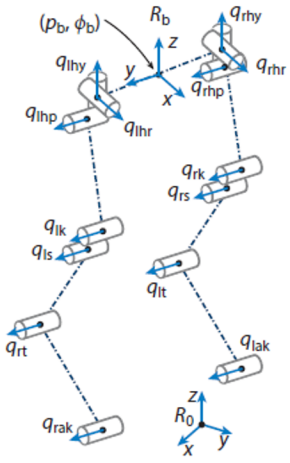

Figure 2.7 The floating-base coordinate system

where $M(q)$ is the mass(inertia) matrix, $H(q, \dot{q})$ contains the Coriolis and gravitational term, B is the actuation matrix,u is the control input,J is the Jacobian of the robot and wrench λ

2. Reduced-order Models [5][7]

   Full-body Dynamics approach captures the full dynamics of the robot, but computing the Full-order model involves calculating the dynamics of the entire system, which can be highly complex and result in excessive computational time. To address this limitation, a Reduced-order model is often proposed, simplifying the dynamics to reduce computational complexity.

   (a) Centroidal Dynamics

   Centroidal dynamics describes the dynamics of the Center of Mass (CoM) and the total angular momentum of the robot using the Newton equation and Euler equation, which can be expressed as follows:

$$m(\ddot{p}_{CoM} + g) = \sum_i f_i \tag{2.5}$$

where m is total mass of the robot, $p_{CoM} = [x_{CoM}, y_{CoM}, z_{CoM}]^T$ is the position of Center of Mass(CoM) and $g = [0, 0, -g]^T$.

$$\dot{L} = \sum_i (p_i - p_{CoM}) \times f_i \tag{2.6}$$

with L is the angular momentum of CoM, p_i is the point that force f_i was applided to robot.

When considering the robot making contact with flat ground (e.g., during walking or running) at a contact point p_i where the vertical coordinate of the contact $p_i^z = 0$ the Newton-Euler equations can be written as:

$$mp_{CoM} \times (\ddot{p}_{CoM} + g) + \dot{L} = \sum_i p_i \times f_i \tag{2.7}$$

From these Newton-Euler equations, the relationship between the Center of Mass

(CoM) and the Zero Moment Point (ZMP) can be derived as:

$$\frac{mp_{CoM} \times (\ddot{p}_{CoM} + g) + \dot{L}}{m(\ddot{p}^z_{CoM} + g^z)} = \frac{\sum_i p_i \times f_i}{\sum_i f^z_i} = p^{x,y}_{ZMP} \tag{2.8}$$

where $p_{ZMP} = [x_{ZMP}, y_{ZMP}, z_{ZMP}]^T$ is the position of Zero Moment point and superscript x,y refer to its x and y axes value.

(b) Invert Pendulum Models

The Inverted Pendulum Model views the entire robot as a 3D inverted pendulum, with the robot's CoM represented as a point mass at the tip, and the stance foot as the rod connecting the point mass and the pivot point.

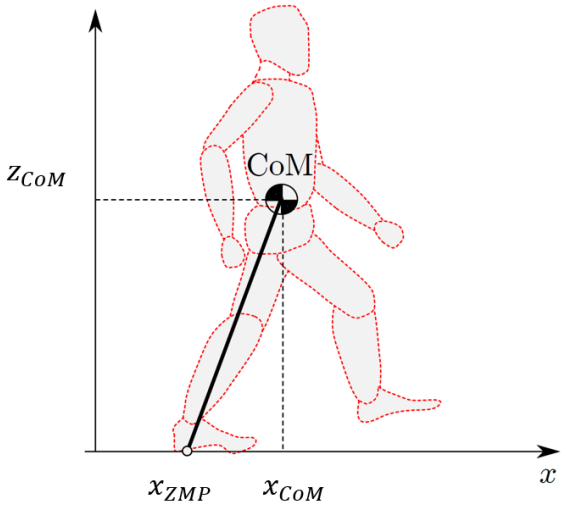

Figure 2.8 LIP in the x direction

In the case of the **Linear Inverted Pendulum (LIP)**, the equation of motion can be written in terms of the CoM and ZMP directly from centroidal dynamics by assuming a constant CoM height and conservation of angular momentum, or calculated directly from the free-body diagram of the inverted pendulum [18].

$$x_{ZMP} = x_{CoM} - \frac{z_{CoM}}{g}\ddot{x}_{CoM} \tag{2.9}$$

$$y_{ZMP} = y_{CoM} - \frac{z_{CoM}}{g}\ddot{y}_{CoM} \tag{2.10}$$

The result of assuming a constant CoM height is that the robot cannot adjust the height of its CoM according to the environment, which becomes a limitation when dealing with obstacles or uneven terrain. To address this issue, the **Variable Height Inverted Pendulum (VH-IP)** was utilized in [10].

$$\ddot{x}_{CoM} = \frac{\ddot{z}_{CoM} + g}{z_{CoM} - z_{ZMP}}(x_{CoM} - x_{ZMP}) \tag{2.11}$$

$$\ddot{y}_{CoM} = \frac{\ddot{z}_{CoM} + g}{z_{CoM} - z_{ZMP}}(y_{CoM} - y_{ZMP}) \tag{2.12}$$

$$\ddot{z}_{CoM} = \frac{f_z}{m} - g \tag{2.13}$$

where m is total mass of the robot and f_z is the vertical component of ground reaction force(GRF).

2.2.3 Model Predictive Control (MPC)

1. Concept of MPC [19]

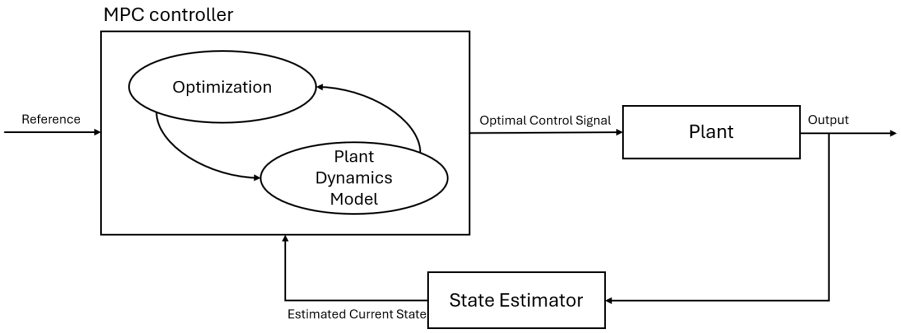

Figure 2.9 Model Predictive Control

The main concept of Model Predictive Control (MPC) is to use the predicted outcomes over the next k-timesteps from current timestep to inform the control policy for the current timestep by solving an online optimization problem.To construct an MPC controller, the following key components need to be defined:

- *Prediction (Plant) Model*: A dynamics model is required to calculate the future output based on the control signal chosen as the input to the model.
- *Cost Function and Constraints*: The cost function is a function that needs to be optimized (minimized) by adjusting decision variables. Constraints (or conditions) such as velocity/torque limits and workspace boundaries help restrict the selection of decision variables.
- *Sampling Time*: This is the duration of each timestep. It must be chosen to account for the time required to compute the MPC at each round.
- *Control Horizon*: This defines how many future timesteps the control signal will be applied to the plant in each optimization process.
- *Prediction (Preview) Horizon*: This defines how many future timesteps the predicted outcomes are considered in each optimization cycle.Note that prediction horizon does not require to equal Control Horizon

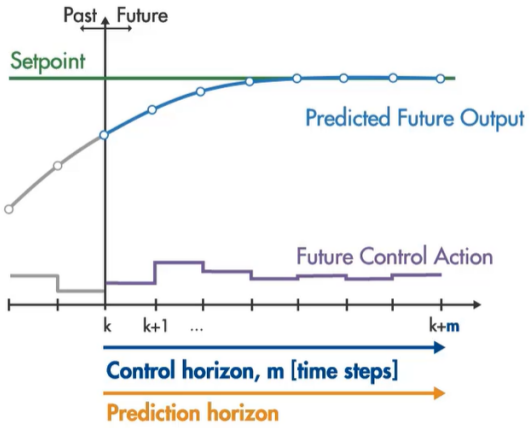

Figure 2.10 Control Horizon and Prediction (Preview) Horizon

2. classifications of MPC [19]

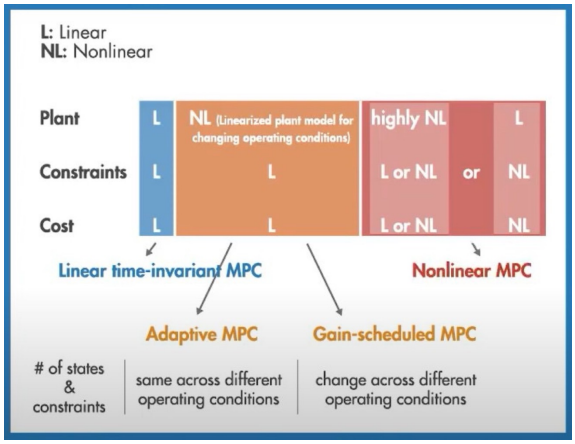

Figure 2.11 classifications of MPC

- *Linear MPC* is used when the prediction model, cost function, and constraints exhibit linearity. It can be further categorized by the approach used to handle nonlinear systems, which can be linearized into a linear system(see Fig. 2.11).
- *Nonlinear MPC* is applied when the prediction model, cost function, or constraints involve nonlinearities.

3. Use Case in Gait Planning (IS-MPC)

   In this section, we will focus on the **IS-MPC** from [11], specifically on the steps leading up to the kinematic controller, assuming that the input for the MPC has already been prepared. Control horizon is C timesteps and prediction horizon is P timesteps, with a sampling time (δ) of 0.01 s/timestep, the linear MPC for gait planning in this work is formulated as follows:

   The candidate footstep generation calculates the candidate position and orientation of the foot placement for the next F steps ($\hat{x}_f^k, \hat{y}_f^k, \phi_f^k$) at timestep k, as well as the time duration for each step, based on the input received from the user (see Fig. 2.13). This information is then passed to the IS-MPC, which adjusts the position of the foot placement in the next step and generates a stable CoM trajectory.

   Prediction model:

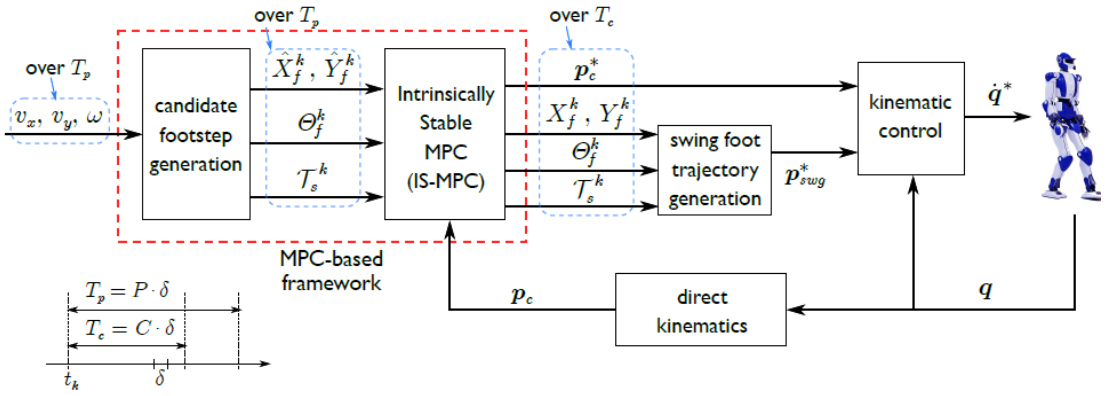

Figure 2.12 Overview of IS-MPC

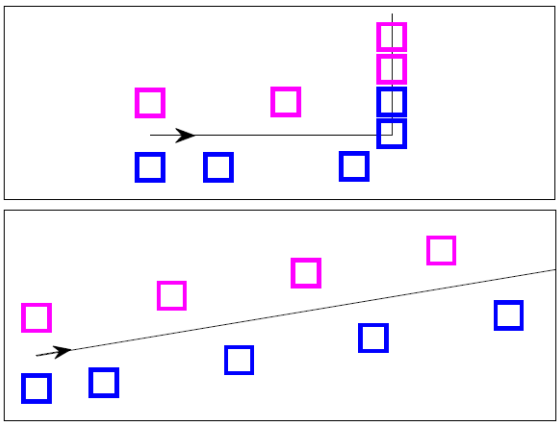

Figure 2.13 Example output from the candidate footstep generation: L walk (top), diagonal walk (bottom).

This work utilizes the Linear Inverted Pendulum (LIP) as the prediction model. From the equations 3.1 and 3.2, the model can be reformulated as follows:

$$\ddot{p}_{CoM} = \eta^2(p_{CoM} - p_{ZMP}) \tag{2.14}$$

where $\eta = \sqrt{g/z_{CoM}}$, g is gravity acceleration and z_{CoM} is the constant height of the CoM.
By defining the velocity of the ZMP as the input to the system, the LTI state-space equation can be written as:

$$\begin{bmatrix} \dot{p}_{CoM} \\ \ddot{p}_{CoM} \\ \dot{p}_{ZMP} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \eta^2 & 0 & -\eta^2 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_{CoM} \\ \dot{p}_{CoM} \\ p_{ZMP} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \dot{p}_{ZMP} \tag{2.15}$$

Constrains:

- *ZMP Constraints*: This constraint ensures that the walking is dynamic by enforcing that the ZMP stays within the support polygon at all times.
- *Kinematics Constraints*: These constraints prevent the output from exceeding the robot's kinematic limits, as illustrated in the figure below.
- *Stability Constraint*: This constraint prevents the CoM trajectory from diverging

beyond the defined boundaries by applying the concept of the capture point to maintain stability.

MPC formulation:

Define the stack of each decision variables as vectors:

$$\begin{aligned} \dot{X}^k_{ZMP} &= (\dot{x}^k_{ZMP} \quad \dots \quad \dot{x}^{k+C-1}_{ZMP})^T \\ \dot{Y}^k_{ZMP} &= (\dot{y}^k_{ZMP} \quad \dots \quad \dot{y}^{k+C-1}_{ZMP})^T \\ X^k_f &= (x^1_f \quad \dots \quad x^{F'}_f)^T \\ Y^k_f &= (y^1_f \quad \dots \quad y^{F'}_f)^T \end{aligned}$$

with F' refer to the last footstep within the control horizon and superscript refer to at timestep k.then optimization problem can then be formulated as:

$$\min_{\dot{X}^k_{zmp},\dot{Y}^k_{zmp},X^k_f,Y^k_f} \left(\|\dot{x}^k_{ZMP}\|^2 + \|\dot{y}^k_{ZMP}\|^2 + \beta \left(\|X_f - \hat{X}_f\|^2 + \|Y_f - \hat{Y}_f\|^2 \right) \right) \quad (2.16)$$

subject to:

- ZMP Constraints
- Kinematics Constraints
- Stability Constraint

MPC Interation:

At each timestep, after obtaining the candidate footstep, the following steps are performed:

(a) Solve the optimization problem 2.16 to obtain $(\dot{X}^k_{ZMP}, \dot{Y}^k_{ZMP}, X^k_f, Y^k_f)$.

(b) from solution, extract the first control sample$(\dot{x}^k_{ZMP}, \dot{y}^k_{ZMP})$.

(c) Solve 2.15 for $x_{CoM}(t)$ for $t \in [t_k, t_{k+1}]$ with $\dot{x}_{ZMP} = \dot{x}^k_{ZMP}$.compute $y_{CoM}(t)$ similarly.

(d) Return the 3D CoM trajectory $p^*_c = (x_{CoM}, y_{CoM}, z_{CoM})$

(e) Return the first footstep in actual footstep sequence(X^k_f, Y^k_f, ϕ^k_f) with unmodified timing T^k_s for generate swing foot trajectory p^*_{swg}.

IS-MPC was further developed to enable planning in environments with step-like terrain in [12], where the prediction model was changed to the Variable Height Inverted Pendulum (VH-IP). Additionally, the optimization problem was extended to account for the height of the CoM (z_c), as shown in the figure.

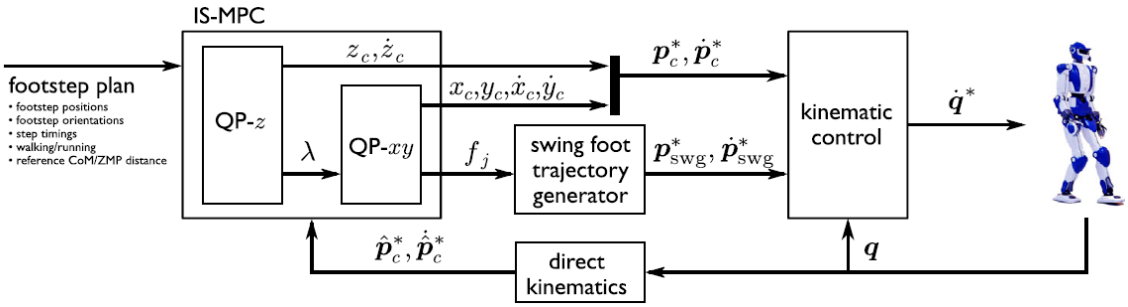

Figure 2.14 Overview of IS-MPC for uneven terrain(2 states IS-MPC).

2.2.4 Whole-Body Control (WBC)

Whole-Body Control (WBC) [20] is a sensor-based feedback control method that takes advantage of the robot's full-body motion, allowing it to perform multiple tasks simultaneously. For example, when a humanoid robot is carrying an object and an external obstacle approaches, WBC enables the robot to avoid the obstacle while maintaining its stability and ensuring that the object it carries doesn't fall. Similarly, when walking on compliant surfaces, WBC helps the robot not only track reference trajectories but also stabilize the stance foot that is in contact with the ground.

WBC can be categorized based on the type of output generated:

1. Velocity-Based WBC

   In this approach, the system determines a set of k stable control actions represented as velocities in the task space $(\dot{x}_1, \dot{x}_2, \ldots, \dot{x}_k)$ for each individual task e.g. $\dot{x}_i = k_i(x_i^d - x_i)$. The result is the joint velocities of the robot($\dot{q}$), computed through:

   $$\dot{q} = J_1^{-1}\dot{x}_1 + J_2^{-1}\dot{x}_2 + \ldots + J_k^{-1}\dot{x}_k \tag{2.17}$$

   where J_i is the kinematic Jacobian that map $\dot{q}$ to $\dot{x}_i$

2. Torque-Based WBC

   Similar to the velocity-based approach, this method determines a set of k stable control actions but represented as forces in the task space $(F_1, F_2, \ldots, F_k)$ for each task e.g. $F_i = k_{pi}(x_i^d - x_i) + k_{di}(-\dot{x}_i)$. The result is the joint efforts (τ), computed through:

   $$\tau = J_1^T F_1 + J_2^T F_2 + \ldots + J_k^T F_k \tag{2.18}$$

   Additionally, WBC allows for task prioritization by assigning weights to each task

   WBC can be classified into three different cases:

   - case 1 Hierarchical WBC: In this approach, the priority depends on the sequence of tasks. The controller gives precedence to the task that comes first and ensures its successful completion before moving on to the next task. This means the task with higher order in the sequence is guaranteed to be completed with greater certainty than later tasks.
   - case 2 Weighted WBC: In contrast, this method assigns task importance based on the weights given to each task. Each task influences the final outcome proportionally to its assigned weight. This means a later task with a higher weight has a better chance of being completed successfully than an earlier task with a lower weight.
   - case 3 Hybrid Hierarchical/Weighted WBC: This combines the two previous approaches. Initially, the first task is given priority and must be completed first. After that, the remaining tasks are prioritized based on their assigned weights.

CHAPTER 3 METHODOLOGY

3.1 Humanoid Robot Platform

1. 3D Model in Solidwork

Figure 3.1 Isometric View and Real Robot Platform

Table 3.1 Table of Properties

Property Name	Values	Unit
Mass*	6	kg
Height*	73	cm
Total Joints**	12	Joints
Actuate Joints**	12	Joints

*Measure from SolidWorks
**Include left leg 6 joints and right leg 6 joints.

2. Robot's links

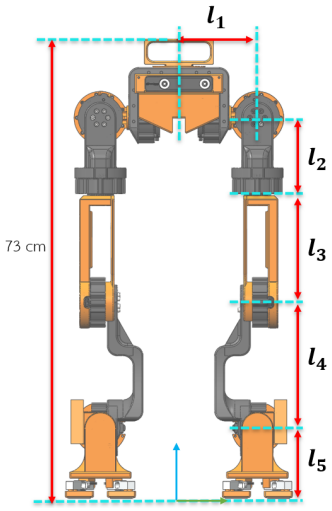

Figure 3.2 Link length

Table 3.2 Link length

Section	Link length	Value* (mm)
Leg	l_1	125
	l_2	124
	l_3	170
	l_4	200
	l_5	112

*measure from Solidwork

3. Robot's joints

Fig. 3.3 illustrates the joint locations on the robot and their naming conventions. The naming format follows the same method as the link names. For example, the left leg knee joint is named JL_knee, where "JL" indicates a joint on the left side of the robot and "knee" specifies the joint type. All joints are revolute joints whose axes of rotation are shown in Figure 3.4, with each joint driven by a single motor. Additional information, such as joint limits, motor brand/IDs, and full joint names, can be found in Table 3.3.

Table 3.3 Joint Information

Session	Motor ID	Motor brand	Joint name	Mechanical Limit (degree)		
				Minimum	Home	Maximum
Left Leg	01	CyberGear	JL_hip_y	-180	0	+180
	02	CyberGear	JL_hip_r	-30	0	+20
	03	CyberGear	JL_hip_p	-130	0	+30
	04	CyberGear	JL_knee	-100	0	+80
	05	CyberGear	JL_ankle_p	-100	0	+80
	06	CyberGear	JL_ankle_r	-90	0	+90
Right Leg	11	CyberGear	JR_hip_y	-180	0	+180
	12	CyberGear	JR_hip_r	-30	0	+20
	13	CyberGear	JR_hip_p	-130	0	+30
	14	CyberGear	JR_knee	-100	0	+80
	15	CyberGear	JR_ankle_p	-100	0	+80
	16	CyberGear	JR_ankle_r	-90	0	+90

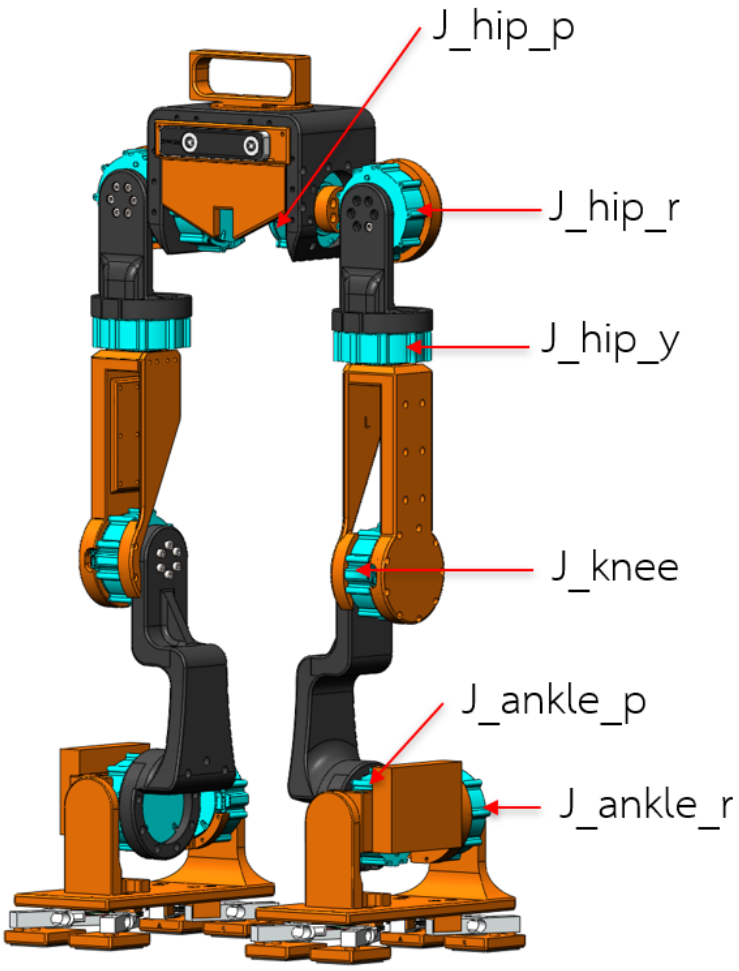

Figure 3.3 Joints' name and home configuration of robot

4. Wiring diagram

   The following diagram in Fig. 3.5 illustrates the wiring configuration for the Volta humanoid's lower body, showing how CyberGear motor are connected to the power supply and computation unit via the CAN adapter. In this work we are using 3 motors/ 1 can adapter due to limited hardware space. For others feedback data we have T265 tracking camera for base pose and twist and loadcells attract on the robot's sole to measure contact forces.

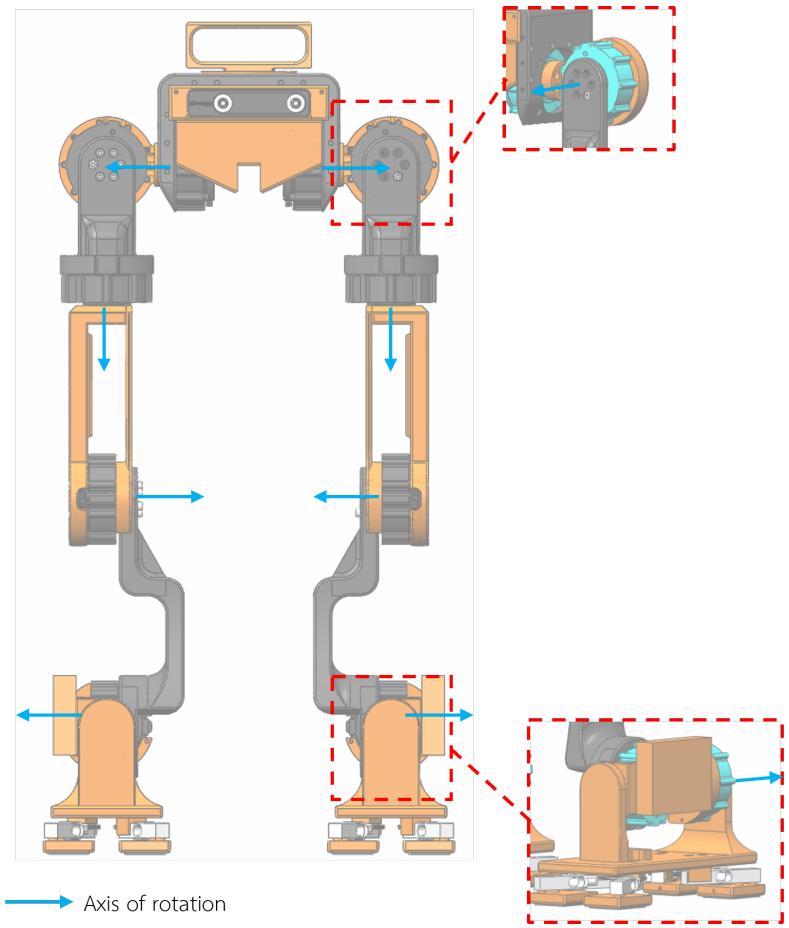

Figure 3.4 Axis of rotation of each joints

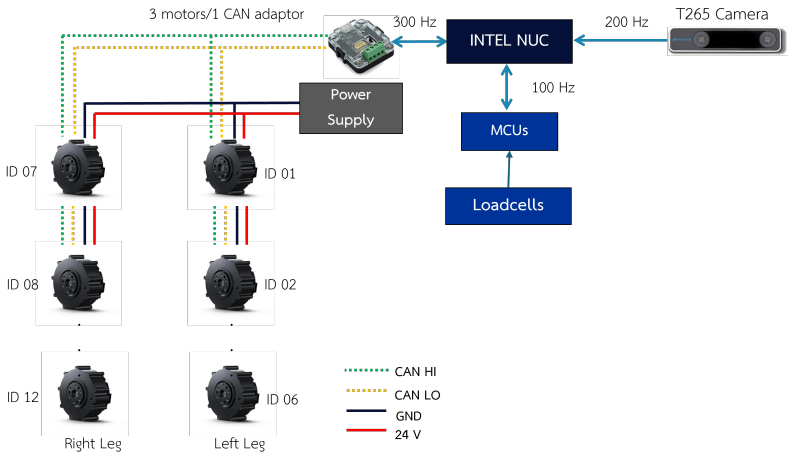

Figure 3.5 Wiring Diagram

3.2 Drake : Model-Based Design and Verification for Robotics

Drake [21] ("dragon" in Middle English) is a C++ toolbox started by the Robot Locomotion Group at the MIT Computer Science and Artificial Intelligence Lab (CSAIL). The development team has now grown significantly, with core development led by the Toyota Research Institute. It is a collection of tools for analyzing the dynamics of our robots and building control systems for them, with a heavy emphasis on optimization-based design/analysis. In this project, the Python API of Drake, known as *pydrake*, is utilized to simplify the implementation and learning process.

Table 3.4 Feature comparison of Drake, Gazebo, and Pinocchio

Features/Softwares	Drake	Gazebo	Pinocchio
Visualization	x	external	x
Physic engin	x	x	external
Multibody Kinematic/Dynamics	x	external	x
Optimization Tools	x	external	external

1. **Concept in drake**

   Drake employs a block diagram approach to construct a *diagram* by connecting multiple *systems* or *diagrams*, as illustrated in Fig. 3.6. Each system is equipped with input and output ports for data exchange. Additionally, users can define custom systems in Drake, referred to as leaf systems, which can publish data similarly to publishers in ROS.

   Another key concept in Drake is the *context*. Each system and diagram maintains its own context, which provides access to essential data such as the system's state, time, fixed parameters, or input values. These data elements can be accessed and managed through the respective system's context.

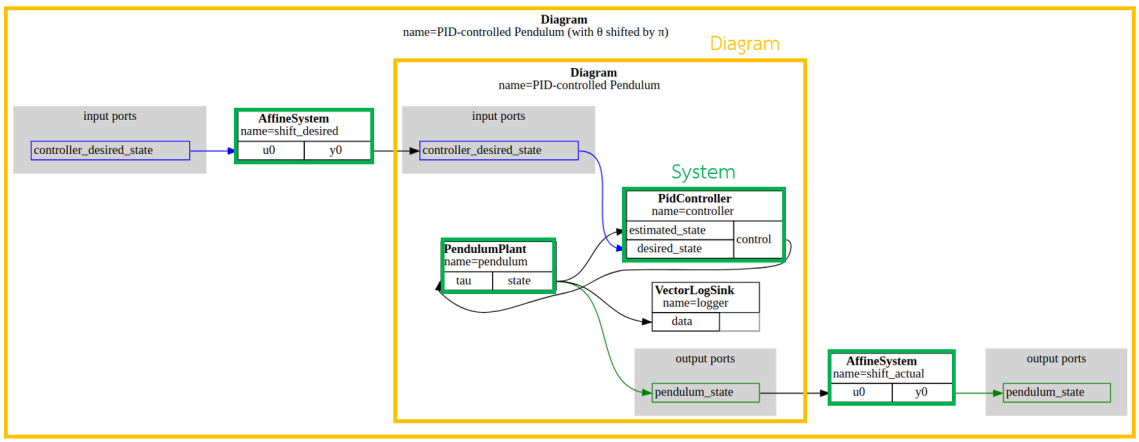

Figure 3.6 Example Diagram

2. **Visualization**

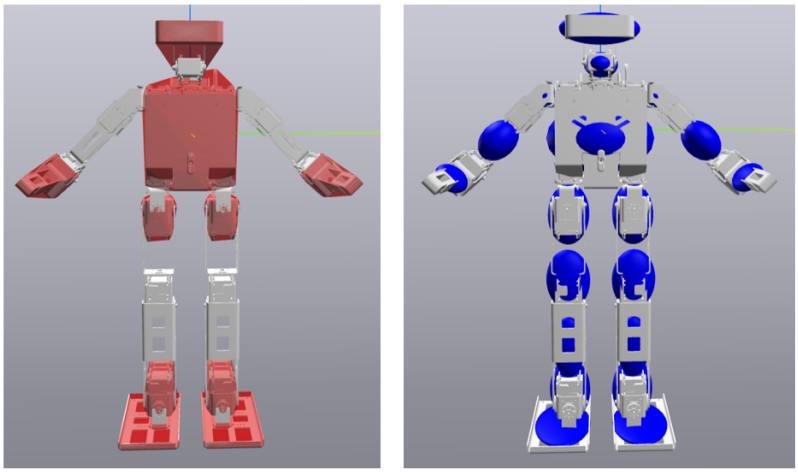

Figure 3.7 Robot collision(red) and inertia(blue) Visualization with Drake

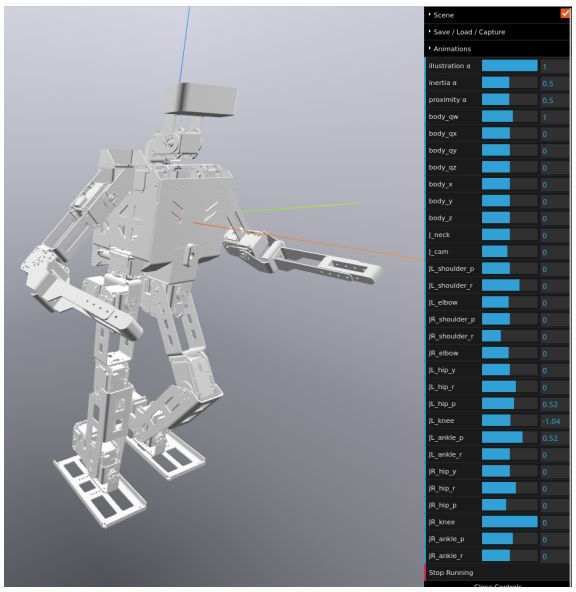

Figure 3.8 Command joint position with slider bar on the right.

Drake provides functionality for visualizing robot models and joint interfaces, similar to the capabilities of Rviz. This is achieved by importing the robot's URDF or SDFormat file into Drake. The visualization is rendered using Meshcat, a WebGL-based 3D visualization framework. Furthermore, the visualization can be shared across multiple users connected to the same network, facilitating collaborative analysis.

3.3 System Overview: Simulation

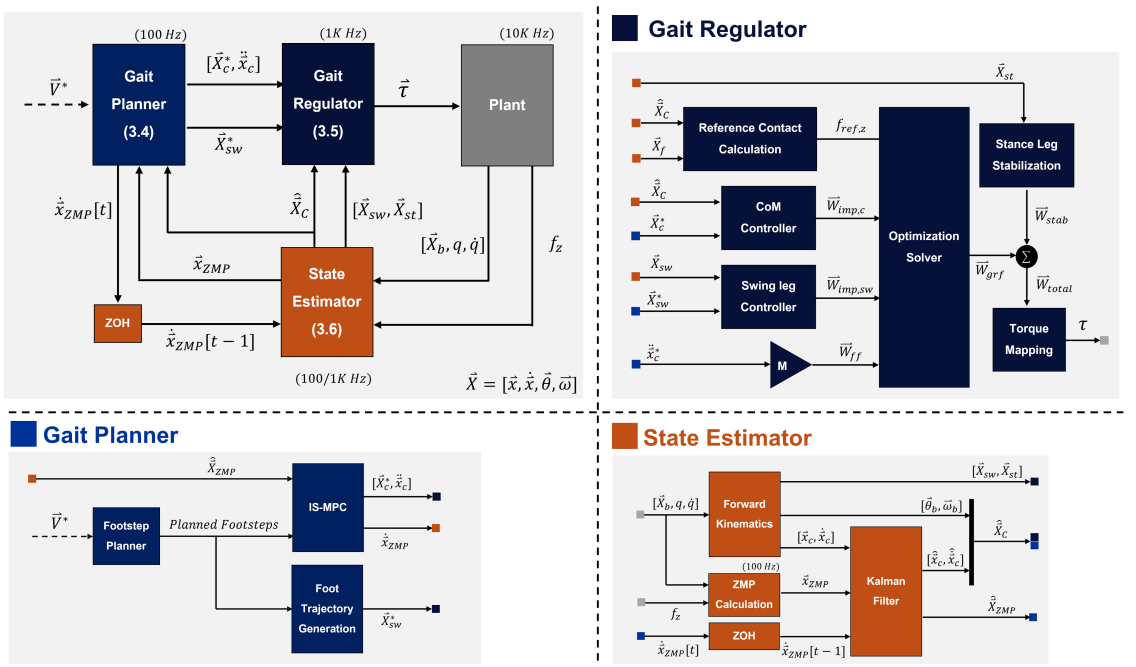

Figure 3.9 System overview in simulation

The proposed framework comprises three primary components: gait planner and the gait regulator. For the gait planning phase, the strategy employs IS-MPC [11] to generate task space trajectories. These include the center of mass trajectory $[\vec{X}_c^*, \ddot{\vec{x}}_c]$ and the swing foot trajectory $(\vec{X}_{sw}^*)$, which correspond to the desired robot velocity (V^*). Subsequently, passivity-based Whole-body control [16] will be utilized for task space tracking. Lastly, the state estimator will be used for estimate CoM position and velocity and also ZMP position.

In simulation each system were implement at ideal sampling rate, 100 Hz for gait planner and ZMP calculation, 1 kHz for state estimator and gait regulator. The plant is updated at 10 kHz.

3.4 Gait Planning

3.4.1 Offline Gait Planning

- Center of Mass Trajectory

  1. LIP dynamics

$$\ddot{x}_{CoM} = \frac{g}{z_{CoM}} x_{CoM} \tag{3.1}$$

$$\ddot{y}_{CoM} = \frac{g}{z_{CoM}} y_{CoM} \tag{3.2}$$

where $x_{CoM}, y_{CoM}, z_{CoM}$ is the position of Center of Mass (CoM) and g is gravitational acceleration.

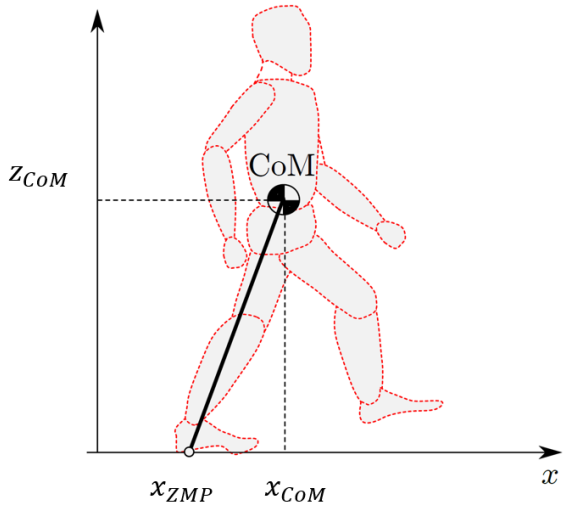

Figure 3.10 LIP in the x direction

From equation 3.1 and 3.2 can observe that LIP is a linear system so it can write in state space equation like

$$\begin{bmatrix} \dot{x}_{CoM} \\ \ddot{x}_{CoM} \\ \dot{y}_{CoM} \\ \ddot{y}_{CoM} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{g}{z_{CoM}} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{g}{z_{CoM}} & 0 \end{bmatrix} \begin{bmatrix} x_{CoM} \\ \dot{x}_{CoM} \\ y_{CoM} \\ \dot{y}_{CoM} \end{bmatrix} \tag{3.3}$$

The results of LIP dynamics can be categorized into two behaviors: a symmetric trajectory used in CoM planning and an asymmetric trajectory that depends on the initial states of the LIP dynamics.

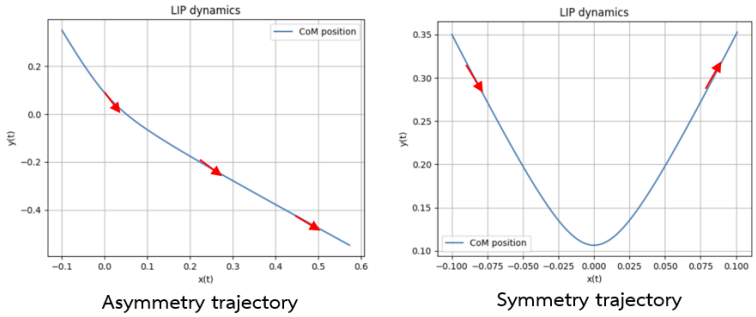

Figure 3.11 Symmetric and asymmetric CoM trajectory

2. Finding initial state
   To achieve a symmetric trajectory, we can determine the initial state $(x_0, \dot{x}_0, y_0, \dot{y}_0)$ and final time (t_f) using the orbital energy, which remains conserved along the trajectory, along with the time-domain solutions of equations 3.1 and 3.2.
   The natural time constant of the Linear Inverted Pendulum (LIP) is given by:

$$T_c = \sqrt{\frac{z_c}{g}} \tag{3.4}$$

   The initial position and offset of the CoM:

$$x_{\text{mid}} = 0, \quad x_0 = -\frac{\text{step length}}{2}$$

   The total energy of the system :

$$E = \frac{1}{2}\dot{x}^2 - \frac{g}{2z_c}x^2 \tag{3.5}$$

   The energy at mid point in trajectory when $\dot{x}_{mid} \in R^+$ is from user define value :

$$E = \frac{1}{2}\dot{x}_{mid}^2 - \frac{g}{2z_c}x_{mid}^2 \tag{3.6}$$

   Solving for the initial velocity in the x direction :

$$\dot{x}_0 = \sqrt{2E + \frac{g}{z_c}x_0^2} \tag{3.7}$$

   The step duration :

$$T_f = 2T_c \sinh^{-1}\left(\frac{\text{step length}/2}{T_c\dot{x}_{\text{mid}}}\right) \tag{3.8}$$

$$\dot{x}_{\text{mid}} = \frac{\text{step length}}{2 \cdot T_c \cdot \sinh\left(\frac{T_f}{2T_c}\right)} \tag{3.9}$$

   from equation 3.9 we can use step duration as input instead of $\dot{x}_{mid}$
   The initial velocity in the y direction:

$$\dot{y}_0 = -\frac{y_0}{T_c}\frac{\sinh(T_f/2T_c)}{\cosh(T_f/2T_c)} \tag{3.10}$$

3. Result of CoM planning

   The swing leg will start swing when CoM position is in the area of stance leg to ensure that robot don't fall.

- Swing Leg Trajectory
  To generate the swing leg trajectory in this process, we use a cubic (3^{rd} order polynomial) trajectory with zero start and end velocities. The required inputs for generating the trajectory include the swing leg placement position at the end of the trajectory, the swing duration, and the swing height.

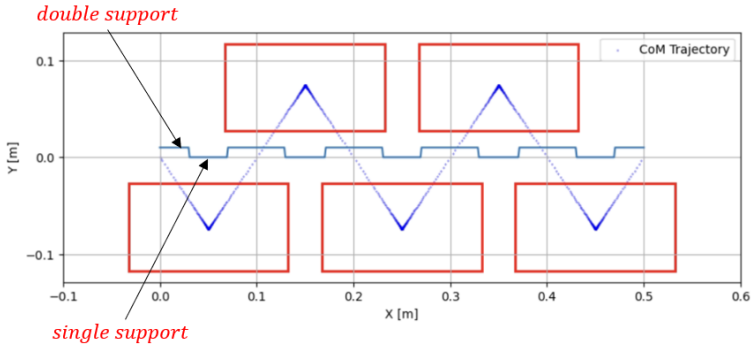

Figure 3.12 CoM planning with LIP

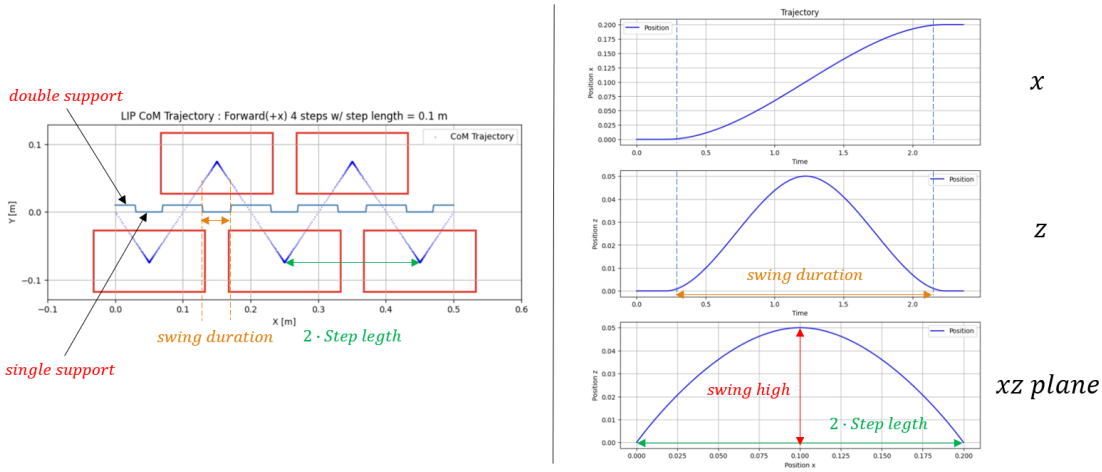

Figure 3.13 Swing leg trajectory

3.4.2 Online Gait Plannig

1. Footstep Planner

   In this module, the reference velocity input, defined as a function $\vec{v}^*(t) = [v_x, v_y, \omega]$ with respect to the base frame of the robot, alongside the interval T, is used to generate the footsteps $\vec{p}_f = [x_f^*, y_f^*, \theta_f^*, T_s]$, encompassing both footstep pose and timing. To reduce computational overhead during the simulation, this module is simplified by employing a fixed step duration t_d and computing the footsteps offline using Quadratic Programming (QP).

$$
\begin{aligned}
x_f^* &= [x_f^0, x_f^1, ..., x_f^F] \\
y_f^* &= [y_f^0, y_f^1, ..., y_f^F] \\
\theta_f^* &= [\theta_f^0, \theta_f^1, ..., \theta_f^F] \\
T_s &= [t_s^0, t_s^1, ..., t_s^F]
\end{aligned}
$$

   Where x_f^*, y_f^* and θ_f^* is list of footsteps pose w.r.t. world frame W.

   The process for generating the footstep sequence consists of three steps. First, after defining the velocity input, the number of footsteps F and the step timing T_s are computed as

$$F = \frac{T}{t_d} \tag{3.11}$$

$$T_s = [0, t_d, 2t_d, ..., Ft_d] \tag{3.12}$$

Then, The first QP problem (3.13) is solved to determine foot orientation:

$$\begin{aligned} \min_{\Theta_f^*} & \sum_{j=1}^{F} \left(\theta_f^j - \theta_f^{j-1} - \int_{t_s^{j-1}}^{t_s^j} \omega(\tau) d\tau \right)^2 \\ \text{subject to:} \quad & |\theta_f^j - \theta_f^{j-1}| \leq \theta_{\max} \end{aligned} \tag{3.13}$$

Where θ_{max} is the maximum permissible foot rotation.
Lastly, Lastly, the second QP problem (3.15) is solved for footstep placement.

- Kinematics constraint

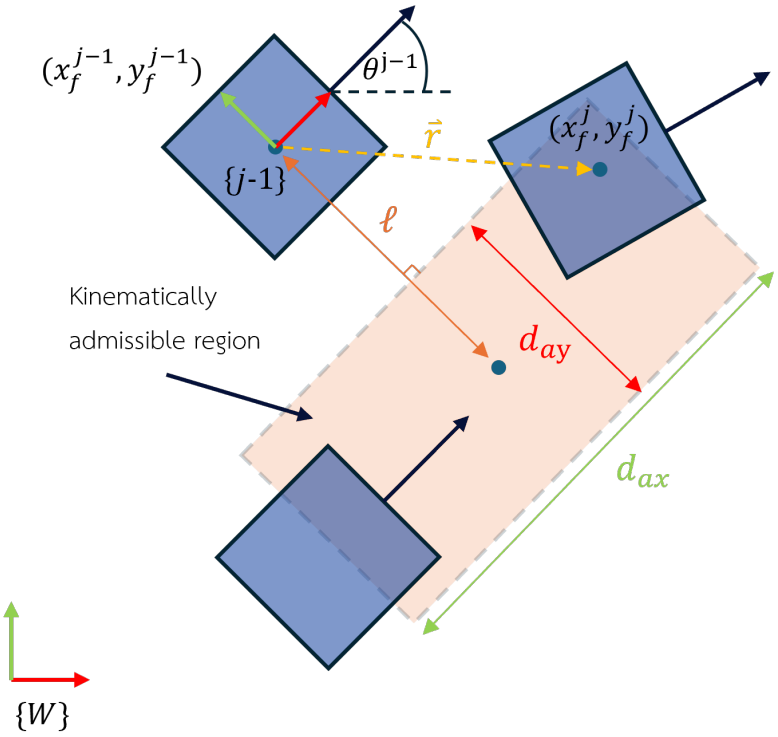

Figure 3.14 Kinematics constraint

Kinematic constraint is enforced to ensure that footstep placement remains feasible within the robot's kinematic limits. For the j^{th} footstep, the kinematic constraint is represented by the orange rectangular region illustrated in Figure 3.14. This area defines the feasible region reachable by the robot without violating joint limits. The kinematic constraint can be expressed as

$$R_{j-1}^T \begin{pmatrix} x_f^j - x_f^{j-1} \\ y_f^j - y_f^{j-1} \end{pmatrix} \leq \pm \begin{pmatrix} 0 \\ \ell \end{pmatrix} + \frac{1}{2} \begin{pmatrix} d_{ax} \\ d_{ay} \end{pmatrix}, \tag{3.14}$$

The left-hand side of the aforementioned constraint is the position vector $\vec{r}$, which represents the relative position between the j^{th} and $(j-1)^{th}$ footsteps, expressed in the $(j-1)^{th}$ coordinate frame. The right-hand side defines the admissible

region, where the sign of the nominal offset l depends on the designated stance foot. While the parameters d_{ax} and d_{ay} can be derived via forward kinematics or direct measurement from the physical robot, this work utilizes the latter approach.

Once the kinematic constraints are established, the second QP problem for footstep placement is formulated as:

$$\min_{x_f^*, y_f^*} \sum_{j=1}^{F} (x_f^j - x_f^{j-1} - \Delta x^j)^2 + (y_f^j - y_f^{j-1} - \Delta y^j)^2 \tag{3.15}$$

subject to: Kinematics constrain (3.14)

The displacement variables Δx^j and Δy^j is given by:

$$\begin{pmatrix} \Delta x^j \\ \Delta y^j \end{pmatrix} = \int_{t_s^{j-1}}^{t_s^j} R_\theta \begin{pmatrix} v_x(\tau) \\ v_y(\tau) \end{pmatrix} d\tau \pm R_j \begin{pmatrix} 0 \\ \ell/2 \end{pmatrix} \tag{3.16}$$

Where R_θ, R_j are rotation matrix of $\theta(\tau)$, foot orientation at a specific time τ, and θ_j, foot orientation at footstep j^{th}, and l is the distance between 2 feet. The sign $\pm$ is defined as what we use in kinematics constraint.

The result of footstep planner when we want the robot to walk in diagonal direction, command v_x and v_y for 10 seconds, was shown in Figure 3.15

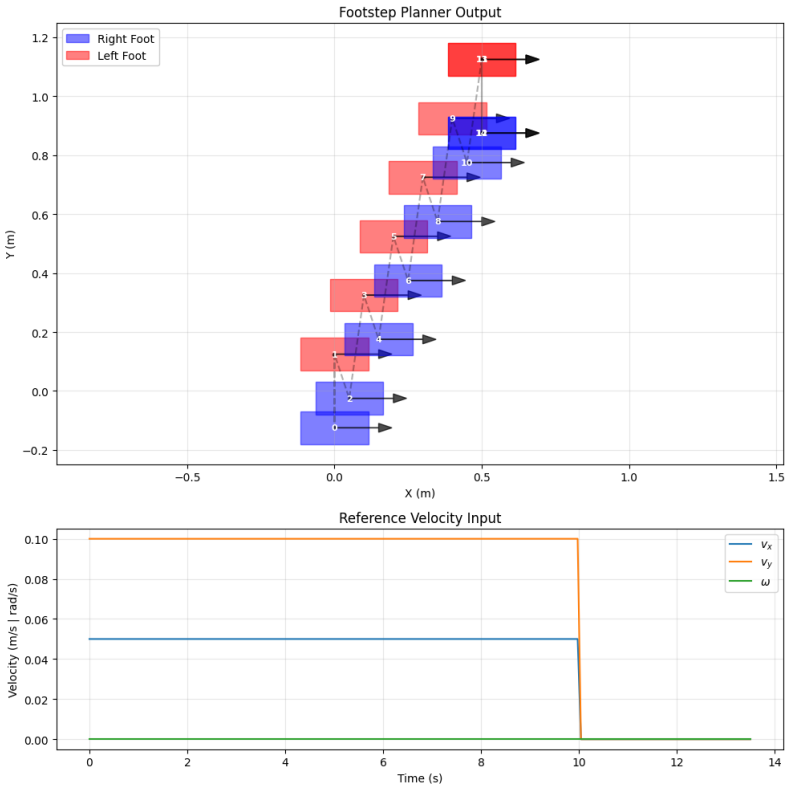

Figure 3.15 Diagonal walking's footsteps

2. Prediction Model and Constraints

- Extended Linear Inverse Pendulum
  Considered the dynamics of a linear inverse pendulum in x-axis, as shown in Figure 3.10. The robot is represented as a point mass at the Center of Mass (CoM), with the stance leg acting as a massless link. The CoM-ZMP relationship is defined as:

$$\begin{aligned} \ddot{x}_c &= \eta^2(x_c - x_z), \\ \eta &= \sqrt{\frac{g}{z_c}} \end{aligned} \tag{3.17}$$

  Where $\ddot{x}_c$, x_c and z_c are the Center of Mass acceleration, position and height; x_z is ZMP position, and g is gravitational acceleration.

$$\dot{\vec{X}} = \begin{bmatrix} \dot{x}_c \\ \ddot{x}_c \\ \dot{x}_z \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \eta^2 & 0 & -\eta^2 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ \dot{x}_c \\ x_z \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{x}_z \end{bmatrix} \tag{3.18}$$

  Equation 3.17 represents a linear system that can be formulated into the state-space representation shown in Equation 3.18. By defining the state vector as $\vec{X} = [x_c, \dot{x}_c, x_z]^T$, where $\dot{x}_z$ serves as the control input, the system dynamics are established. Because Equation 3.18 is defined in continuous time, Euler's method (Equation 3.19) is employed for discrete-time updates with a sampling period of $\delta t = 0.01$ s (100 Hz). This discretized model is subsequently utilized as the predictive model within the Model Predictive Control (MPC) framework.

$$\vec{X}[n+1] = \vec{X}[n] + \delta t \cdot \dot{\vec{X}}[n] \tag{3.19}$$

  Since the x and y axes in the standard Linear Inverted Pendulum (LIP) model are completely decoupled, the state equations for the y-axis maintain an identical structure. Consequently, the y-axis dynamics can be obtained by substituting the x variables with their corresponding y counterparts.
- ZMP Constraint
  The first constraint is the Zero Moment Point (ZMP) constraint, which ensures that the ZMP remains within the boundaries of the support polygon. To implement this, the geometry of the support polygon must first be defined. In this work, during the single-stance phase, the support polygon is modeled as a square with dimensions slightly smaller than the physical foot area of the robot. By centering this square at the origin of the contact foot, the orientation of the current footstep can be neglected, thereby simplifying the constraint formulation.

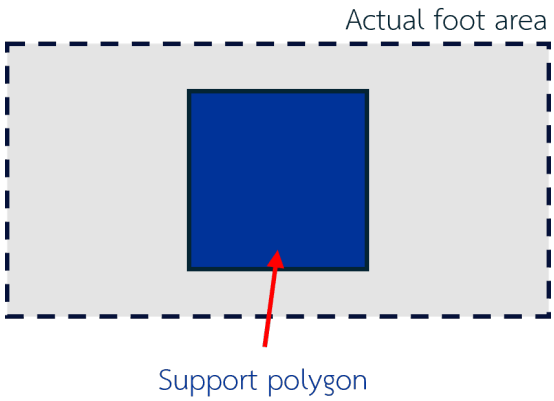

Figure 3.16 Support polygon when robot is in single stance phase

During the double-stance phase, a moving constraint is implemented via a box whose center, $\vec{p}_{mc}$, is generated by interpolating between the current j-th footstep position $\vec{p}^{\,j}$ and the subsequent footstep position $\vec{p}^{\,j+1}$ (Equation 3.20). This moving constraint configuration is illustrated in Figure 3.17.

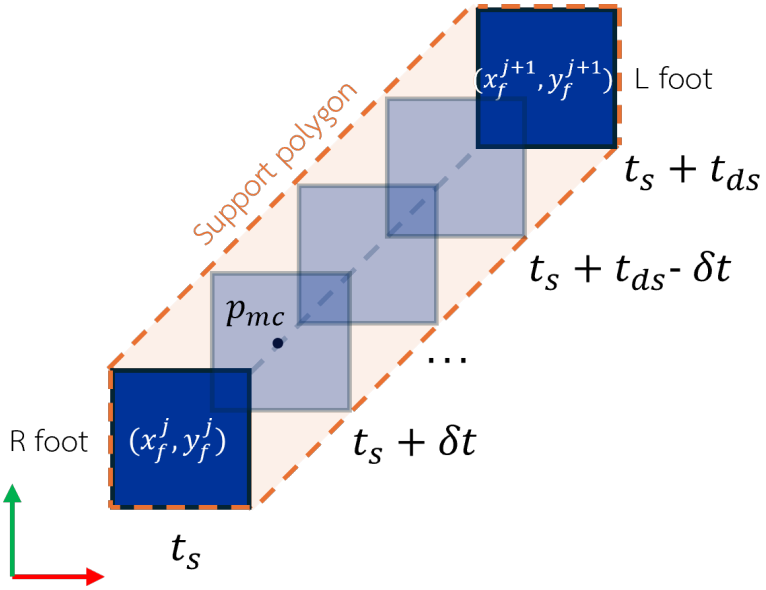

Figure 3.17 Moving constraint when robot is in double stance phase

$$\vec{p}_{mc}(t) = \begin{cases} \vec{p}^{\,j} & ,t \in [t_s^j, t_s^j + T_{ss}) \\ \vec{p}^{\,j} + \sigma(t)(\vec{p}^{\,j+1} - \vec{p}^{\,j}) & ,t \in [t_s^j + T_{ss}, t_s^{j+1}) \end{cases} \tag{3.20}$$

During the double-support phase, the interpolation parameter $\sigma(t) = \frac{t-(t_s^j+T_{ss})}{T_{ds}}$ is utilized, where T_{ss} and T_{ds} represent the durations of the single-stance and double-stance phases, respectively. Within this framework, $\vec{p}_{mc}$ originates at $\vec{p}^{\,j}$ and progresses toward $\vec{p}^{\,j+1}$, reaching the target position by the conclusion of the double-support phase.
The ZMP constraint can be expressed as:

$$-d/2 \leq \left(\vec{p}_z^{\,k+i} - \vec{p}_{mc}^{\,k+i}\right) \leq d/2 \tag{3.21}$$

Where $\vec{p}_z^{\,k+i}$ and $\vec{p}_{mc}^{\,k+i}$ is ZMP position and center of moving box at time $k+i$, $d = [d_x, d_y]$ is a size of the support polygon as we ention before our support polygon is defined as a square, so we can use the same value in both d_x and d_y.

- Stability Constraint
  While the ZMP constraint ensures that the Zero Moment Point remains within the support polygon—a necessary condition for dynamic walking—it does not guarantee that the LIP dynamics will remain stable, as the state may still diverge toward infinity. To prevent such instability, a stability constraint is proposed to ensure that the LIP dynamics remain bounded throughout the gait cycle.
  Prior to discussing this constraint, it is necessary to examine the LIP dynamics introduced in Equation 3.17. Attributed to the assumption of a constant Center of Mass (CoM) height, the system is linear and can be expressed in state-space form as:

$$\begin{bmatrix} \dot{x}_c \\ \ddot{x}_c \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ \eta^2 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ \dot{x}_c \end{bmatrix} + \begin{bmatrix} 0 \\ -\eta^2 \end{bmatrix} x_z \tag{3.22}$$

  Which is diagonalized as

$$\begin{bmatrix} \dot{x}_u \\ \dot{x}_s \end{bmatrix} = \begin{bmatrix} \eta & 0 \\ 0 & -\eta \end{bmatrix} \begin{bmatrix} x_u \\ x_s \end{bmatrix} + \begin{bmatrix} -\eta \\ \eta \end{bmatrix} x_z \tag{3.23}$$

  For any linear system, stability can be assessed by examining its eigenvalues. In the case of Equation 3.22, the eigenvalues are $\pm\eta$, indicating that the system possesses a saddle-point equilibrium. This implies that the system can be decoupled into unstable (x_u) and stable (x_s) subsystems. These subsystems are represented in modal coordinates $[x_u, x_s]^T$, as shown in Equation 3.23. The transformation between the original state coordinates $[x_c, \dot{x}_c]^T$ and the modal coordinates is expressed as:

$$x_u = x_c + \frac{\dot{x}_c}{\eta} \tag{3.24}$$

$$x_s = x_c - \frac{\dot{x}_c}{\eta} \tag{3.25}$$

  Where

$$x_c = \frac{1}{2}(x_u + x_s) \tag{3.26}$$

  From Equation 3.23 and 3.26 we can rewritten dynamics of unstable subsystem as

$$\dot{x}_u = \eta(x_u - x_z) \tag{3.27}$$

  Which its solution is

$$x_u(T) = e^{\eta T}(x_u(0) - \eta \int_0^T e^{-\eta\tau} x_z(\tau) d\tau) \tag{3.28}$$

  Based on Equation 3.28, in the absence of control intervention, the unstable component diverges toward infinity; consequently, this causes x_c to diverge as well. To address this, the stability constraint focuses on bounding the unstable subsystem to prevent such divergent behavior. To achieve this, following the methodology in [**initial condition of x_u**], the initial condition $x_u(0)$ was selected as:

$$x_u(0) = \eta \int_0^\infty e^{-\eta\tau} x_z(\tau) d\tau \tag{3.29}$$

At the end of control horizon we get

$$x_u(c) = \eta \int_{t_c}^\infty e^{-\eta(\tau - t_c)} x_z(\tau) d\tau \tag{3.30}$$

To formulate Equations 3.29 and 3.30 as constraints, it is necessary to discretize the expressions and rewrite them as a function of $\dot{x}_c$. Following the methodology detailed in [11], the discretized stability constraint is expressed as:

$$\sum_{i=0}^{C-1} e^{-i\eta\delta} \dot{x}_z^{k+i} = -\sum_{i=C}^{\infty} e^{-i\eta\delta} \dot{\tilde{x}}_z^{k+i} + \frac{\eta}{1 - e^{-\eta\delta}} (x_u^k - x_z^k), \tag{3.31}$$

And the terminal constraint is expressed as:

$$x_u^{k+C} = x_z^{k+C} + \frac{1 - e^{-\eta\delta}}{\eta} e^{C\eta\delta} \sum_{i=C}^{\infty} e^{-i\eta\delta} \dot{\tilde{x}}_z^{k+i}. \tag{3.32}$$

Where k is a current timestep, C is a number of timestep of control horizon and δ is sampling rate of MPC.

Equations 3.31 and 3.32 are referred to as the stability and terminal constraints, respectively. Both involve the summation of the ZMP velocity beyond the control horizon, $\tilde{x}_z^{k+i}$. Because these future values are unknown, specific assumptions must be established to solve the optimization problem. As detailed in [11], the term $\tilde{x}_z^{k+i}$, commonly referred to as the *tail*, can be modeled using three distinct assumptions: truncated, periodic, and anticipative. In this work, the periodic tail assumption is employed.

The assumption behind periodic tail is

$$\dot{\tilde{x}}_z^{k+i} = \dot{x}_z^{k+i-C} \quad , i = C, ..., 2C - 1 \tag{3.33}$$

The ZMP velocities $\dot{\tilde{x}}_z^{k+i}$ required for future steps are assumed to be identical to those calculated for the current steps. Under this assumption, the infinite summations in Equations 3.31 and 3.32 can be reformulated as:

$$\sum_{i=C}^{\infty} e^{-i\eta\delta} \dot{\tilde{x}}_z^{k+i} = \frac{e^{-C\eta\delta}}{1 - e^{-C\eta\delta}} \sum_{i=0}^{C-1} e^{-i\eta\delta} \dot{x}_z^{k+i} \tag{3.34}$$

Substituting Equation 3.34 into Equations 3.31 and 3.32, the stability constraints are expressed as:

$$\sum_{i=0}^{C-1} e^{-i\eta\delta} \dot{x}_z^{k+i} = \eta \frac{1 - e^{-C\eta\delta}}{1 - e^{-\eta\delta}} (x_u^k - x_z^k) \tag{3.35}$$

And the terminal constraint become

$$x_u^{k+C} - x_z^{k+C} = x_u^k - x_z^k \tag{3.36}$$

Which can be rewritten using Equation 3.27 as

$$\dot{x}_u^{k+C} = \dot{x}_u^k \tag{3.37}$$

The terminal constraint defined in Equation 3.37 is integrated into the MPC framework due to its inherent simplicity and the ease with which it can be implemented within the control software.

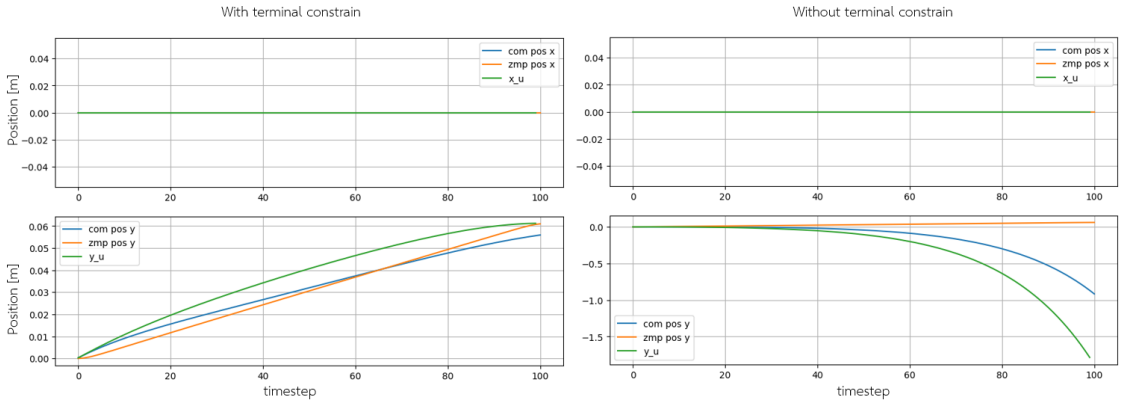

Figure 3.18 Prediction result of MPC

Figure 3.18 illustrates the effectiveness of the terminal constraint in preventing the divergence of y_c. As shown in the left panel, the application of the terminal constraint ensures that y_u remains bounded relative to the ZMP trajectory. Conversely, the right panel demonstrates that without this constraint, y_u diverges toward $-\infty$, subsequently causing y_c to diverge as well.

3. Optimization Problem Formulation

Let the state sequence $X = [X_0, \ldots, X_{C-1}]$, derived from Equation 3.18, and the ZMP velocity sequence $U = [U_0, \ldots, U_{C-1}]$ be defined as the decision variables for the optimization problem.

$$
\begin{aligned}
X_i &= [x_c, \dot{x}_c, x_z, \quad y_c, \dot{y}_c, y_z]_i^T \\
U_i &= [\dot{x}_z, \dot{y}_z]_i^T
\end{aligned}
$$

Where $i \in [0, 1, ..., C-1]$

The optimization problem can be defined as

$$
\begin{aligned}
&\min_{X,U} \sum_{i=0}^{C-1} (\|U_i\|^2 + \beta \|\vec{p}_z^{\,i} - \vec{p}_{mc}^{\,i}\|^2) \\
&\text{subject to: ZMP constrain (3.21)} \\
&\qquad\qquad \text{Terminal constrain (3.37)} \\
&\qquad\qquad |\dot{x}_z| \leq \dot{x}_z^{max} \\
&\qquad\qquad X_0 = \text{current state of robot}
\end{aligned}
\tag{3.38}
$$

The objective function of the optimization problem comprises two primary terms: the first term minimizes the ZMP velocity, while the second term, weighted by a positive coefficient β, regulates the ZMP position to remain proximal to the center of the moving constraint. Upon solving this optimization problem, the state X_1 is extracted to serve as the reference Center of Mass (CoM) trajectory for the gait regulator. The implementation utilized in this research is based on the Python framework provided in [22].

3.5 Gait Regulation

3.5.1 Robot Dynamics

$$\begin{bmatrix}\mathbf{M}_1\\ \mathbf{M}_2\end{bmatrix}\begin{pmatrix}\dot{\nu}_c^d\\ \ddot{q}^d\end{pmatrix}+\begin{bmatrix}\mathbf{C}_1\\ \mathbf{C}_2\end{bmatrix}\begin{pmatrix}\nu_c^d\\ \dot{q}^d\end{pmatrix}=\begin{pmatrix}\vec{W}_g\\ \tau\end{pmatrix}+J^T\begin{pmatrix}\vec{W}_{\text{imp,c}}\\ \vec{W}_{\text{grf}}\\ \vec{W}_{\text{imp,sw}}\end{pmatrix} \tag{3.39}$$

In Equation (3.39), the system matrices and vectors are defined as follows: $[M_1, M_2]^T$ represents the inertia matrix partitioned into the center of mass component ($M_1 \in R^{6\times\bar{n}}$) and the actuated joint component ($M_2 \in R^{n\times\bar{n}}$), n denotes the number of actuated joints, while $\bar{n} = n+6$ represents the total number of degrees of freedom (DoF) for the humanoid robot, accounting for the 6 unactuated floating base coordinates.

The vector $\left(\dot{\nu}_c^d, \ddot{q}^d\right)^T$ denotes the desired system accelerations, consisting of the desired Center of Mass (CoM) velocity derivative $\dot{\nu}_c^d$ and the desired joint accelerations $\ddot{q}^d$. Similarly, $[C_1, C_2]^T$ corresponds to the Coriolis and centrifugal matrix partitioned to match the inertia matrix structure, multiplying the desired velocity vector $\left(\nu_c^d, \dot{q}^d\right)^T$.

On the right-hand side, w_g is the gravitational wrench acting on the center of mass, and τ represents the vector of actuated joint torques. Finally, J^T is the transpose of the generalized task Jacobian matrix, which maps the task-space wrenches into the generalized force space; specifically, w_c^{imp} denotes the CoM impedance wrench, w_{grf} represents the ground reaction forces (contact forces) subject to optimization, and w_{imp} denotes the impedance forces for the remaining generalized tasks.

3.5.2 Controller Formulation

1. Task Controller
   For the task controller we use the concept impedance control in translation and orientation controller.
   Translation controller

$$\mathbf{f}_{imp} = K_p(\mathbf{x}^d - \mathbf{x}) + K_d(\dot{\mathbf{x}}^d - \dot{\mathbf{x}}) \tag{3.40}$$

   Where

   - f_{imp}: The vector of impedance forces generated to track the desired trajectory.
   - K_p: The positive diagonal stiffness matrix representing the linear spring stiffness of the system.
   - K_d: The positive diagonal damping matrix representing the linear damping of the system.
   - x^d: The reference (desired) Cartesian position vector of the task.
   - x: The actual Cartesian position vector of the task.
   - $\dot{x}^d$: The reference (desired) Cartesian velocity vector.
   - $\dot{x}$: The actual Cartesian velocity vector.

   Orientation controller
   For orientation control, the quaternion representation is adopted to circumvent the singularities associated with Euler angles, as mentioned in [**Six-DOF impedance control**]. The unit quaternion, comprising a vector part (ϵ) and a scalar part (δ), can be extracted

from the relative rotation matrix $R_c^d = R_d^T R_c$, where R_c and R_d represent the current and desired orientations of the torso in the world frame, respectively.

$$\mathbf{n}_{imp} = -2(\delta I + \hat{\epsilon}) K_r \epsilon + D_r(w^d - w) \tag{3.41}$$

Where

- K_r: The positive diagonal stiffness matrix representing the rotational spring stiffness of the system.
- D_r: The positive diagonal damping matrix representing the rotational damping of the system.
- $\hat{\epsilon}$: Screw symmetric matrix of ϵ
- I: Identity matrix

The task controller can be construct by combine 3.40 and 3.41 into wrench impedance($\vec{W}_{imp}$) of i^{th} task.

$$\vec{W}_{imp} = \begin{bmatrix} \mathbf{n}_{imp} \\ \mathbf{f}_{imp} \end{bmatrix} \tag{3.42}$$

In this study, the control framework utilizes two wrench impedances: the CoM impedance $\vec{W}_{imp,c}$ for tracking the CoM trajectory, and the swing leg impedance $\vec{W}_{imp,sw}$ for tracking the swing leg trajectory.

2. Optimization Problem Formulation
Form dynamics in 3.39 it can extract the Centroidal dynamics from first 6 row then we got

$$J_{\text{grf},c}^T \vec{W}_{\text{grf}} = \vec{W}_{\text{ff},c} - \vec{W}_g - \vec{W}_{\text{imp,c}} - J_{\text{imp},c}^T \vec{W}_{\text{imp}}, \tag{3.43}$$

where

$$\vec{W}_{\text{ff},c} = M_1 \left(\dot{\nu}_c^d\right) + C_1 \left(\nu_c^d\right), \tag{3.44}$$

$$\vec{W}_g = [0, 0, 0, 0, 0, mg]^T \tag{3.45}$$

The mass matrix M_1 is diag(m) when m is the total mass of robot and C_1 is zero.Follow the work in [16] the constrain optimization problem to solve for $\vec{W}_{grf}$ can be construct by

$$\min_{\vec{W}_{\text{grf}}} \left(\frac{1}{2} \delta_c^T \mathbf{Q}_c \delta_c + \frac{1}{2} \delta_f^T \mathbf{Q}_f \delta_f \right) \tag{3.46}$$

with the residuals defined as:

$$\delta_c = J_{\text{grf},c}^T \vec{W}_{\text{grf}} - \vec{W}_{\text{ff},c} + \vec{W}_g + \vec{W}_{\text{imp,c}} + J_{\text{imp},c}^T \vec{W}_{\text{imp,sw}} \tag{3.47a}$$

$$\delta_f = \vec{W}_{\text{grf}} - \vec{W}_{\text{grf}}^d \tag{3.47b}$$

The optimization problem is subject to *contact unilaterality*, *friction cone constraints*, and *Center of Pressure constraints*. Q_c and Q_f are positive definite weighting matrices, where $Q_c \ll Q_f$ is selected to assign higher priority to the CoM task. The vector $\vec{W}_{grf}^d$ represents the precomputed reference ground reaction force, which is set to zero by default. To mitigate torque discontinuity during phase switching (e.g., from stance to swing phase), a transition strategy proposed in [16] is implemented. This constrained optimization problem is solved using the nonlinear programming solver in Drake at a frequency of 1 kHz.

3. Precomputed Contact Forces

   In contrast to the formulation presented in [16], where the desired ground reaction wrench $\vec{W}_{\text{grf}}^d$ was assigned as zero—causing the optimization variable δ_f to function purely as a minimization term for the total contact force—this work introduces a force reference pre-calculation module based on the Center of Mass (CoM) position. This precomputed wrench provides a physical feedforward baseline for the ground reaction forces, optimizing the force distribution profile specifically while the robot is in the double-support phase. Although this pre-calculation does not reduce the overall computation time of the quadratic program solver, it yields a more physically intuitive force distribution and mitigates abrupt command transitions during phase changes.

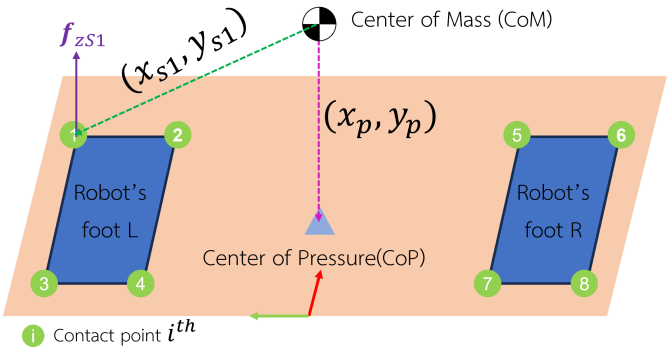

Figure 3.19 Robot's contact points

We assume the robot has four contact points on each foot ($\alpha = 8$) as shown in Figure 3.19. When full contact is established, all contact points are assumed to lie on the same plane. Given the reaction force f_{zSi} at the i^{th} contact point and its position vector relative to the Center of Mass (x_{si}, y_{si}), the solution for $f_{grf,z}^d$ can be explicitly computed using the method proposed in [**Full-Body Compliant Human–Humanoid Interaction**]. Let (x_p, y_p) denote the location of the Center of Pressure relative to the Center of Mass. Based on the definition of the Center of Pressure, the system of equations can be rear-

ranged into matrix form as follows:

$$\begin{bmatrix} x_P \\ y_P \\ 1 \end{bmatrix} mg = \underbrace{\begin{bmatrix} x_{S1} & x_{S2} & \cdots & x_{S\alpha} \\ y_{S1} & y_{S2} & \cdots & y_{S\alpha} \\ 1 & 1 & \cdots & 1 \end{bmatrix}}_{A_z \in \mathcal{R}^{3\times\alpha}} \underbrace{\begin{bmatrix} f_{zS1} \\ f_{zS2} \\ \vdots \\ f_{zS\alpha} \end{bmatrix}}_{F_z \in \mathcal{R}^{\alpha\times 1}} \tag{3.48}$$

From 3.48 can observe that the matrix A_z not a square matrix so to solve for F_z we need to use pseudo-inverse $A_z^{\#}$ instead of A_z^{-1} the explicit solution of F_z calculated as

$$\begin{bmatrix} f_{zS1} \\ f_{zS2} \\ \cdots \\ f_{zS\alpha} \end{bmatrix} = A_z^{\#} \begin{bmatrix} x_P \\ y_P \\ 1 \end{bmatrix} mg \tag{3.49}$$

Subsequent to computing F_z, the contact forces for each leg are aggregated to obtain $f^d_{grf,z}(l)$ for the left leg and $f^d_{grf,z}(r)$ for the right leg during the double stance phase (whereas only one exists during the single stance phase). Consequently, the reference wrench for 3.47b is derived as
$\vec{W}^d_{grf} = \begin{bmatrix} \mathbf{0}_{1\times 5} & f^d_{grf,z}(l) & \mathbf{0}_{1\times 5} & f^d_{grf,z}(r) \end{bmatrix}^T$. In this work, the remaining values in $\vec{W}^d_{grf}$ are set to zero, as the objective is to minimize forces and torques in other axes. Experimental results indicate that providing a reference solely in the Z-axis is sufficient to achieve an optimal solution. Moreover, F_z is utilized to formulate the initial guess; similar to the construction of $\vec{W}^d_{grf}$, only the force in the Z-direction is initialized using F_z.

4. Stance Leg Stabilization

   To mitigate foot oscillations when traversing compliant surfaces, [16] proposed a stance-leg stabilization wrench, $\mathbf{w}_{stab} = -D\mathbf{v}_{stance}$, where D is a positive-definite diagonal damping matrix. While damping effectively reduces oscillatory effects on compliant surfaces, a damping-only approach is insufficient for low-stiffness or low-dissipation surfaces. In such environments, the stance foot orientation may undergo rapid fluctuations that the original stabilization controller cannot effectively regulate back to the desired orientation. To address this limitation, an orientation controller is integrated into the stabilization framework, replacing the pure damping term for the rotational components. This stabilization wrench is applied exclusively to the stance leg.

5. Joints Torque Calculation

   At this stage, having acquired $\vec{W}_{imp,sw}$ from 3.42 and $\vec{W}_{grf,c}$ from solving 3.46, the next process is to convert these wrenches into joint torques ($\tau_{actuate}$). This is achieved by utilizing the principle of virtual work and incorporating a feedforward torque to compensate for gravity (τ_g):

$$\tau_{actuate} = \tau_g + \begin{bmatrix} J_l^T & J_r^T \end{bmatrix} \begin{bmatrix} \vec{W}_l + \vec{W}_{stab,l} \\ \vec{W}_r + \vec{W}_{stab,r} \end{bmatrix} \tag{3.50}$$

   where τ_g is the torque due to gravity, J_l and J_r denote the Jacobian matrices of the left and right legs relative to the torso, while $\vec{W}_l$ and $\vec{W}_r$ represent the wrenches at the end-effectors of the left and right legs, respectively.During the double support phase, $\vec{W}_l$ corresponds to the first six elements and $\vec{W}_r$ to the final six elements of $\vec{W}_{grf,c}$. In the

single stance phase (e.g., when the left leg is the stance leg), the assignment is defined as $\vec{W}_l = \vec{W}_{grf,c}$ and $\vec{W}_r = \vec{W}_{grf,sw}$.

3.6 State Estimator

3.6.1 Forward Kinematics

Forward kinematics—including the computation of foot positions and Center of Mass (CoM) states—as well as robot dynamics are evaluated using the Drake. . These computations are performed by passing generalized position, based and joint position, and generalized velocity, based and joint velocity, into the Drake's function.

3.6.2 ZMP Calculation

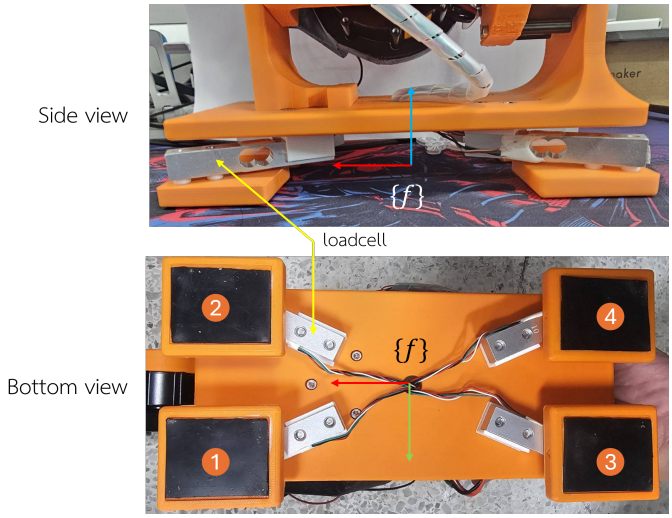

Figure 3.20 Loadcells on physical robot's sole

To determine the Zero Moment Point (ZMP) position, load cells are affixed to the corners of each robot sole, as illustrated in Figure 3.20. Contact forces are sampled at a frequency of 100 Hz ($\delta t = 0.01$ s). For the purposes of this study, the forces measured by the load cells are modeled as point contacts, as depicted in Figure 3.21.

As illustrated in Figure 3.21, the Zero Moment Point (ZMP) position in the world frame, $\vec{p}_{zmp}$, is calculated as the weighted average of the individual contact point positions, $\vec{p}_i$. In this formulation, the weighting factors correspond to the vertical ground reaction forces, $f_{z,i}$, measured at each respective contact point.

$$\vec{p}_{zmp} = {}^{w}T_f \frac{\sum_{i=1}^{n} f_{z,i} \cdot \vec{p}_i}{\sum_{i=1}^{n} f_{z,i}} \tag{3.51}$$

Where ${}^{w}T_f$ is the transformation matrix from world frame w to foot frame f.

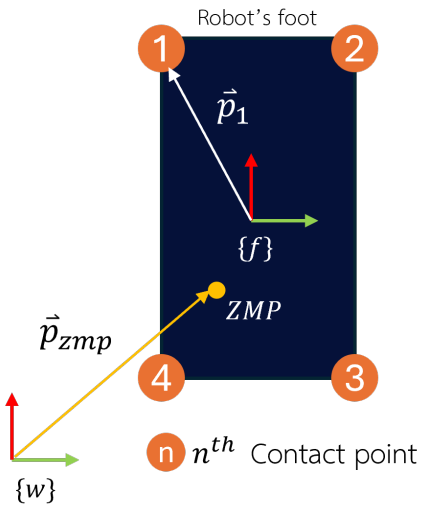

Figure 3.21 Contact point of robot's foot

3.6.3 Kalman Filter

When deploying the framework on a physical robot, a critical consideration is the presence of sensor measurement noise, particularly in joint positions and velocities. As previously noted, these joint-level states directly affect the computation of the Center of Mass (CoM) position and velocity; consequently, noisy measurements can induce undesirable oscillations in the robot. To mitigate these effects, a discrete Kalman filter is employed to provide filtered estimates of the CoM position, CoM velocity, and the Zero Moment Point (ZMP) position.

- Prediction and Measurement Models
  The prediction model for the discrete Kalman filter is derived by discretizing the continuous-time system dynamics presented in Equation 3.18.

$$\begin{aligned} \vec{X}[n+1] &= A\vec{X}[n] + BU[n] + Q \\ \vec{X}[n+1] &= \begin{bmatrix} 0 & 1 & 0 \\ \eta^2\delta t & 0 & -\eta^2\delta t \\ 0 & 0 & 0 \end{bmatrix} \vec{X}[n] + \begin{bmatrix} 0 \\ 0 \\ \delta t \end{bmatrix} \dot{x}_{zmp}[n] + Q \end{aligned} \tag{3.52}$$

  Which measurement model is

$$\begin{aligned} \vec{Y}[n] &= C\vec{X}[n] + R \\ \vec{Y}[n] &= [1,1,1]\vec{X}[n] + R \end{aligned} \tag{3.53}$$

  Where $\vec{X}[n] = [x_c[n], \dot{x}_c[n], x_{zmp}[n]]^T$ represents the state vector at time step n. The control input, $U[n] = \dot{\vec{x}}_{zmp}[n-1]$, is obtained from the Model Predictive Control (MPC) solution of the previous time step. Furthermore, Q and R denote the process noise and measurement noise covariance matrices, respectively, while T represents the sampling period of the Kalman filter.
- Observability
  A system is observable if and only if its observability matrix, $\mathcal{O}$, possesses full rank, such that $\text{rank}(\mathcal{O}) = n$, where n denotes the dimension of the state vector. This matrix is constructed as follows:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \tag{3.54}$$

If $\text{rank}(\mathcal{O}) < n$, the system contains an unobservable subspace, indicating that certain state combinations cannot be uniquely determined from the output measurements. For the state-space model implemented in the aforementioned Kalman filter, the observability matrix $\mathcal{O}$ possesses a rank of 3. Since this rank is equal to the dimension of the state vector ($n = 3$), the system is confirmed to be fully observable.

3.7 System Overview: Physical Robot

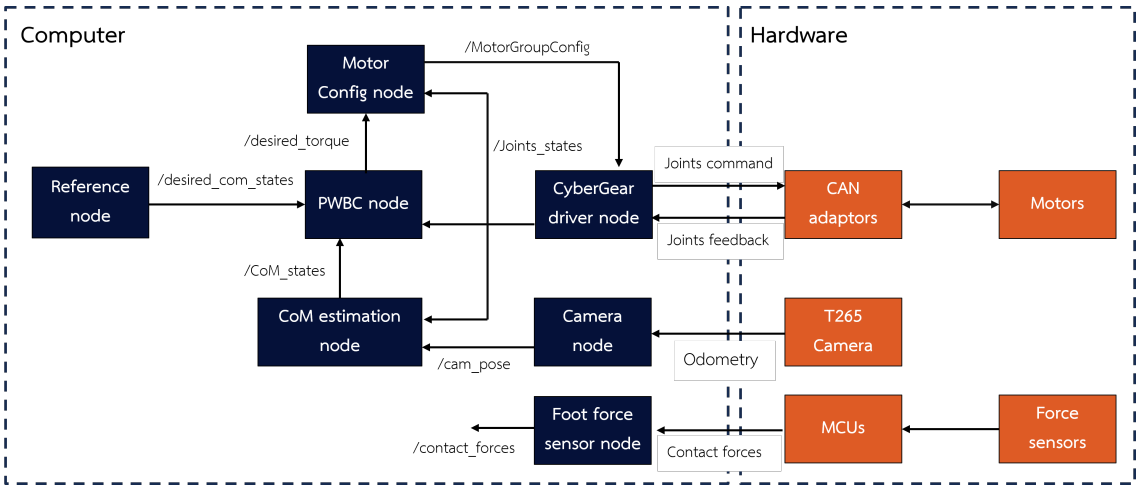

Figure 3.22 Physical robot's system overview

For the physical robotic platform, Robot Operating System 2 (ROS2) was employed as the communication middleware to bridge the hardware and software layers. In the current implementation, only the Whole-Body Control (WBC) framework has been deployed on the physical system. Unlike the simulation environment, hardware deployment necessitates dedicated sensor integration; consequently, a camera node and a foot force sensor node were developed to acquire data from the vision system and load cells, respectively. Due to hardware constraints, the execution frequency of the WBC node is capped at 200 Hz—matching the maximum sampling rate of the camera—while the foot force sensor node operates at 100 Hz.

To facilitate communication with the actuators, a Cybergear driver node was developed, operating at a frequency of 300 Hz. This node manages bidirectional communication by receiving motor feedback—including position, velocity, and torque—and transmitting commands such as target torques and torque enable/disable signals. This driver interacts with a motor configuration node, which encapsulates necessary parameters—such as controller configurations, target torques, and motor IDs—into the `/MotorGroupConfig` message format.

CHAPTER 4 EXPERIMENT AND RESULT

4.1 Experiments in Simulation

In this research, the Drake multibody physics engine (`MultibodyPlant`) is utilized as the primary robotic simulation environment. The simulations are executed on an ASUS ROG Strix G18 (2023) laptop with a sampling frequency of 10 kHz, achieving a 15% real-time execution rate. The robot's Unified Robot Description Format (URDF) was exported from SolidWorks, preserving the designated coordinate frames and inertial properties. The experimental validation is categorized into four primary scenarios:

1. **State Estimator:** Evaluating the performance of the Kalman filter under various measurement models.
2. **Whole-Body Control:** Validating the tracking performance and robustness of the control framework.
3. **Footstep Planner:** Verifying the planner's capability to generate feasible footstep sequences based on arbitrary velocity commands.
4. **Integrated Gait Planning:** Assessing the system's ability to achieve omnidirectional locomotion across diverse terrains through full integration.

- Simulation Configuration

  In this work, the floor within the simulation is modeled as a viscoelastic spring-damper system, characterized by two primary adjustable parameters: stiffness and dissipation. The stiffness parameter dictates the rigidity or "hardness" of the contact, while the dissipation parameter accounts for the energy loss during impact. By tuning the combination of these two values, various real-world surface types and material properties can be accurately represented.

Table 4.1 Contact Parameter Mapping for Simulation Surfaces

Stiffness (k)	**Dissipation (d)**	**Surface Analogy**
High	High	Concrete / Steel
High	Low	Hardwood / Marble
Low	High	Mud / Memory Foam
Low	Low	Jelly / Trampoline

In the simulation environment, the default parameters are set to a stiffness of 5×10^6 N/m and a dissipation of 50 s/m, characterizing a surface with both high rigidity and high energy dissipation.

4.1.1 Experiment 1: State Estimator

Table 4.2 presents the parameters and their corresponding values utilized in the Kalman filter. The output matrices C_1 and C_2 represent scenarios where CoM velocity is either included in or excluded from the measurement model. This experiment evaluates the estimator's performance and the specific impact of utilizing these different output matrices.

Regarding the CoM and ZMP position estimation, both output matrices C_1 and C_2 demonstrate the capability to effectively estimate the CoM position. As illustrated in Figure 4.1, the Root Mean Square Error (RMSE) of the CoM position is reduced from 0.003

Table 4.2 Kalman filter parameters and value

Parameters	**Value**
Sampling time, δt [second]	0.001
Process noise, Q	$[1, 1, 1]$
Measurement noise, R	$[10, 10, 1e4]$
C_1	$[1, 1, 1]$
C_2	$[1, 0, 1]$

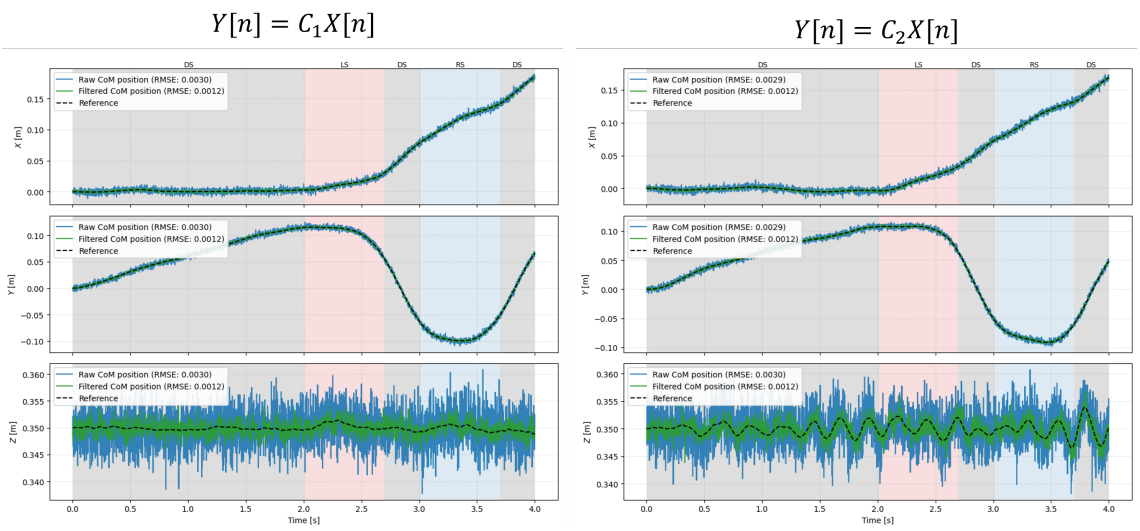

Figure 4.1 Filtered CoM position

m to 0.0012 m. Furthermore, Figure 4.2 shows that the filtered ZMP position successfully mitigates the sharp transient spikes that typically occur during transitions between single-stance and double-stance phases.

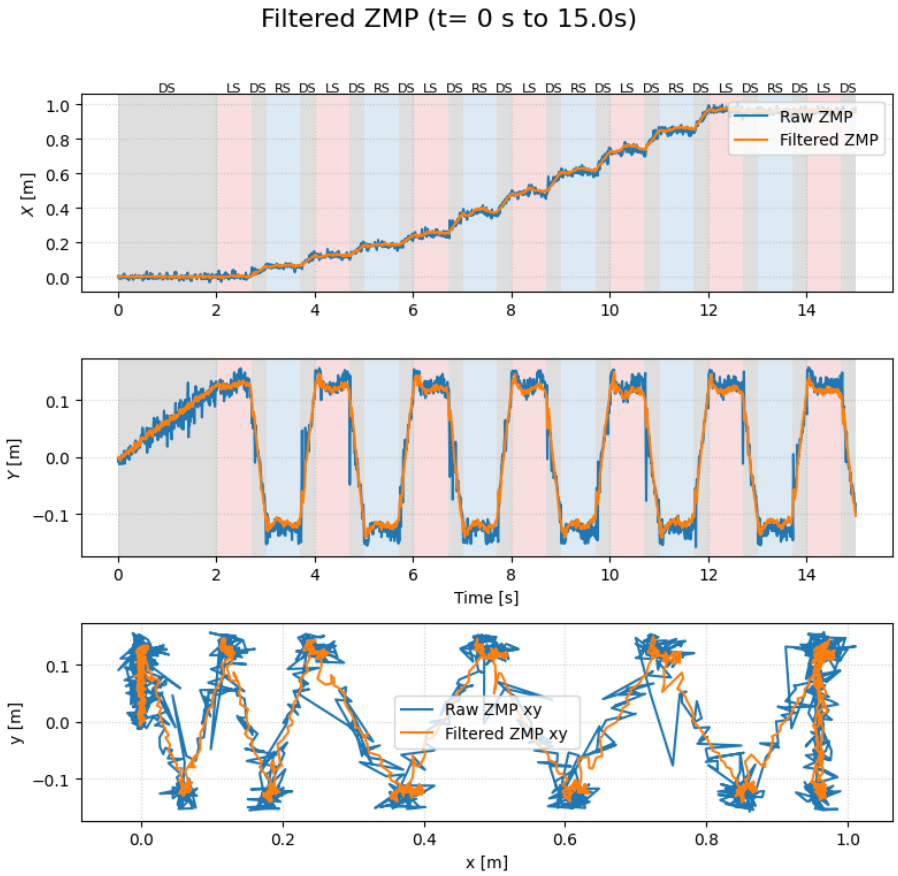

Figure 4.2 Filtered ZMP position

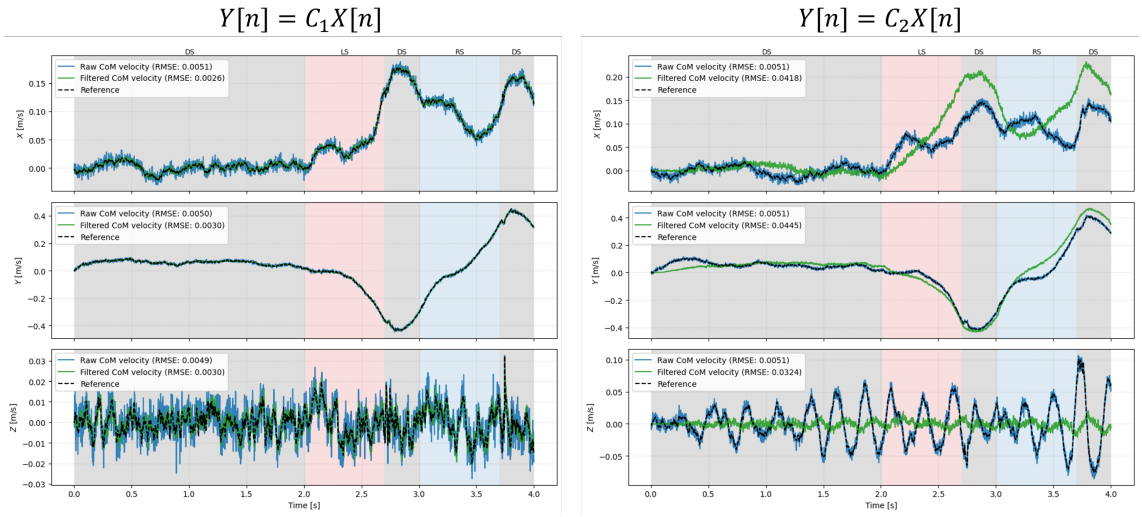

Figure 4.3 Filtered CoM velocity

However, regarding CoM velocity estimation, the Kalman filter exhibits significantly lower accuracy when velocity is omitted from the measurement model compared to configurations where it is explicitly included. The experimental results indicate that if the primary objective is restricted to estimating CoM and ZMP positions, the C_2 measurement model is sufficient.

4.1.2 Experiment 2: Whole-Body Control

In these experiments, two sub-scenarios were conducted using the controller gains detailed in Table 4.3. These gains were applied to each task controller, which operated at an execution frequency of 1 kHz. Within the simulation environment, the floor was modeled as a rigid surface, and the gravitational acceleration vector was defined as $\vec{g} = [0, 0, 9.81]^T$ m/s^2.

Table 4.3 Controller Gain and Values

Controller Types	Parameters	Value
CoM Controller	K_p	diag(4000)
	K_d	diag(300)
	K_r	diag(20)
	D_r	diag(1.5)
Swing Leg Controller	K_p	diag(400)
	K_d	diag(20)
	K_r	diag(10)
	D_r	diag(0.7)
Stance Leg Stabilize Controller	K_p	-
	K_d	diag(0.5)
	K_r	diag(20)
	D_r	diag(5)

- **Experiment 2.1**: Center of Mass Stabilization

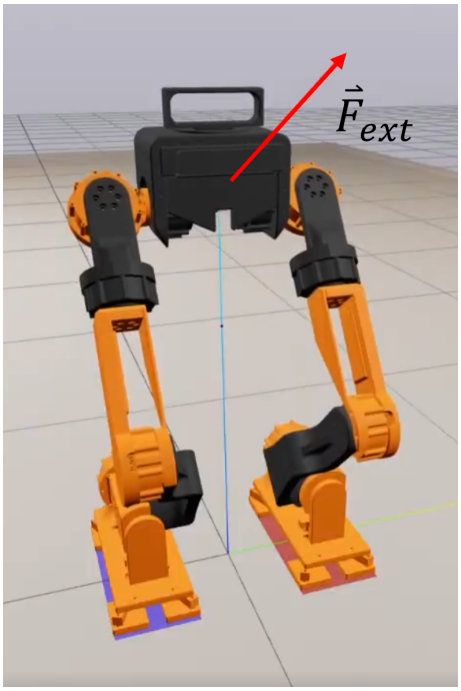

Figure 4.4 External forces applied to base frame.

In this experiment, the robot's disturbance rejection capabilities were evaluated by applying an external force vector, $\vec{F}_{ext} = [12, 10, 15]^T$ N, to the base frame while the system was in a static standing posture, as illustrated in Figure 4.4.

As illustrated in Figure 4.5, an external force $\vec{F}_{ext}$ was applied for a duration of 0.2 seconds, beginning at $t = 1$ s. The controller effectively restored the CoM to its

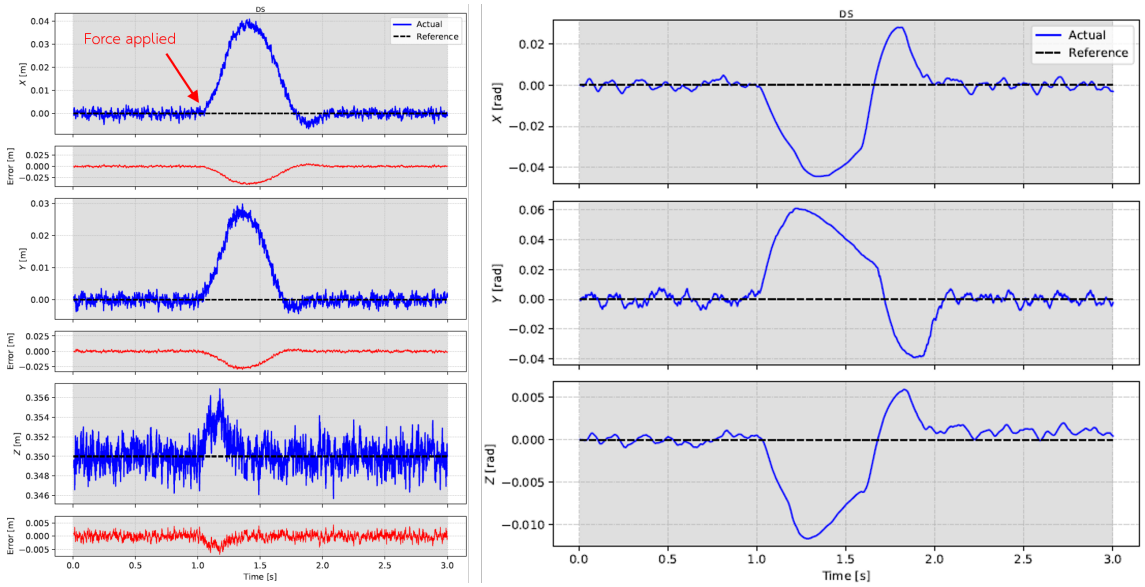

Figure 4.5 CoM Stabilization

initial pose by approximately $t = 2$ s, exhibiting minimal overshoot. Furthermore, the left plot demonstrates that the robot undergoes a displacement in the direction of the applied force before returning to its equilibrium state. This behavior highlights the performance of the CoM impedance controller, which allows the robot to act as a virtual linear spring. In practical applications, this inherent compliance enhances safety during human-robot interaction by avoiding the potential hazards associated with strictly rigid robotic systems.

In a secondary scenario, a vertical disturbance force, $\vec{F}_{ext} = [0, 0, F_{ext,z}]^T$, was applied with the same duration. This test focused on observing the resulting variations in the contact forces acting on each foot.

Figure 4.6 demonstrates that the CoM controller is capable of not only rejecting external disturbances but also facilitating their estimation. The vertical contact force feedback illustrated in both plots indicates that the difference in ground reaction forces between the disturbed and static standing states is exactly $\frac{F_{ext,z}}{2}$ per foot. When a downward force $\vec{F}_{ext} = [0, 0, -F_{ext,z}]^T$ is applied, it is physically equivalent to adding an external payload to the robot. Consequently, the mass of this external load, m_{ext}, can be estimated as:

$$m_l = \frac{|F'_z - F_z|}{2g} \tag{4.1}$$

Where m_l is estimated external load's mass. F'_z and F_z is contact force when the load is added and the robot stands still. Note that Equation 4.1 is true when the robot stand still with CoM positin is in the middle between 2 stance feet.

- **Experiment 2.2**: Center of Mass Trajectory Tracking

  This experiment evaluates the tracking performance of the control system relative to a desired Center of Mass (CoM) trajectory. As illustrated in Figure 4.7, the robot was commanded to perform a lateral shift of $+9$ cm along the y-axis (leftward). The experimental results demonstrate that the CoM task controller accurately tracks the reference trajectory, reaching the target position while successfully suppressing deviations in the other degrees of freedom.

  Furthermore, the orientation controller effectively maintains the torso's orientation,

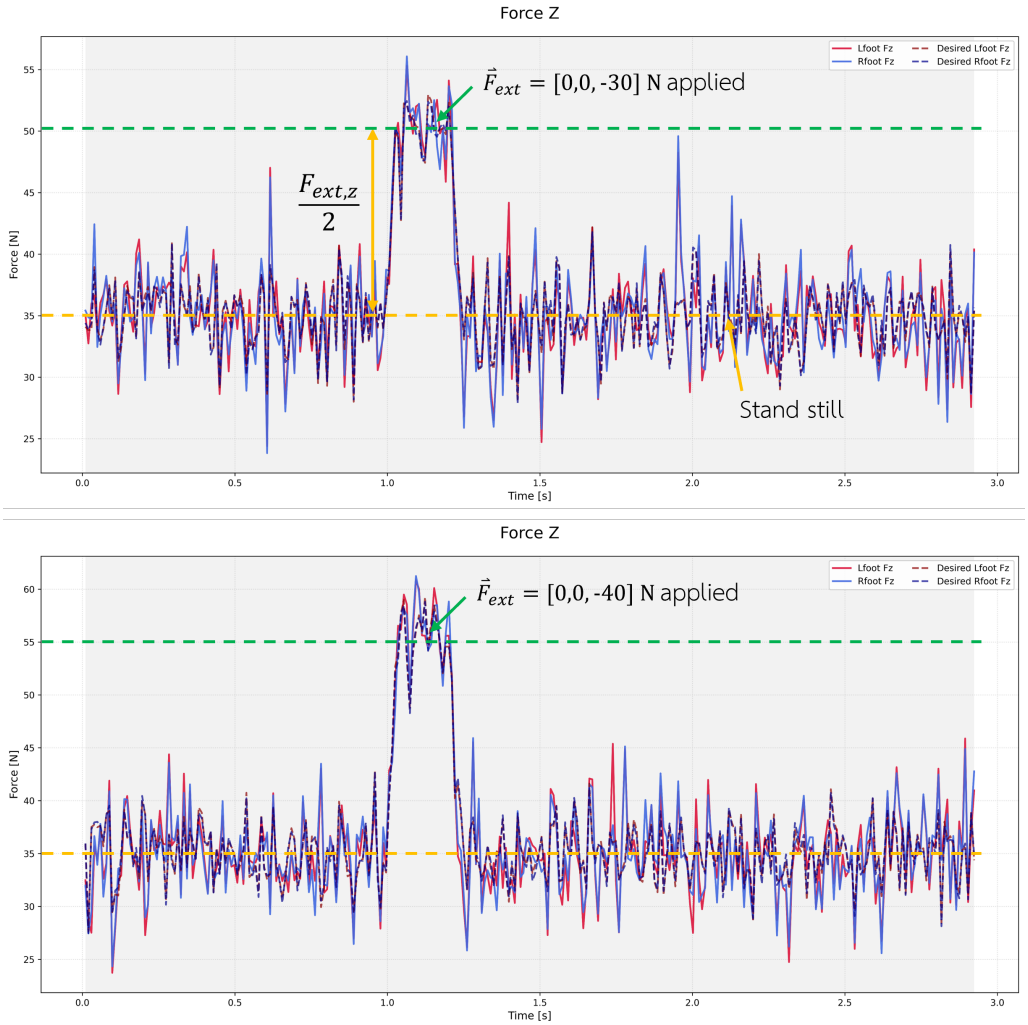

Figure 4.6 External load estimation

as depicted in Figure 4.8. Notably, tracking the CoM trajectory without the active orientation controller results in unwanted torso rotation about the y-axis.

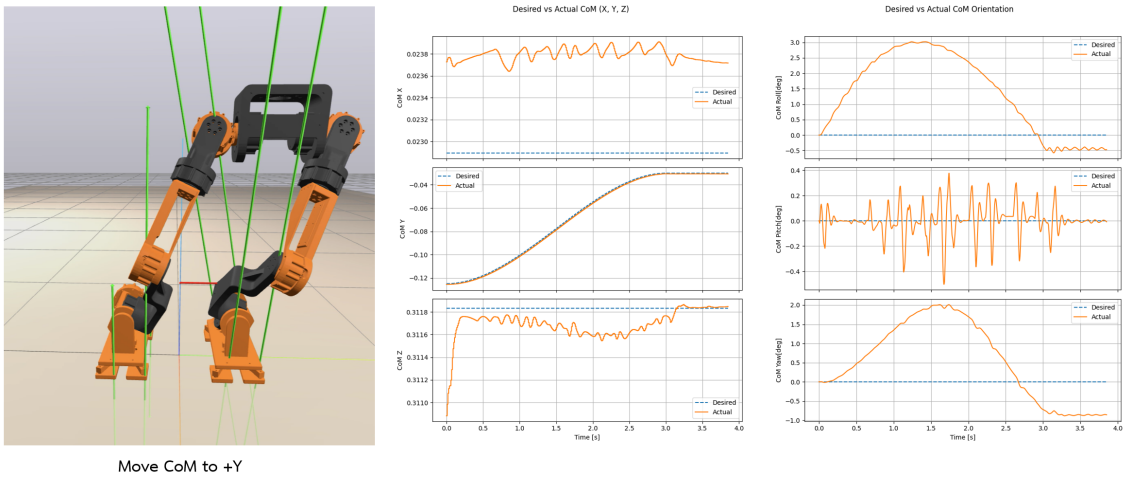

Figure 4.7 CoM trajectory tracking

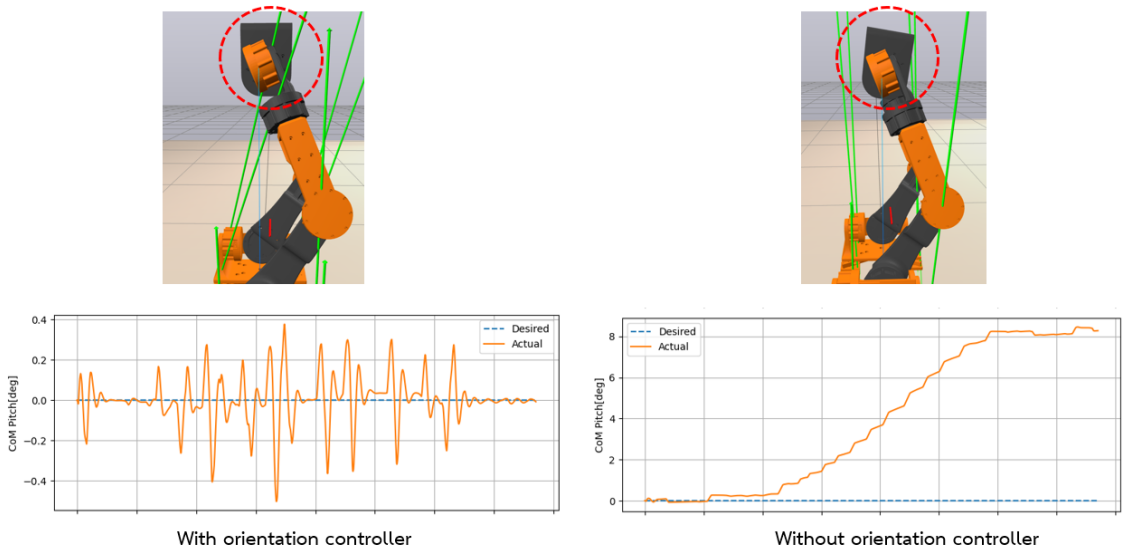

Figure 4.8 Effect from orientation controller while tracking trajectory

As observed in Figure 4.9, the ground reaction wrench (w_{grf}), derived from solving Eq. (3.46), is distributed in accordance with the CoM position. Specifically, the reaction force on the left foot increases while the force on the right foot decreases toward its minimum boundary, thereby enabling the robot to execute the CoM task simultaneously.

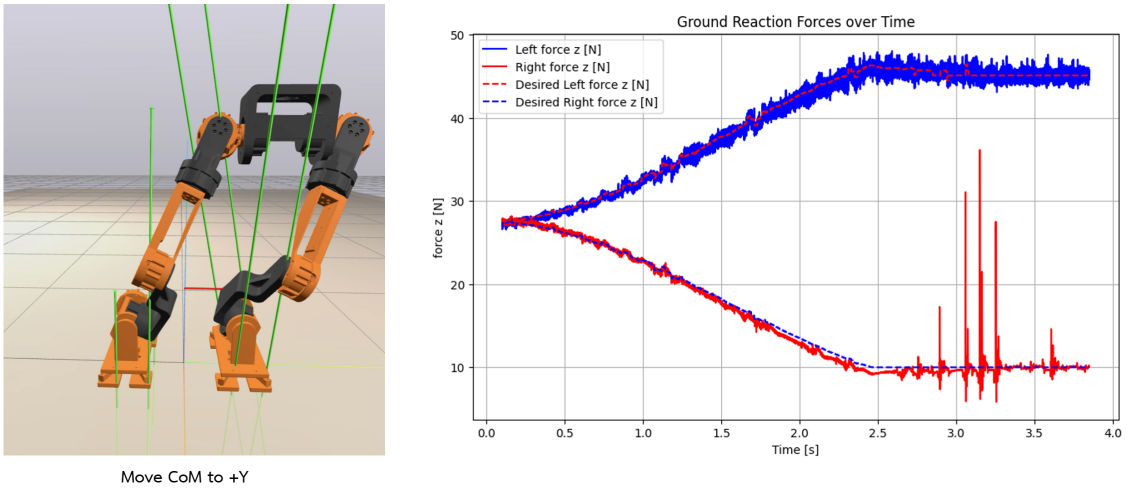

Figure 4.9 Contact forces distribution

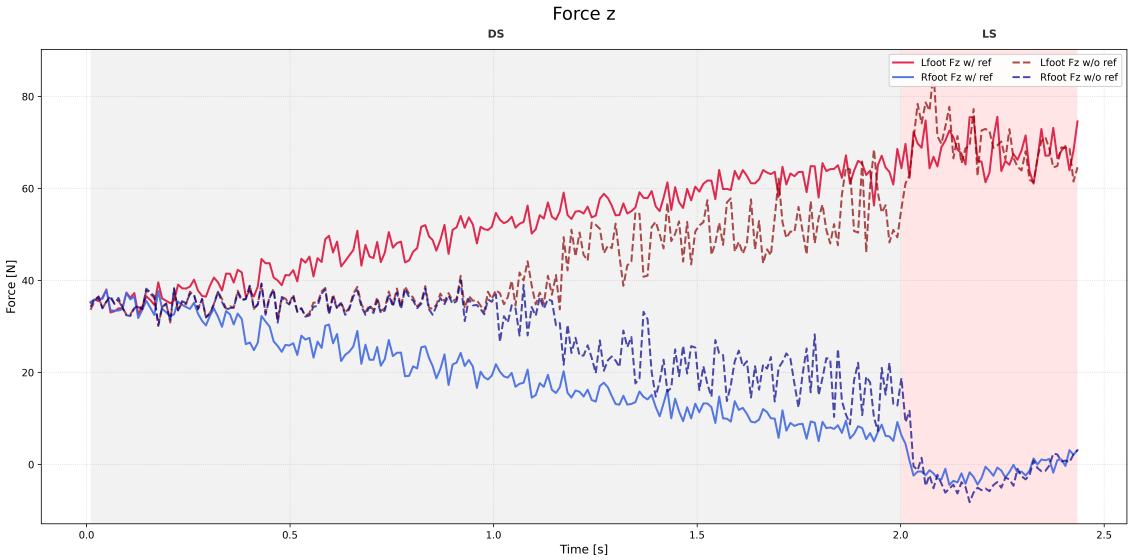

Figure 4.10 Comparison of vertical ground reaction forces (F_z) with and without the proposed force pre-calculation.

Figure 4.10 illustrates the vertical ground reaction forces (F_z) for both feet during the transition from double-support (DS) to single-support (LS) phases. In the baseline formulation without the force reference pre-calculation, the system exhibits severe force chattering and high-frequency oscillations during the DS phase, particularly as the weight distribution shifts. This behavior is caused by the optimizer attempting to minimize the ground reaction wrench to zero arbitrarily.

Conversely, by introducing the CoM-based force reference pre-calculation, the ground reaction forces transition smoothly and linearly. The proposed feedforward baseline effectively suppresses the numerical chatter in the QP solver, ensuring stable force distribution and preventing abrupt command fluctuations prior to entering the single-support phase.

4.1.3 Experiment 3: Footsteps Planner

Table 4.4 Footsteps and Geometric Parameters

Parameters	Value	Unit
Step time	1	second
SS:DS	7:3	-
d_{ax}	0.6	m
d_{ay}	0.2	m
l	0.125	m
$theta_{max}$	$\pi/4$	rad

Table 4.4 summarizes the parameters and corresponding values utilized in the footstep planning module. In this study, seven distinct footstep scenarios were evaluated, comprising six difference reference velocity inputs, as illustrated in Figure 4.11, and one integrated trajectory utilizing a combined reference velocity profile, as shown in Figure 4.12.

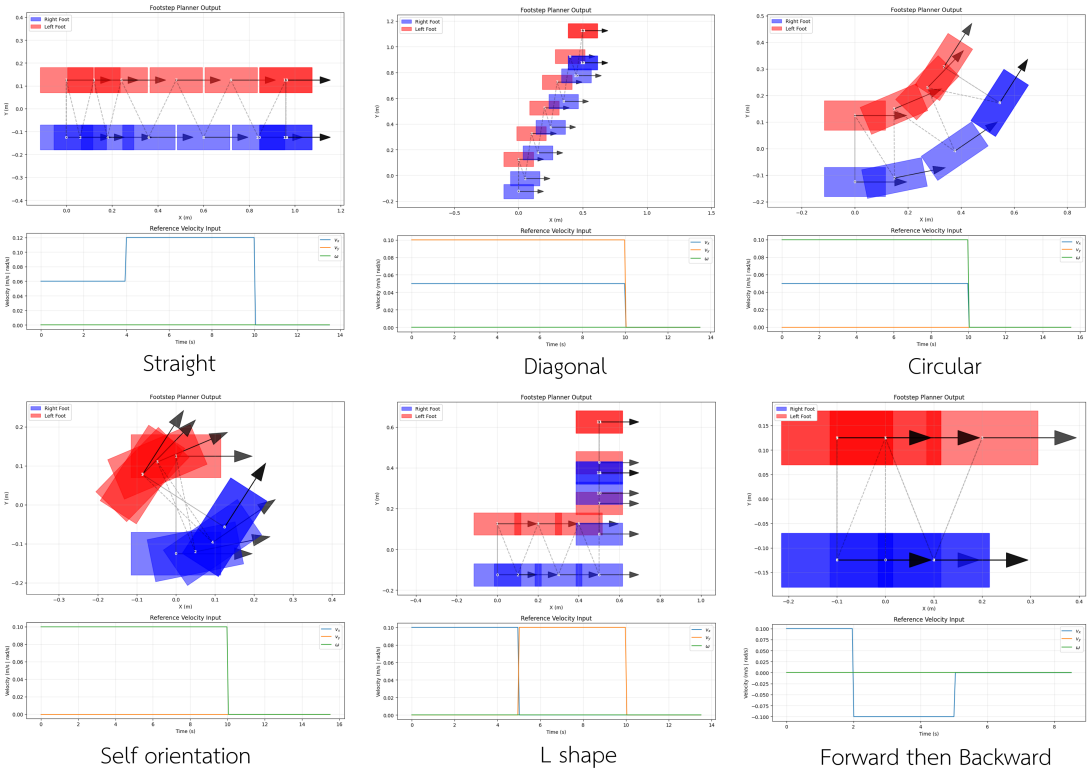

Figure 4.11 6 Differences footsteps

As illustrated in Figures 4.11 and 4.12, the red and blue rectangles denote the left and right feet, respectively. The numerical values within each rectangle indicate the step sequence index, while the black arrows represent the robot's heading. Detailed descriptions of each walking scenario are provided below:

- **Straight walk:** Forward locomotion involving varying longitudinal velocities, including a controlled deceleration to a full stop.
- **Diagonal walk:** Omnidirectional translation achieved by simultaneous movement along both the longitudinal (x) and lateral (y) axes.

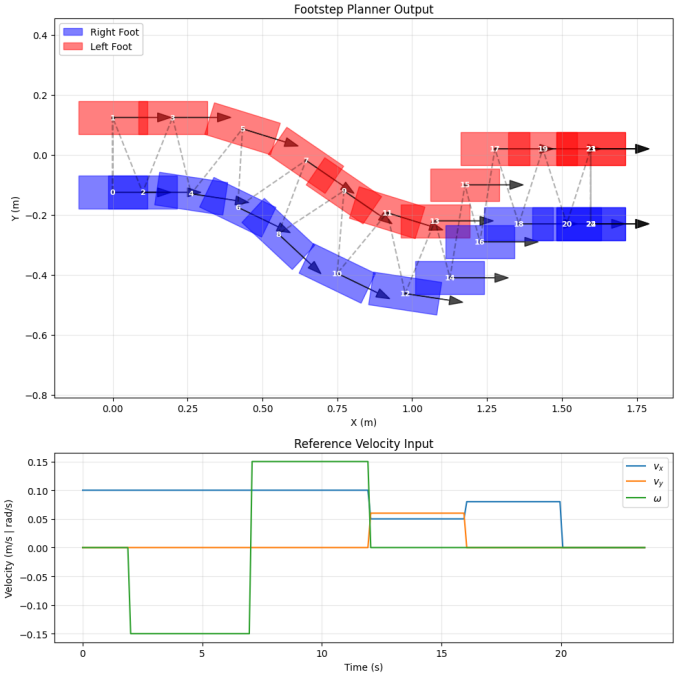

Figure 4.12 Combined footsteps

- **Circular walk:** Curvilinear locomotion generated by combining longitudinal velocity (v_x) and angular velocity (ω) to follow a circular trajectory.
- **Self-orientation:** In-place rotation (yaw) executed by applying angular velocity (ω) in the absence of translational velocity.
- **L-shape walk:** A directional transition characterized by a period of longitudinal movement (v_x) followed by a switch to lateral movement (v_y).
- **Forward and backward walk:** A sequence consisting of forward locomotion followed by a direction reversal to backward motion, concluding with a stop.
- **Combined walk:** A complex, multi-phase trajectory that integrates straight-line, circular, and diagonal walking segments into a single continuous sequence.

4.1.4 Experiment 4: Integrate with Gait planner

Table 4.5 MPC parameters and values

Parameters	**Value**	**Unit**
Sampling time, δ	0.01	second
Control horizon, C	100	steps
β	3000	-
Support polygon size	0.08×0.08	m^2
CoM height, z_c	0.35	m

1. **Experiment 5.1**: Walking on Rigid Floor
   This experiment evaluates the locomotion performance on a planar, rigid surface, characterized by a floor stiffness of $k = 5 \times 10^6$ N/m and a dissipation coefficient of $d = 50$ s/m. The evaluation incorporates the diverse range of footstep patterns detailed in the previous section (Experiment 4).

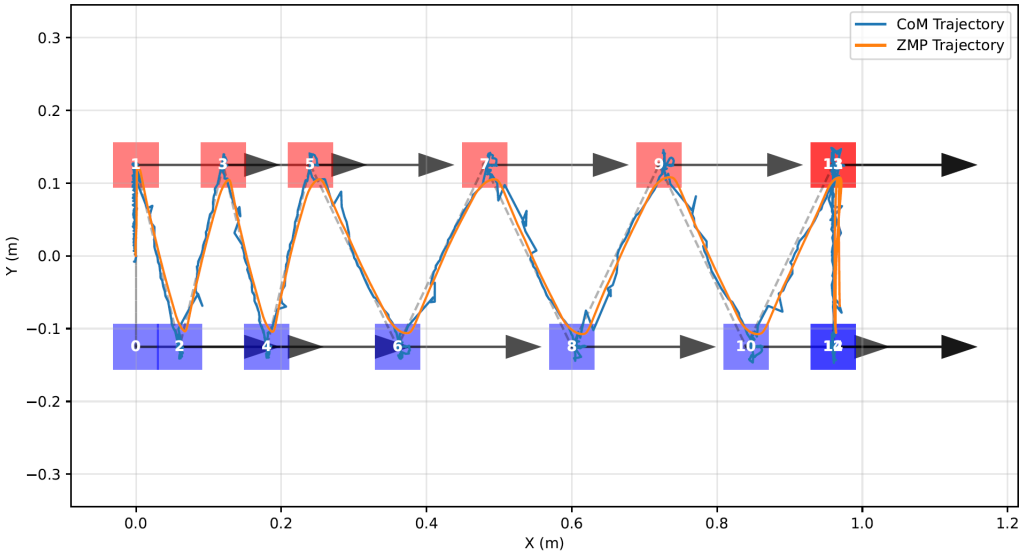

Figure 4.13 Straight walk on rigid floor

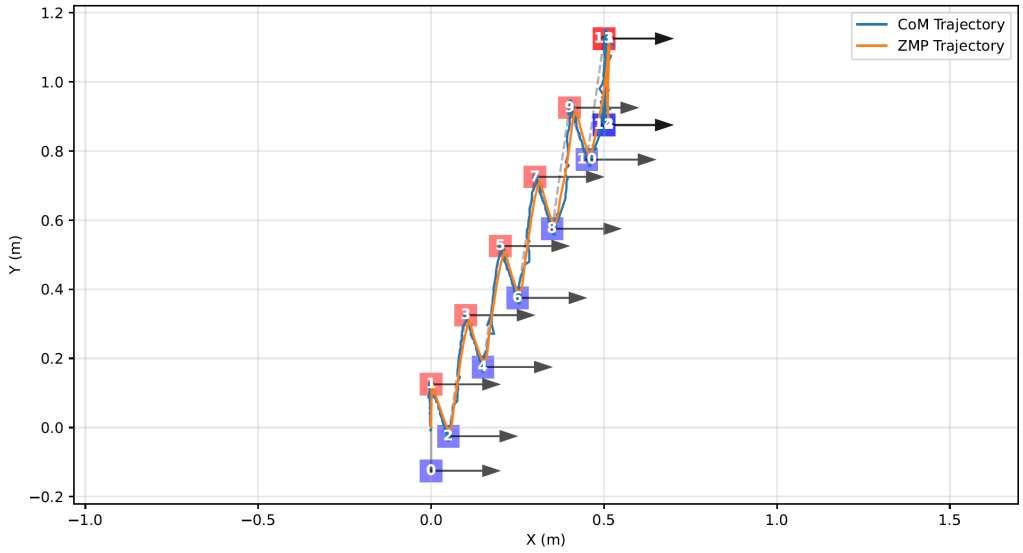

Figure 4.14 Diagonal walk on rigid floor

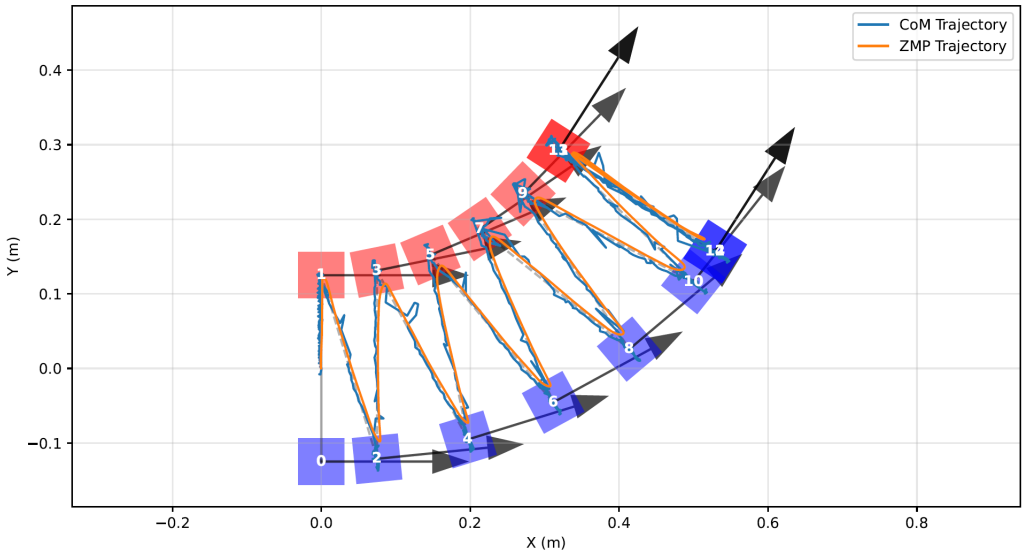

Figure 4.15 Circular walk on rigid floor

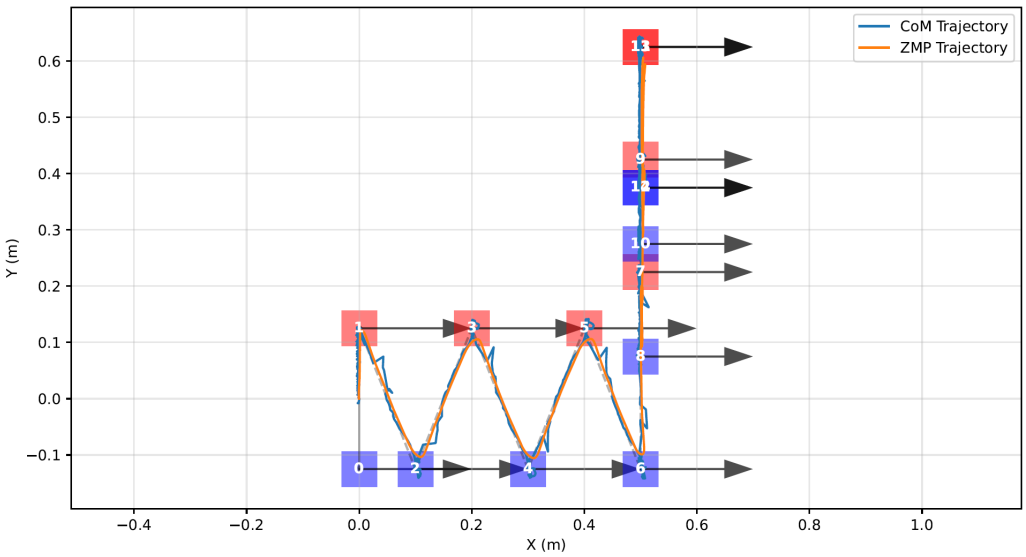

Figure 4.16 L-shape walk on rigid floor

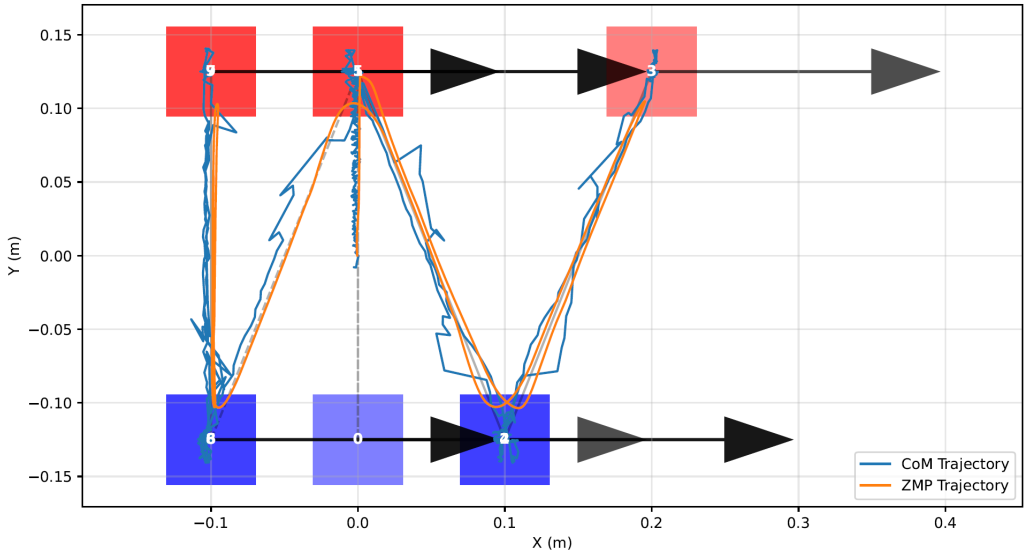

Figure 4.17 Forward/Backward walk on rigid floor

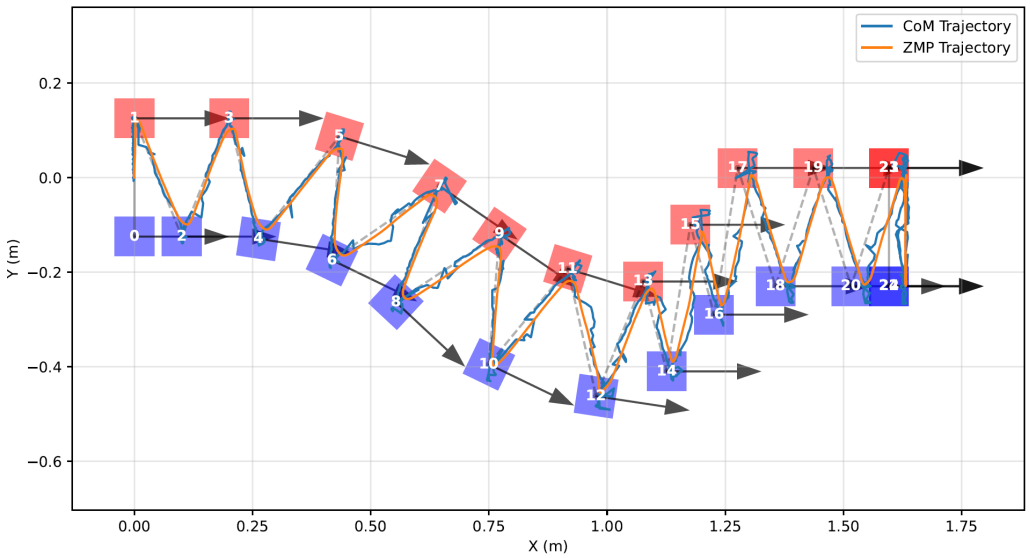

Figure 4.18 Combined walk on rigid floor

As demonstrated in Figures 4.13 through 4.18, the proposed framework successfully enables stable locomotion on rigid surfaces under a wide range of reference velocity inputs. Specifically, the results illustrated in Figures 4.16 and 4.18 confirm that the robot can execute dynamic directional transitions while walking. Furthermore, Figure 4.20 highlights the efficacy of the orientation controller, which ensures that the base attitude accurately tracks the desired reference trajectory throughout the maneuver.

During the combined walking sequence on a rigid floor, the maximum absolute CoM position tracking error is maintained below 5 mm, as illustrated in Figure 4.19. Additionally, the orientation tracking error remains within a margin of 0.1 rad, as depicted in Figure 4.20.

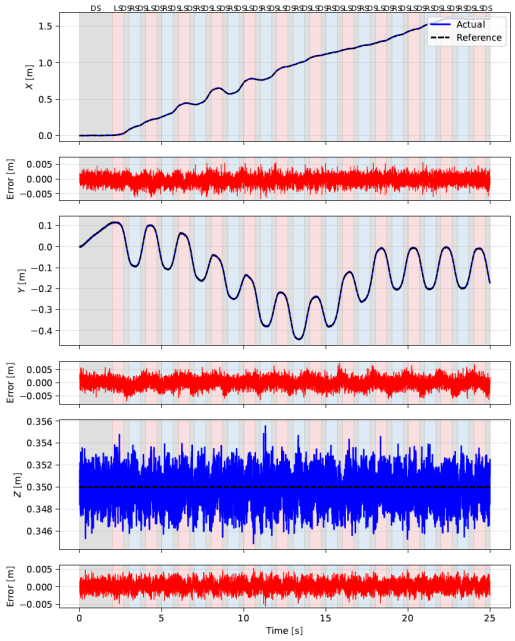

Figure 4.19 CoM position tracking (Combined walk)

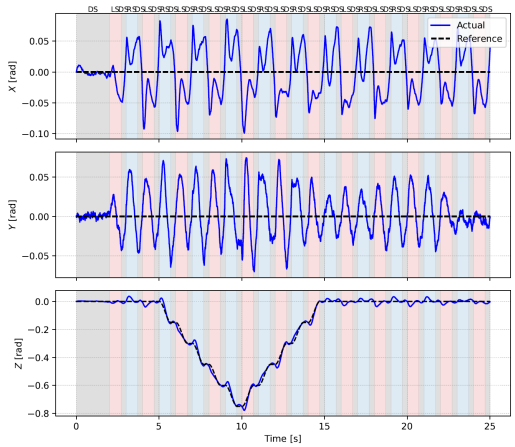

Figure 4.20 CoM orientation tracking (Combined walk)

2. **Experiment 4.2**: Walking on Compliant Floor
   In this experiment, the combined footstep sequence was utilized exclusively, with the floor parameters adjusted to simulate a compliant surface that allows for foot penetration into the ground. The specific ground contact configurations and the resulting performance metrics are summarized in Table 4.6.

Table 4.6 Floor properties and result

Experiment	Stiffness	Dissipation	Result
5.2a	5.00E+03	50	Success
5.2b	2.00E+03	50	Success
5.2c	2.00E+03	10	Fail

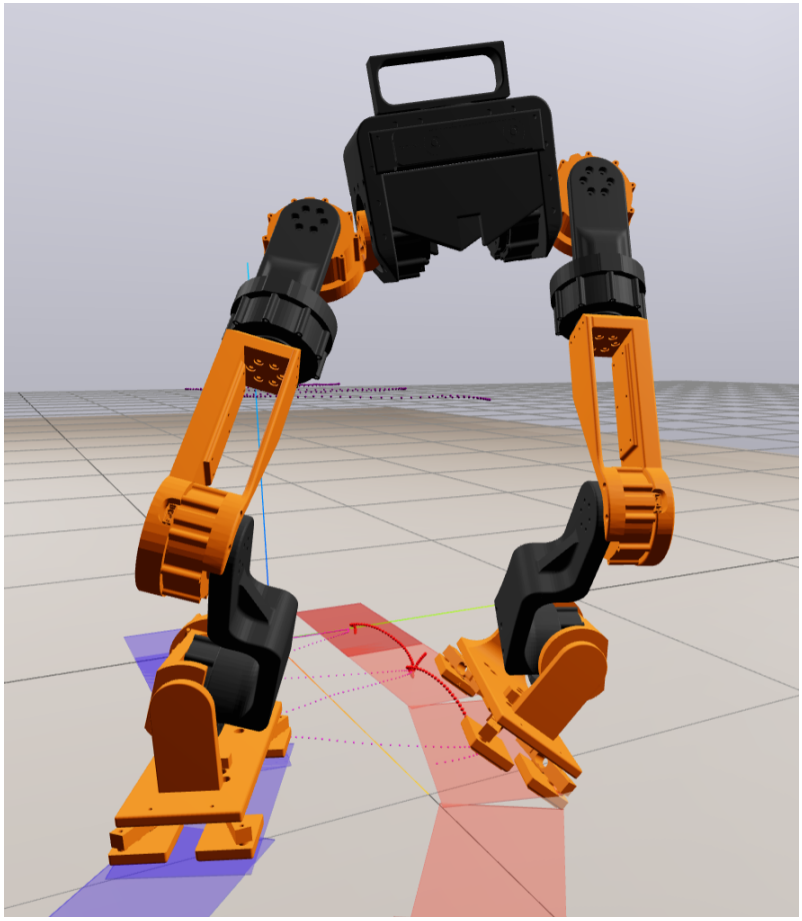

Figure 4.21 Snapshot from simulation when MPC solver fail in Experiment 5.2c

- The Effects of Compliance Floor
  The stiffness and dissipation of the floor directly influence the stance leg dynamics, which consequently affect all other dependent robot states. If the stance leg undergoes rotation due to surface compliance, it induces both a rotation of the entire robot body and a reconfiguration of the support polygon. Specifically, the stiffness parameter dictates the steady-state penetration depth of the foot into the floor; lower stiffness values result in deeper penetration. Conversely, the dissipation parameter influences the sinking velocity and the damping of the impact, as illustrated in Figures 4.22 and 4.23.

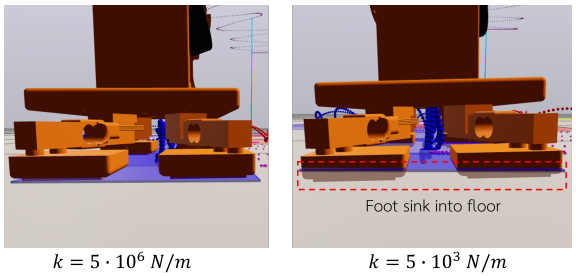

Figure 4.22 Stiffness effect to stance leg position

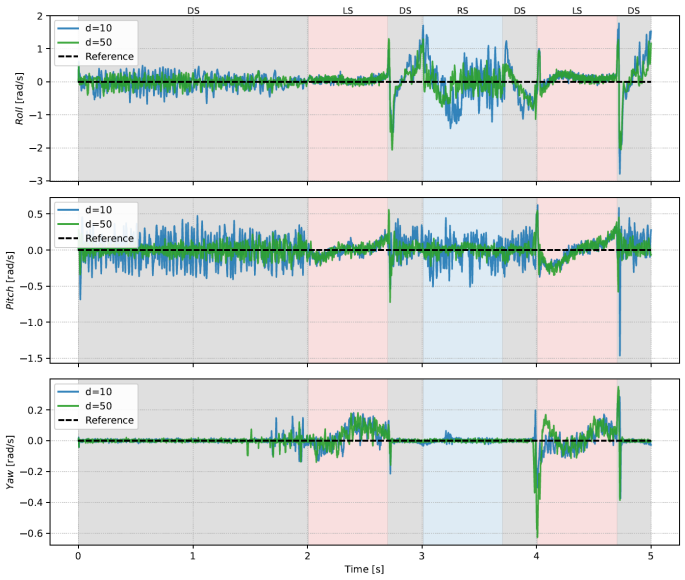

Figure 4.23 Dissipation effect to stance leg velocity

The addition of a stance leg orientation controller significantly improves the tracking performance of the stance foot, as demonstrated in Figure 4.24. In the absence of this controller, the roll error of the foot increases to 0.3 rad due to surface compliance. Conversely, with the controller active, the error remains below 0.1 rad and successfully converges to 0 rad, ensuring the foot maintains proper alignment with the terrain.
However, during the single-support phase on excessively compliant surfaces as in the Experimenat 5.2c. In such cases, the MPC solver often reaches its maximum iteration limit without achieving convergence.

- Comparison: Rigid VS Compliance Floor
  The results presented below provide a comparison between the robot's performance during the combined footstep sequence on rigid and compliant surfaces,

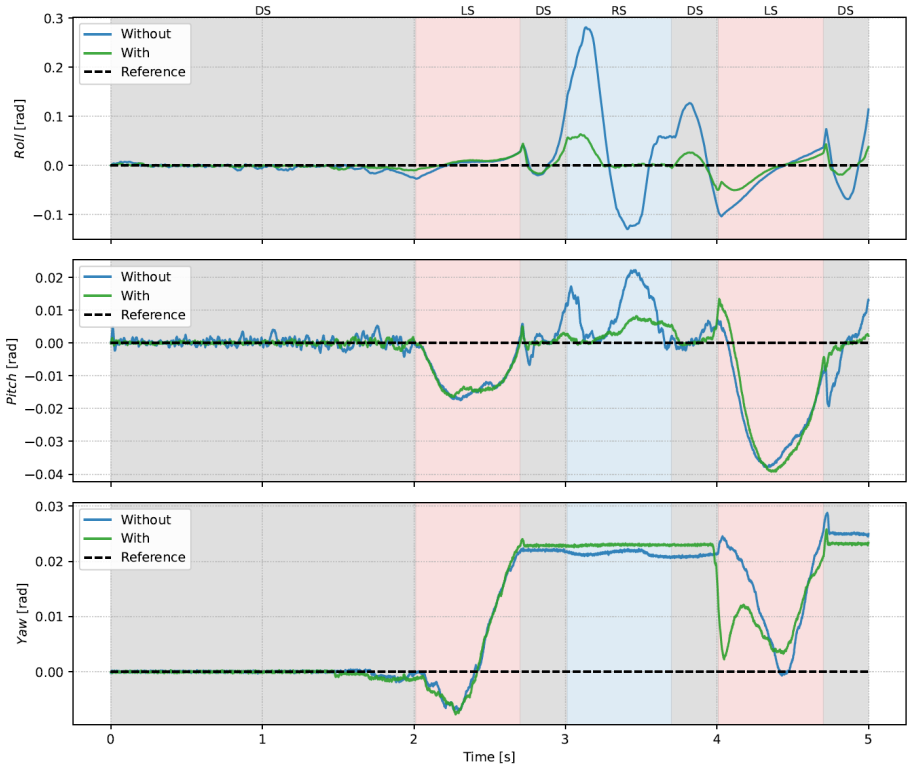

Figure 4.24 Effect of stance leg orientation controller

utilizing the physical properties established in Experiment 5.2b. By incorporating the stance leg stabilization controller, the framework enables the robot to achieve stable locomotion across both floor types.

As illustrated in Figure 4.25, the orientation tracking error during locomotion on the compliant floor is significantly larger than that observed on the rigid floor. This discrepancy is particularly pronounced during the single-support phase, where the entirety of the ground reaction force is concentrated on a single foot. Despite observable oscillations in the roll and pitch axes, the CoM orientation controller effectively maintains the base attitude, ensuring it tracks the reference trajectory regardless of the surface compliance.

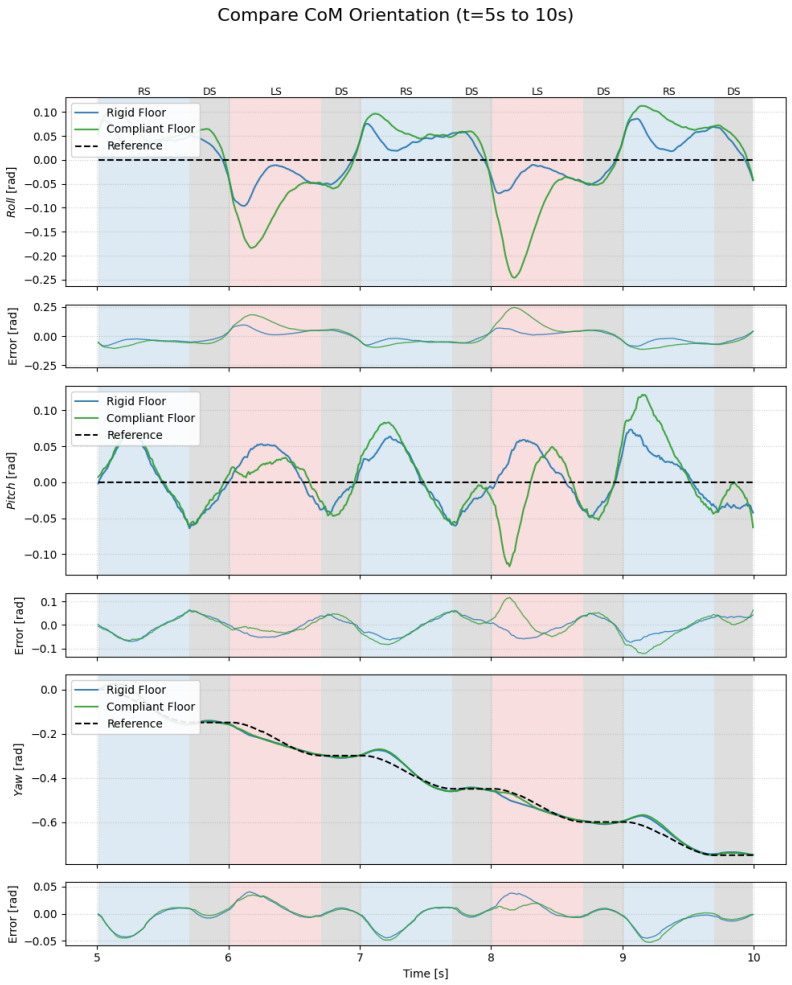

Figure 4.25 CoM orientation tracking comparison

Consistent with the orientation tracking results, the Center of Mass (CoM) position tracking illustrated in Figure 4.19 confirms that the control framework maintains high tracking accuracy across both floor conditions.

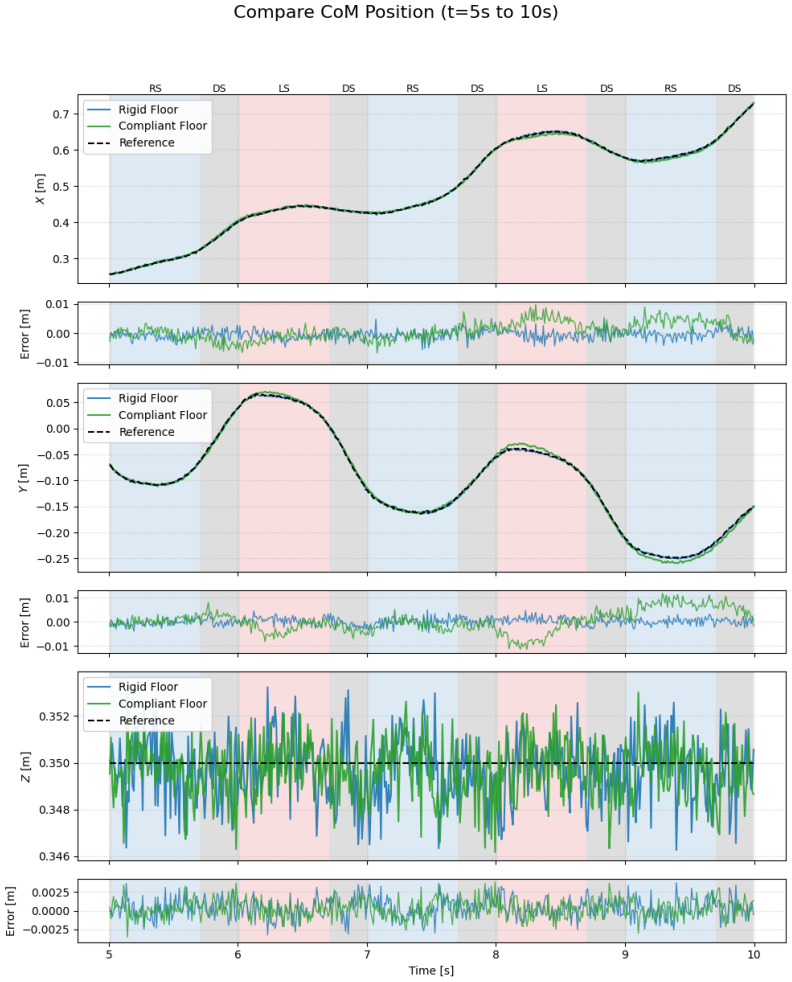

Figure 4.26 CoM position tracking comparison

As previously noted, a compliant surface induces both ground penetration and rotational displacement of the stance leg. This phenomenon is illustrated in Figure 4.27; during the left-stance phase (indicated by the red-shaded region), the foot orientation remains at 0 rad on a rigid surface but exhibits significant rotation about the x and y axes on a compliant floor. By employing the stance leg stabilization controller, the foot orientation is effectively restored to the desired 0 rad for both roll and pitch, while unwanted oscillations are successfully suppressed.

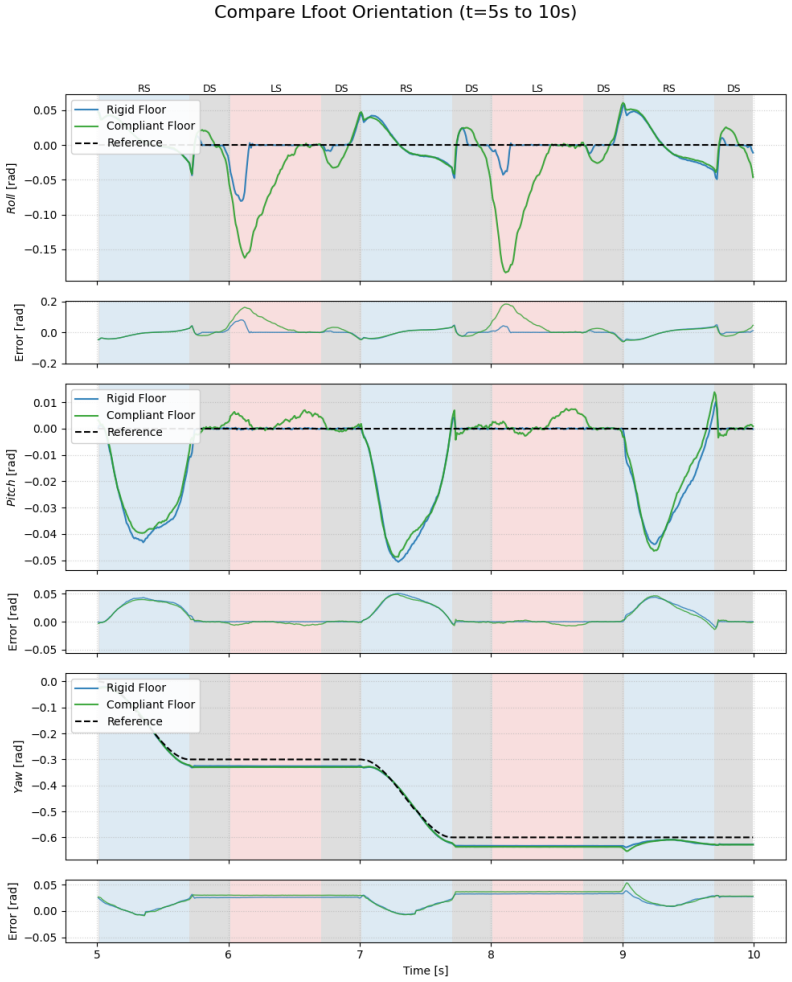

Figure 4.27 Left foot orienation tracking comparison

Regarding foot position tracking, surface compliance primarily affects the vertical (z-axis) component, as illustrated in Figure 4.28. The foot penetration into the floor results in a marginal increase in the relative CoM height; however, this does not adversely impact the gait planner. The planning module remains robust and functional provided that the vertical CoM position error remains within a 4 cm threshold.

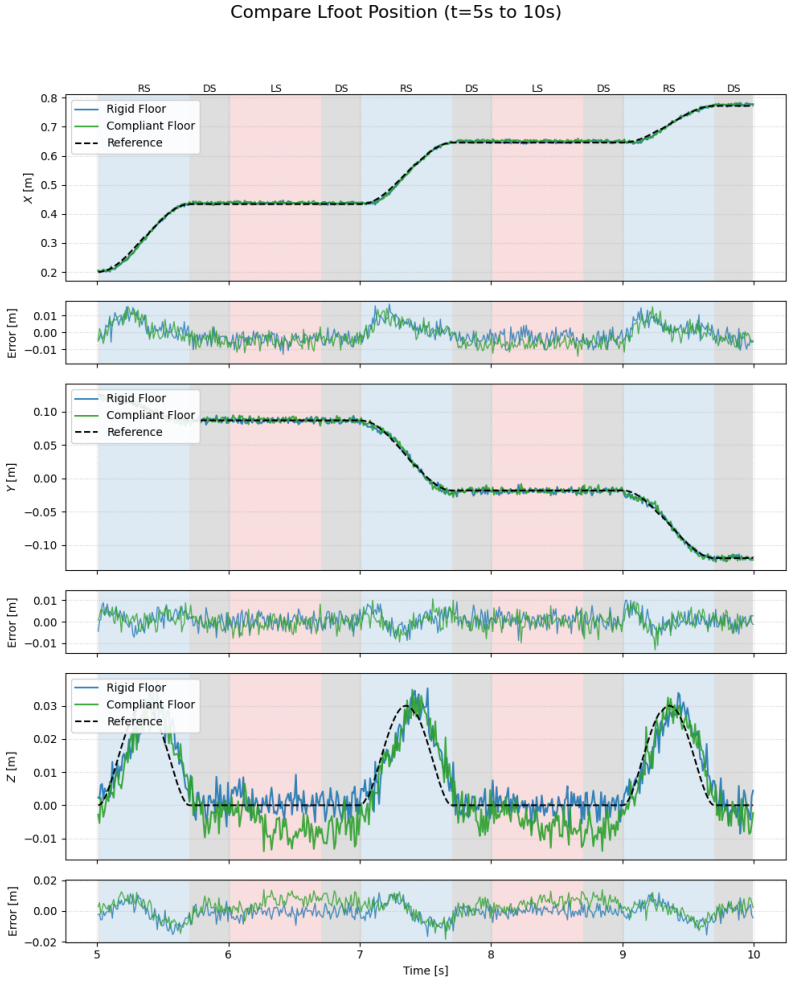

Figure 4.28 Left foot position tracking comparison

3. **Experiment 4.3**: Walking on Multi-property Floor

   This experiment investigates locomotion across surfaces with heterogeneous physical properties—varying stiffness and dissipation—as well as rigid stepped terrain with multiple heights. In both scenarios, the surface characteristics and step heights were unknown to the control system a priori. A straight-line walking trajectory with a constant forward velocity of $v_x = 0.1$ m/s was employed for both tests.

- **Experiment 4.3a**: Multi-properties floor
  The floor parameters utilized in this experiment are detailed in Table 4.7. Due to constraints within the simulation environment, the evaluation was restricted to straight-line locomotion; specifically, the simulator precludes the simultaneous interaction with heterogeneous surface properties within a single contact region.

Table 4.7 Material Properties by Tile Color

Tile colors	Stiffness	Dissipation
Orange	5.00E+06	50
Green	4.00E+03	30
Yellow	5.00E+03	50
Blue	2.00E+04	10

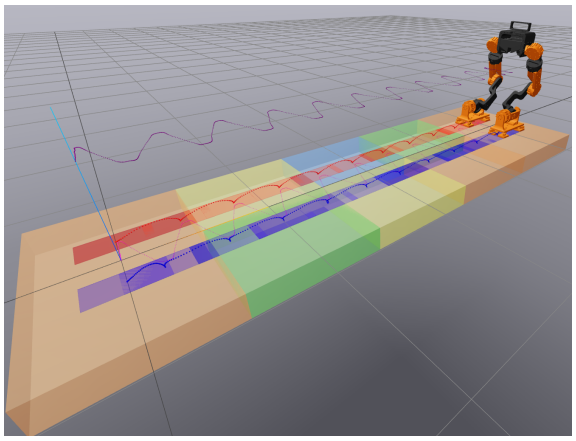

Figure 4.29 The robot walking on muti-properties floor, which the difference tile colors represent diference properties

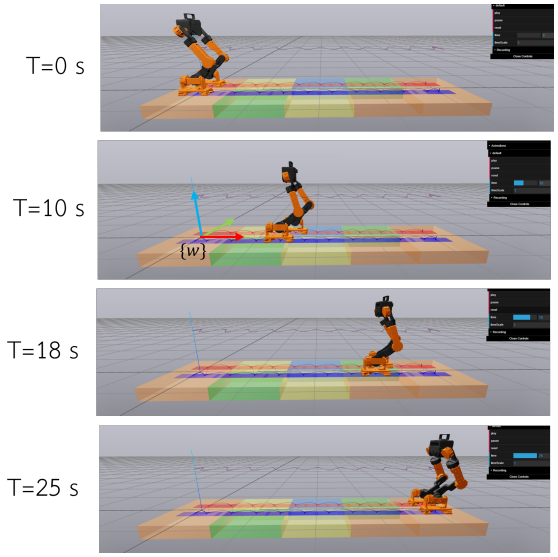

Figure 4.30 Snapshot when walking on muti-properties floor.

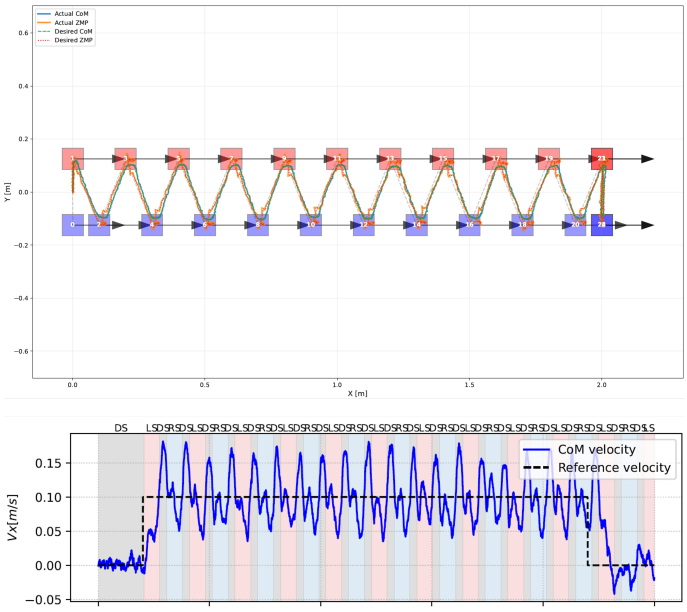

Figure 4.31 CoM, ZMP and reference velocity tracking

- **Experiment 4.3b**: Step floor

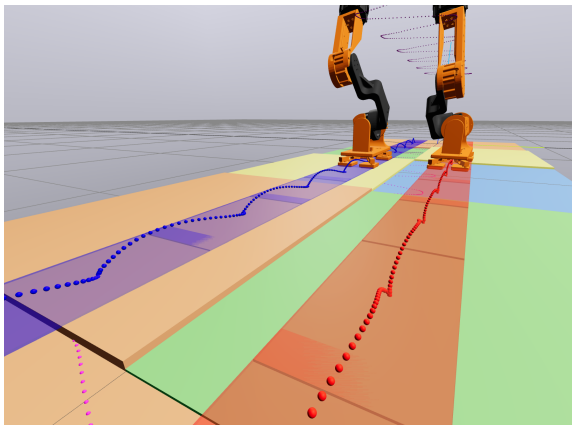

Figure 4.32 The robot walking on step floor, which the difference tile's color represent diference heigh (the maximum height difference is 1 cm)

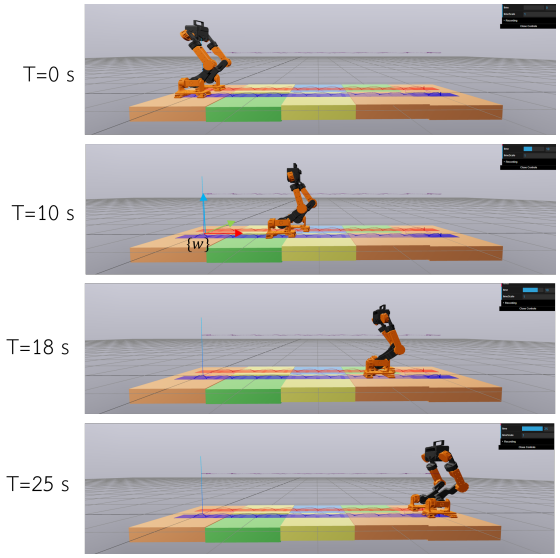

Figure 4.33 Snapshot when walking on step floor.

newpage

Building on the results of the two aforementioned experiments, the proposed framework demonstrates a robust capability for locomotion across multi-terrain environments without prior surface information. Experiment 5.3a illustrates that the stance leg controller effectively manages varying surface properties; this robustness is maintained even during the double-support phase, where each foot may simultaneously interact with a different surface type. Furthermore, Experiment 5.3b confirms that the gait regulation module can successfully negotiate unknown stepped terrain with varying heights.

4. **Experiment 4.4**: Walking with Small Disturbance

   In this experiment, external disturbances were applied to the robot's base, following a procedure analogous to Experiment 3.1. The robustness of the control framework was evaluated under two distinct perturbation scenarios: a longitudinal force $\vec{F}_{ext,1} = [12, 0, 0]^T$ N and a lateral force $\vec{F}_{ext,2} = [0, -12, 0]^T$ N. Both disturbances were introduced while the robot maintained a straight walking.

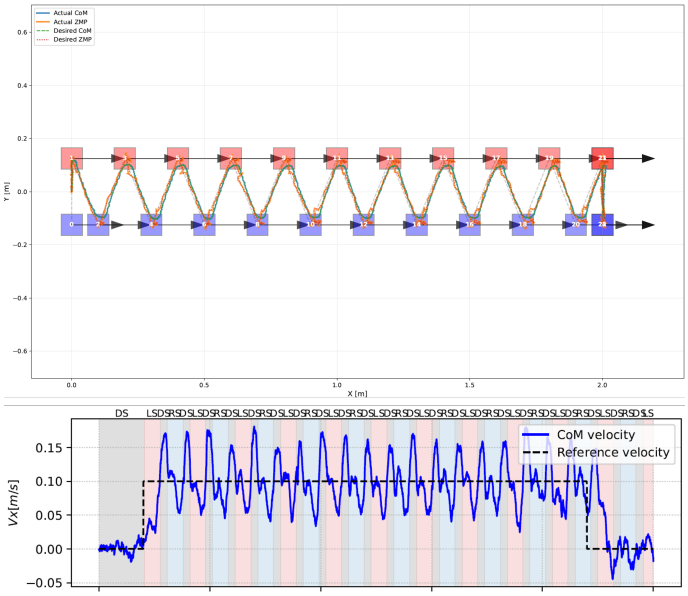

Figure 4.34 CoM, ZMP and reference velocity tracking

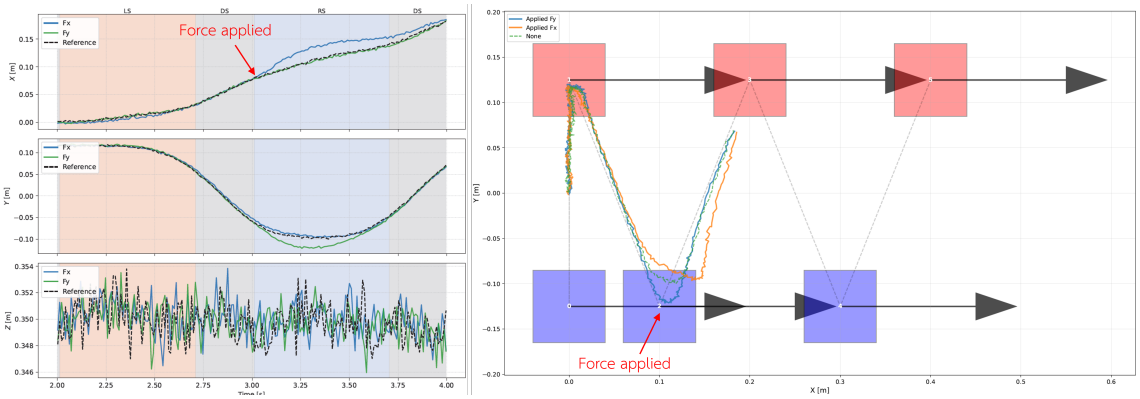

Figure 4.35 CoM trajectory when an external force is applied. The dashed line is original trajectory if no external forces allpied.

In contrast to Experiment 3.1, where the Whole-Body Control (WBC) module acts as a regulator to restore the CoM to a static pose, the framework during locomotion utilizes the Model Predictive Control (MPC) to respond to disturbances through trajectory re-planning. As demonstrated in Figure 4.36, external forces induce a shift in the ZMP position in the direction of the perturbation. The MPC attempts to mitigate this by generating a new CoM trajectory that restores the ZMP to the center of the support polygon.

However, the current implementation is subject to significant control constraints: both the footstep locations and the gait timing are unadjustable. Consequently, the framework's ability to reject disturbances is strictly limited to modulating the ZMP within the boundaries of the pre-planned support polygon. While this approach is effective for small disturbances or short-duration impacts, it cannot accommodate larger perturbations that drive the ZMP to the edge of the support area. To improve robustness against high-magnitude external forces, future work will involve integrating footstep adaptation and adjustable phase durations into the MPC cost function, as discussed in [11].

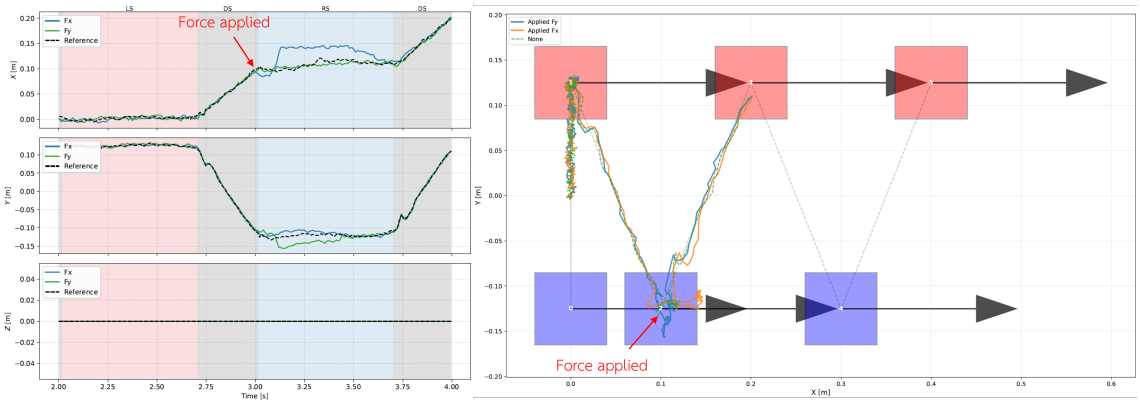

Figure 4.36 ZMP trajectory when an external force is applied. The dashed line is original trajectory if no external forces allpied.

4.2 Experiments in Physical Robot

4.2.1 Experiment 5: Pure Gravity compensation

As demonstrated in Figure 4.37, upon the removal of external support, the robot successfully maintains a self-supporting standing posture (holding its current position). This result validates that the proposed method enables the robot to stand without active control intervention. Furthermore, Figure 4.38 illustrates the wrench distribution characteristics, which exhibit behavior consistent with the simulation results presented in Figure 4.9. Consequently, this experiment serves as a verification of the inertial parameters—specifically mass and inertia—defined within the robot's URDF model.

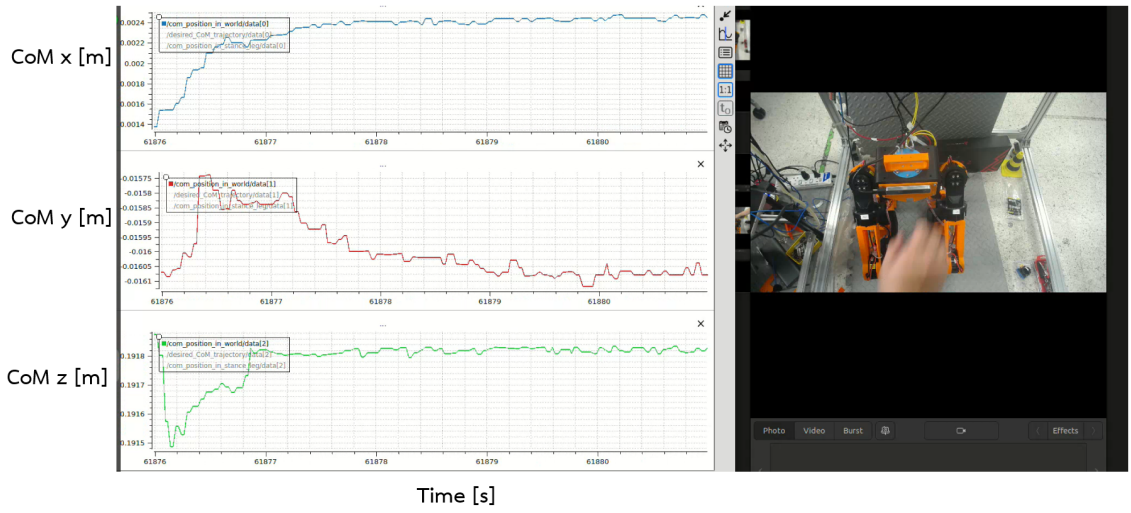

Figure 4.37 Pure gravity compensation effect in physical robot

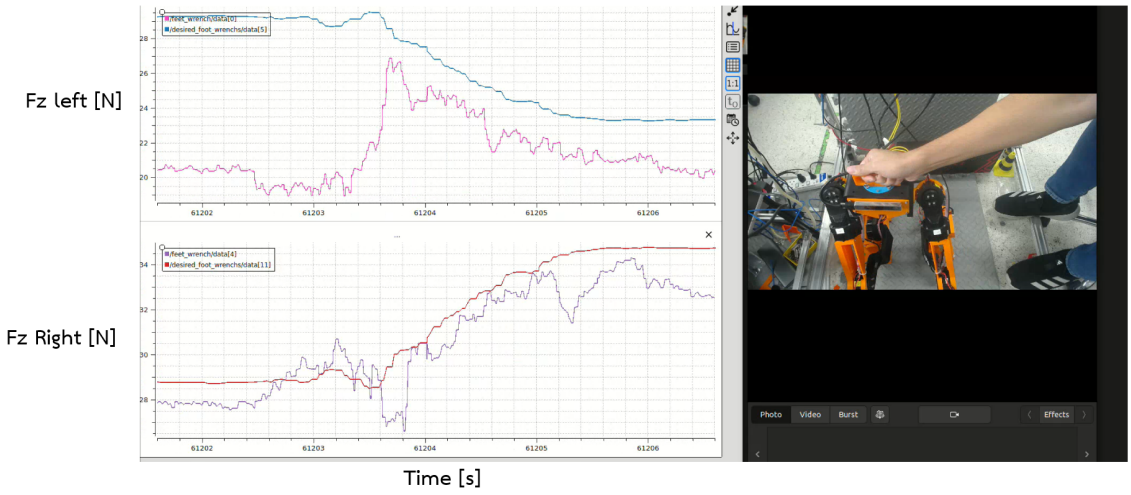

Figure 4.38 Wrench distribution in physical robot

CHAPTER 5 DISCUSSION AND CONCLUSION

5.1 Discussion

The experimental results validate the efficacy of the proposed hierarchical control structure in achieving stable humanoid walking. A primary finding is the critical role of the stance leg orientation controller in maintaining stability on non-rigid surfaces. On compliant floors, the absence of this controller led to roll errors of up to 0.3 rad; however, its integration reduced tracking errors to below 0.1 rad, effectively neutralizing foot oscillations and sinking.

Comparative analysis between rigid and compliant floor locomotion revealed that while surface compliance induces larger orientation tracking errors during single-support phases, the CoM orientation controller successfully ensures the base attitude remains aligned with the reference trajectory. The framework's robustness was further highlighted in Experiment 4.3, where the robot successfully traversed heterogeneous floors (varying stiffness and dissipation) and unknown stepped terrain without prior information. The ability of the WBC to manage different surface properties simultaneously for each foot during double-support phases is a significant advancement for real-world robustness.

However, the disturbance rejection experiments exposed a notable limitation: the current system's reliance on pre-planned footstep locations and gait timing. While the MPC effectively modulates the ZMP to reject small perturbations, it cannot compensate for larger forces that drive the ZMP to the edge of the support polygon. This underscores the necessity for future iterations to include online footstep adaptation to enhance recovery from high-magnitude external impacts.

5.1.1 Current Limitations and Implementation Challenges

Although the proposed framework has been successfully validated in a simulation environment, physical deployment—specifically sim-to-real transfer—remains an active area of development. While the static calibration experiments verified the accuracy of the inertial parameters (mass and inertia) defined in the URDF using the standalone WBC module, real-time closed-loop implementation of the complete framework on the physical humanoid platform is currently constrained by the following technical challenges:

- **Sensor Feedback and Communication Latency:**
  The current hardware architecture limits motor feedback loop execution to 300 Hz. This bandwidth restriction stems from the bus topology, which routes three motor actuators through a single CAN adapter (a 3:1 ratio). Achieving the target 1 kHz feedback rate necessitates transitioning to a dedicated 1:1 adapter-to-motor ratio; however, this solution requires a structural redesign of the robot chassis to house additional USB-to-CAN interfaces, alongside increased hardware costs. Furthermore, the external camera system utilized for floating-base state estimation is capped at a 200 Hz frame rate. Because accurate base attitude tracking is critical for the stability of the WBC layer, a sensor fusion framework incorporating a high-frequency Inertial Measurement Unit (IMU) must be implemented to bridge this latency gap.
- **Real-Time Computational Constraints:**
  Systems integration and hardware interfacing (actuators, cameras, and load cells) are managed via the ROS 2 (Humble) ecosystem. However, the serialization overhead inherent to the ROS 2 publish/subscribe architecture introduces non-deterministic

communication latencies that inflate the overall loop time. Migrating the core Python architecture into a native C++ codebase is planned to reduce computational overhead, though this optimization is secondary to resolving the raw sensor feedback bottleneck. A second critical computational constraint involves hardware resources: both the Drake dynamics engine and the CasADi optimization stack rely entirely on sequential, CPU-bound computations. While simulations were executed on a high-performance workstation equipped with an Intel Core i9-13980HX processor, the onboard computation unit on the physical robot is restricted to an Intel NUC utilizing an i5-series processor, posing a severe threat to real-time constraint satisfaction during optimization.

- **Sensor Noise and State Estimation:**
  Physical deployment introduces significant high-frequency measurement noise that is absent in ideal simulations. This noise destabilizes the damping terms within the joint-space impedance controllers, inducing unmodeled physical oscillations that degrade tracking performance. While a Kalman-filter-based state estimator has been developed and validated in simulation to mitigate this noise, its deployment on the physical platform remains a future milestone. Furthermore, evaluating tracking accuracy in the real world is severely limited by the lack of absolute ground-truth pose data (such as an external motion-capture system), which complicates benchmarking and experimental debugging relative to the simulation baseline.

5.2 Conclusion

This thesis has presented a robust hierarchical locomotion framework for the VOLTA humanoid robot, designed to navigate diverse and challenging terrains. By integrating an online Intrinsically Stable Model Predictive Control (IS-MPC) for gait planning with a passivity-based Whole-Body Control (WBC) for task-space execution, the system achieves stable, omnidirectional walking. A critical contribution of this work is the development of a stance leg orientation controller, which significantly enhances stability on compliant and non-rigid surfaces—environments that typically pose substantial risks to humanoid balance.

The experimental results, conducted within the Drake simulation environment and validated through initial hardware tests, demonstrate that the proposed framework effectively manages multi-property terrains and unknown stepped surfaces. The WBC's ability to distribute contact wrenches optimally allows the robot to maintain its Center of Mass (CoM) trajectory even when surface properties vary between the left and right support feet. While the current implementation successfully handles moderate external disturbances, the fixed footstep timing and placement remain the primary bottlenecks for higher-level robustness.

5.2.1 Future Work

To further advance the capabilities of the VOLTA platform, several key areas have been identified for future development:

- **Online Footstep Adaptation:** Future iterations will incorporate footstep modification and timing adjustment directly into the MPC formulation. This will allow the robot to react to large external perturbations by dynamically changing its base of support.

- **Hardware Optimization:** Increasing the control loop frequency beyond 100 Hz is essential for improving the responsiveness of the torque-controlled actuators on the physical robot.
- **Autonomous Navigation:** Implementing high-level path planning and obstacle avoidance algorithms will allow VOLTA to operate autonomously in complex, unstructured environments.

REFERENCES

[1] Wieber, P.-B., Tedrake, R., and Kuindersma, S., 2016, "Modeling and Control of Legged Robots", in **Springer Handbook of Robotics**, pp. 1203-1234.

[2] Pratt, G. and Manzo, J., 2013, "The DARPA Robotics Challenge [Competitions]", **IEEE Robotics & Automation Magazine**, Vol. 20, No. 2, pp. 10-12.

[3] Imamura, Y., Ayusawa, K., Yoshida, E., and Tanaka, T., 2017, "Evaluation Framework for Passive Assistive Device Based on Humanoid Experiments", **International Journal of Humanoid Robotics**, Vol. 15, p. 1750026.

[4] Atkeson, C. G., Hale, J. G., Pollick, F., Riley, M., Kotosaka, S., Schaul, S., Shibata, T., Tevatia, G., Ude, A., Vijayakumar, S., Kawato, E., and Kawato, M., 2000, "Using humanoid robots to study human behavior", **IEEE Intelligent Systems and their Applications**, Vol. 15, No. 4, pp. 46-56.

[5] Reher, J. and Ames, A. D., 2021, "Dynamic Walking: Toward Agile and Efficient Bipedal Robots", **Annual Review of Control, Robotics, and Autonomous Systems**, Vol. 4, pp. 535-572.

[6] Mikolajczyk, T., Mikołajewska, E., Al-Shuka, H., Malinowski, T., Kłodowski, A., Pimenov, D., Paczkowski, T., Hu, F., Giasin, K., Mikołajewski, D., and Macko, M., 2022, "Recent Advances in Bipedal Walking Robots: Review of Gait, Drive, Sensors and Control Systems", **Sensors**, Vol. 22, No. 6, p. 4440.

[7] Katayama, S., Murooka, M., and Tazaki, Y., 2023, "Model predictive control of legged and humanoid robots: models and algorithms", **Advanced Robotics**, Vol. 37, pp. 1-18.

[8] Dantec, E., Naveau, M., Fernbach, P., Villa, N., Saurel, G., Stasse, O., Taix, M., and Mansard, N., 2022, "Whole-Body Model Predictive Control for Biped Locomotion on a Torque-Controlled Humanoid Robot", **2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)**, pp. 638-644.

[9] Scianca, N., Cognetti, M., De Simone, D., Lanari, L., and Oriolo, G., 2016, "Intrinsically stable MPC for humanoid gait generation", **2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)**, pp. 601-606.

[10] Zamparelli, A., Scianca, N., Lanari, L., and Oriolo, G., 2018, "Humanoid Gait Generation on Uneven Ground using Intrinsically Stable MPC", **IFAC-PapersOnLine**, Vol. 51, pp. 393-398.

[11] Scianca, N., De Simone, D., Lanari, L., and Oriolo, G., 2020, "MPC for Humanoid Gait Generation: Stability and Feasibility", **IEEE Transactions on Robotics**, Vol. 36, No. 4, pp. 1171-1188.

[12] Smaldone, F. M., Scianca, N., Lanari, L., and Oriolo, G., 2022, "From Walking to Running: 3D Humanoid Gait Generation via MPC", **Frontiers in Robotics and AI**, Vol. 9.

[13] Habib, A. S., Smaldone, F. M., Scianca, N., Lanari, L., and Oriolo, G., 2022, "Handling Non-Convex Constraints in MPC-Based Humanoid Gait Generation", **2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)**, pp. 13167-13173.

[14] Caron, S. and Kheddar, A., 2017, "Dynamic Walking over Rough Terrains by Nonlinear Predictive Control of the Floating-base Inverted Pendulum", **IROS: Intelligent Robots and Systems**, Sep. 2017, Vancouver, Canada, pp. 5017-5024.

[15] Ding, J., Han, L., Ge, L., Liu, Y., and Pang, J., 2022, "Robust Locomotion Exploiting Multiple Balance Strategies: An Observer-Based Cascaded Model Predictive Control Approach", **IEEE/ASME Transactions on Mechatronics**, Vol. 27, No. 4, pp. 2089-2097.

[16] Mesesan, G., Englsberger, J., Garofalo, G., Ott, C., and Albu-Schäffer, A., 2019, "Dynamic Walking on Compliant and Uneven Terrain using DCM and Passivity-based Whole-body Control", **2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids)**, pp. 25-32.

[17] Koolen, T., de Boer, T., Rebula, J., Goswami, A., and Pratt, J., 2012, "Capturability-based analysis and control of legged locomotion, Part 1: Theory and application to three simple gait models", **The International Journal of Robotics Research**, Vol. 31, No. 9, pp. 1094-1113.

[18] Kajita, S., Kanehiro, F., Kaneko, K., Yokoi, K., and Hirukawa, H., 2001, "The 3D linear inverted pendulum mode: a simple modeling for a biped walking pattern generation", **Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the Next Millennium (Cat. No.01CH37180)**, Vol. 1, pp. 239-246.

[19] MathWorks, 2013, Understanding Model Predictive Control [Online], Available: https://www.mathworks.com/videos/understanding-model-predictive-control-part-1-97131.html [Accessed: 25 January 2025].

[20] Moro, Federico & Sentis, Luis.,2019, Whole-Body Control of Humanoid Robots. In: Goswami, A., Vadakkepat, P. (eds) **Humanoid Robotics: A Reference**. Springer, Dordrecht. pp.1161-1183.

[21] Tedrake, R. and the Drake Development Team, 2019, Drake: Model-based design and verification for robotics [Online], Available: https://drake.mit.edu [Accessed: 3 February 2025].

[22] DIAG Robotics Lab, 2022, ismpc [Online], Available: https://github.com/DIAG-Robotics-Lab/ismpc [Accessed: 11 March 2026].

CURRICULUM VITAE

NAME	Mr. Tanawat Pawanta
DATE OF BIRTH	29 September 2002
EDUCATIONAL RECORD	
HIGH SCHOOL	High School Graduation Suankularb Wittayalai, 2020
BACHELOR'S DEGREE	Bachelor of Engineering (Robotics and Automation Engineering) King Mongkut's University of Technology Thonburi, 2024
MASTER'S DEGREE	Master of Engineering (Robotics and Automation) King Mongkut's University of Technology Thonburi, 2025
EMPLOYMENT RECORD	Research Engineer Intern Famme works(Thailand) co.,ltd., 2024
PUBLICATION	T. Pawanta, E. Pengwang, S. Vongbunyong, and K. Thamrongaphichartkul, 2026, "A Unified Torque-Controlled Locomotion Framework for the VOLTA Humanoid Robot via IS-MPC and Passivity-Based WBC", **Proceedings of the 9th International Conference on Intelligent Robotics and Control Engineering (IRCE)**, 3–6 August 2026, Urumqi, China, pp. 114–119.